

naRedeMundial Corporation

PHP - ASP - JSP

Desenvolvendo Websites Dinâmicos

Cerli Antônio da Rocha

2007

Quarta Capa

As tecnologias da web estão dando um nó em sua cabeça? Resolva este problema, agora, com este livro.

Com uma abordagem direta e prática, apresenta as três linguagens mais usadas na web: PHP, ASP e JSP, com seus recursos mais importantes, incluindo:

- Comandos e Funções mais usados em PHP(Personal Home Pages ou Hypertext Preprocessor);
- Comandos e Funções mais usados em ASP(Active Server Pages);
- Comandos e Funções mais usados em JSP(Java Server Pages);
- Dicas de bons hábitos de programação;
- Dicas de instalação dos respectivos ambientes;
- Aplicativo de exemplo completo nas 3 linguagens de programação;
- Site com todos os códigos de exemplo usados neste livro.

Este livro é, sem dúvida, a forma mais rápida de você conhecer e entender, as principais tecnologias dominantes na web, de uma só vez, em um único lugar!

[www.naredemundial.com.br/livro]

PHP ASP JSP

Autor: Cerli Antônio da Rocha
cerli@naredemundial.com.br

Julho de 2003

Sumário

Sumário.....	3
Apresentação.....	9
1 - Bons Hábitos de Programação.....	11
1 - Inicialização de Variáveis.....	11
2 - Documentação.....	11
3 - Nomes Claros.....	14
4 - Uso de Funções e Classes.....	14
5 - Arquivos Pequenos.....	14
6 - Separação HTML/Código.....	15
7 - Diagramas.....	15
2 - Comparação	
PHP-ASP-JSP.....	16
3 - A Melhor Linguagem.....	17
PHP	
Personal Home pages	18
1 - Introdução.....	19
1 - O que é?.....	19
2 - O que pode ser feito?.....	19
3 - Como surgiu?.....	20
4 - Diagrama.....	22
2 - Sintaxe Básica.....	23
1 - Delimitando o código.....	23
2 - Separando instruções.....	23
3 - Nomes de variáveis.....	23
4 - Comentários.....	24
5 - Imprimindo código html.....	25
3 - Tipos.....	26
1 - Tipos Suportados.....	26
2 - Transformação de Tipos.....	28
4 - Operadores.....	31
1 - Aritméticos.....	31
2 - Strings.....	31
3 - Atribuição.....	31
4 - Bit a bit.....	31
5 - Lógicos.....	32
6 - Comparação.....	32
7 - Expressão Condicional.....	32
8 - Incremento e Decremento.....	32
5 - Estruturas de Controle.....	33
1 - Blocos.....	33
2 - Condicionais.....	33
3 - Repetição.....	35
6 - Funções.....	37
1 - Definição.....	37

2 - Retorno.....	37
3 - Argumentos.....	37
4 - Contexto.....	39
5 - Escopo.....	39
7 - Variáveis e Constantes.....	41
1 - Declaração.....	41
2 - O modificador static.....	41
3 - Variáveis variáveis.....	42
4 - Variáveis enviadas pelo navegador.....	42
5 - Variáveis de Ambiente.....	43
6 - Verificando o tipo de uma variável.....	43
7 - Destrindo uma variável.....	44
8 - Verificando se uma variável possui um valor.....	44
9 - Constantes.....	45
8 - Classes & Objetos.....	46
1 - Classes.....	46
2 - Objetos.....	46
3 - A variável \$this.....	46
4 - SubClasses.....	47
5 - Construtores.....	48
9 - Bancos de Dados.....	49
1 - MySQL.....	49
2 - Microsoft SQL Server(MSSQL).....	51
3 - Fontes de Dados ODBC.....	53
10 - Headers.....	56
11 - Cookies.....	57
1 - Criando.....	57
2 - Recuperando informações.....	57
12 - Manipulação de Arquivos.....	59
1 - Copiando.....	59
2 - Verificando tamanho.....	59
3 - Verificando se existe.....	59
4 - Limpando o cache.....	59
5 - Abrindo arquivos.....	60
6 - Fechando Arquivos.....	61
7 - Lendo arquivos.....	61
8 - Escrevendo em arquivos.....	62
13 - Email.....	63
14 - PHP & Java.....	65
15 - Sessions.....	66
1 - O Objeto Session	66
2 - Quando a sessão começa?.....	66
3 - Quando a sessão termina?.....	66
4 - Gravando e Recuperando Variáveis de Sessão	67
16 - Includes/Requires.....	68
17 - Tratamento de Erros.....	69
1 - Erros de Função.....	69
2 - Erros Globais.....	69
3 - Die/Exit.....	70
ASP	
Active Server Pages.....	71

1 - Introdução.....	72
1 - O que é?.....	72
2 - O que pode ser feito?.....	72
3 - Como Surgiu?.....	72
4 - Diagrama.....	72
2 - Sintaxe básica.....	74
1 - Delimitando o código.....	74
2 - Separando Instruções.....	74
3 - Nomes de variáveis.....	74
4 - Comentários.....	74
5 - Imprimindo código HTML.....	74
3 - Tipos.....	76
1 - Tipos Suportados.....	76
2 - Transformação de Tipos.....	77
4 - Operadores.....	79
1 - Aritméticos.....	79
2 - Strings.....	79
3 - Atribuição.....	79
4 - Lógicos.....	79
5 - Comparação.....	79
6 - Incremento e Decremento.....	79
5 - Estruturas de Controle.....	80
1 - Blocos.....	80
2 - Condicionais.....	80
3 - Repetição.....	81
6 - Funções.....	83
1 - Definição.....	83
2 - Retorno.....	83
3 - Argumentos.....	84
4 - Contexto.....	85
5 - Escopo.....	85
7 - Variáveis e Constantes.....	87
1 - Declaração.....	87
2 - Variáveis enviadas pelo Navegador.....	87
3 - Verificando o Tipo.....	88
4 - Destruindo.....	89
5 - Definindo Constantes.....	90
8 - Classes & Objetos	91
1 - Classes.....	91
2 - Objetos.....	91
3 - Objetos Implícitos.....	92
9 - Bancos de Dados.....	98
1 - MySQL.....	98
2 - Microsoft SQL Server.....	98
10 - Cookies.....	100
1 - Criando.....	100
2 - Recuperando informações.....	100
3 - Cookies com chaves	100
11 - Manipulação de Arquivos.....	102
1 - Copiando.....	102
2 - Verificando Tamanho.....	102
3 - Verificando se existe.....	102
4 - Abrindo Arquivos.....	103

12 - Email.....	104
1 - AspMail.....	104
2 - CDONTS.....	105
13 - ASP & Java.....	106
14 - Sessions.....	107
1 - O Objeto Session	107
2 - Quando a sessão começa?.....	107
3 - Quando a sessão termina?.....	107
4 - Gravando e Recuperando Variáveis de Sessão	108
5 - Removendo Variáveis de Sessão	108
6 - Acessando toda coleção Contents.....	109
7 - Acessando a coleção StaticObjects.....	109
15 - Includes.....	111
16 - Tratamento de Erros.....	112
1 - ERR.....	112
2 - ASPError.....	113
17 - Global.asa.....	115
1 - Eventos.....	115

JSP

Java Server pages.....	117
1 - Introdução.....	118
1 - O que é?.....	118
2 - O que pode ser feito?.....	118
4 - Diagrama.....	118
2 - Sintaxe Básica.....	120
1 - Delimitando o código.....	120
2 - Separando Instruções.....	120
3 - Nomes de Variáveis.....	120
4 - Comentários.....	121
5 - Imprimindo código HTML.....	122
3 - Tipos.....	123
1 - Tipos Suportados.....	123
4 - Operadores.....	127
1 - Aritméticos.....	127
2 - Strings.....	127
3 - Atribuição.....	127
4 - Lógicos.....	127
5 - Comparação.....	127
6 - Expressão Condicional.....	127
7 - Incremento e Decremento.....	127
5 - Estruturas de Controle.....	128
1 - Blocos.....	128
2 - Condicionais.....	128
3 - Repetição.....	129
6 - Funções.....	131
1 - Definição.....	131
2 - Retorno.....	131
2 - Argumentos.....	131
3 - Escopo.....	131
7 - Variáveis e Constantes.....	132

1 - Declaração.....	132
2 - Variáveis enviadas pelo Navegador.....	132
3 - Variáveis de Ambiente.....	133
4 - Definindo Constantes.....	133
8 - Classes e Objetos.....	134
1 - Classes.....	134
2 - this.....	136
3 - SubClasses.....	137
6 - Construtores.....	137
9 - Bancos de Dados.....	139
1 - MySQL.....	139
2 - ODBC.....	139
10 - Headers.....	141
11 - Cookies.....	142
1 - Criando.....	142
2 - Recuperando informações.....	143
12 - Manipulação de Arquivos.....	146
1 - Copiando.....	146
2 - Verificando Tamanho.....	146
3 - Verificando se existe.....	146
4 - Abrindo Arquivos.....	146
13 - Email.....	148
14 - JSP & Java(Servlets e Beans).....	149
1 - A Classe.....	149
2 - O JSP.....	150
3 - O HTML.....	150
15 - Sessions.....	151
1 - Métodos das Sessões.....	151
2 - Como invalidar a Sessão.....	152
16 - Includes.....	153
17 - Tratamento de Erros.....	154
1 - Diretiva Page.....	154
2 - Exceptions.....	154
<i>Aplicação de Exemplo.....</i>	<i>157</i>
Agenda.....	158
1 - Arquivo Comum.....	158
2 - Menu.....	158
3 - Funções.....	159
4 - Index.....	161
5 - Login.....	164
6 - Listar.....	166
7 - Inserir.....	172
8 - Inserir Agenda.....	176
9 - Buscar.....	179
10 - Search.....	182
11 - Editar.....	188
12 - Atualizar Agenda.....	194
13 - Detalhes.....	198
14 - Excluir.....	204
<i>Apêndices.....</i>	<i>208</i>

1 - Manipulação de Strings.....	209
1 – PHP.....	209
2 – ASP.....	213
3 - JSP.....	215
2 - Manipulação de Números.....	218
1 – PHP.....	218
2 – ASP.....	222
3 – JSP.....	223
3 - Manipulação de Data e Hora.....	224
1 – PHP.....	224
2 – ASP.....	224
4 - Manipulação de Arrays.....	226
1 – PHP.....	226
5 - Templates.....	230
6 - Instalação dos Ambientes.....	235
1 - Ambiente Java/PHP.....	235
2 - Ambiente PHP no Windows.....	238
3 - Ambiente PHP no Linux.....	242
7 - Instalação e Configuração do Ambiente ODBC.....	245

Apresentação

O livro traça o panorama das principais linguagens de programação usadas para oferecer dinamismo de informações nos sites da web atuais.

As tecnologias PHP, ASP e JSP possuem cada uma, com sua particularidade, suas vantagens, desvantagens e ambientes específicos.

Criada durante o outono de 1994 por **Rasmus Lerdorf**, o PHP (Personal Home Pages) é uma linguagem de script Open Source de uso geral, utilizada especialmente para o desenvolvimento de aplicações Web com código embutido dentro do próprio HTML, além de rodar em plataformas como Windows e Linux e com sintaxe muito semelhante à linguagem C.

O ASP(MicrosoftTM Active Server Pages) é uma linguagem de script usada para criar aplicações dinâmicas na web em Ambientes Microsoft e podendo, também ser embutido em meio ao próprio HTML.

O JSP(Java Server Pages), é a linguagem de script da Sun Microsystems, que permite o rápido desenvolvimento de sites dinâmicos. O JSP faz parte da Família JavaTM, e com isso, é independente de plataforma e facilita a separação das camadas de design e programação, sendo também é semelhante à linguagem C, porém possui várias características diferentes do PHP.

Aos iniciantes este livro oferece subsídios para a aprendizagem clara e objetiva das três tecnologias dominantes no mercado, bem como à escolha de sua linguagem preferida.

Aos programadores mais experientes o livro apresenta uma ótima fonte de referência com riqueza de detalhes e sugestões de bons hábitos de programação.

Com estes temas abordados, o livro pretende atingir seu objetivo maior, que é se tornar veículo de convergência dos recursos possíveis com as tecnologias PHP, ASP e JSP, além de ser ferramenta para que novos recursos possam ser criados.

1 - Bons Hábitos de Programação

Independente da linguagem que utilizar, é muito importante que utilizemos técnicas ou hábitos de programação que torne nosso código-fonte fácil de ser lido e entendido por outra pessoa e por nós mesmos.

Os principais hábitos que cultivo em meus códigos são: inicialização de variáveis, documentação através de comentários no próprio código-fonte, nomes claros, uso intensivo de funções e classes, arquivos com poucas linhas, separação de HTML e código e ter sempre o Diagrama das Bases de Dados.

1 - Inicialização de Variáveis

Mesmo em linguagens como PHP e ASP, em que não é obrigatório a declaração de variáveis, deve-se fazer uso deste hábito para que possamos ter um melhor controle dos tipos de dados que estão sendo armazenados nas variáveis e também para debugar os scripts.

2 - Documentação

Para resolver certos problemas, as vezes criamos algoritmos tão complexos que nem nós mesmos os entenderíamos depois de algum tempo.

Nomes de variáveis, também são um bom motivo para documentação dos códigos, imagine daqui um ano você lembrar pra que serve uma variável chamada "xd45" em um script que você precisa dar manutenção urgente.

Para esses problemas eu defendo a documentação em todo código, que para nossa alegria tanto o PHP, quanto o ASP ou o JSP possuem comandos para inserir esses comentários no código.

A documentação se divide em três partes: a documentação de identificação do script, a identificação de variáveis, constantes e funções, e identificação de ações.

No caso de classes(PHP, ASP e JSP) é aconselhável que logo abaixo a identificação de script seja documentado como usar seus recursos desta.

1 - Identificação do Script

No início do script, nas primeiras linhas é importante identificar o script com o nome do desenvolvedor, função do script, data de criação e atualização.

Em PHP seria assim:

```
<?
/*
Função:          Montar a página inicial do site
Desenvolvedor:   Cerli Antônio da Rocha
Data:            30/05/2003
Atualização:     30/06/2003
*/
?>
```

Em ASP seria assim:

```
<%
'Função:          Montar a página inicial do site
'Desenvolvedor:   Cerli Antônio da Rocha
'Data:            30/05/2003
'Atualização:     30/06/2003
%>
```

Em JSP seria assim:

```
<%
/*
Função:          Montar a página inicial do site
Desenvolvedor:   Cerli Antônio da Rocha
Data:            30/05/2003
Atualização:     30/06/2003
*/
%>
```

2 - Identificação de Variáveis, Constantes e Funções

Ao declarar as variáveis, constantes e funções é aconselhável que seja descrito onde será usada.

Em PHP seria assim:

```
<?
$codempresa = "";           //Armazena o código da empresa

function imprime( $nome ) {  //Retorna o nome recebido
    return $nome;
}
?>
```

Em ASP seria assim:

```
<%
Dim codempresa = ""         'Armazena o código da empresa

function imprime( nome )    'Retorna o nome recebido
    imprime = nome
end function
%>
```

EM JSP seria assim:

```
<%
String codempresa = "";     //Armazena o código da empresa

public String imprime( String nome ) {  //Retorna o nome recebido
    return nome;
}
%>
```

3 - Identificação de Ações

A identificação de ações visa deixar claro aqueles algoritmos mais complexos.

Em PHP seria assim:

```
<?
$contador = 0;               //Usada para controlar o loop

while ( $contador <= 10 ) {   //Loop até 10
    if ( ( $contador % 2 ) == 0 ) { //Testa se o contador é par
        print "número par";
    } else {
        print "número impar";
    }
    $contador++;              //Incrementa o contador
}
?>
```

Em ASP seria assim:

```
<%
contador = 0                 'Usada para controlar o loop

while ( contador <= 10 )     'Loop até 10
    if ( ( contador MOD 2 ) = 0 ) 'Testa se o contador é par
        response.write("número par")
```

```
        else
            response.write("número impar")
        end if
        contador = contador + 1           `Incrementa o contador
wend
%>
```

Em JSP Seria assim:

```
<%
int contador = 0;                                //Usada para controlar o loop
while ( contador <= 10 ) {                        //Loop até 10
    if ( ( contador % 2 ) == 0 ) {                //Testa se o contador é par
        out.println("número par");
    } else {
        out.printl("número impar");
    }
    contador++;                                  //Incrementa o contador
}
$>
```

3 - Nomes Claros

O próprio nome de uma variável, constante, função ou classe quando é definido de forma clara já é por si só uma documentação.

Os nomes de variáveis e constantes devem ter nomes que expressem o tipo de informação ou a função a que se destinam.

Uma variável para guardar o nome de uma pessoa, por exemplo, deve se chamar **nome** ou **pessoa** e não **x** ou **y**. Já vi variáveis e funções com nomes assim e até hoje não descobri sua utilidade.

4 - Uso de Funções e Classes

Use sempre funções e classes para códigos que necessitem ser repetidos mais de uma vez, além de tornar o código mais limpo e organizado torna mais fácil sua reutilização.

5 - Arquivos Pequenos

Já me deparei varias vezes com páginas que fazem "submit" de formulários para dentro de si mesmas, sendo que as próprias processam as informações e submetem novamente para si mesmas e fazem outro processamento ainda.

Isso gera um código confuso e de baixa performance, o ideal é criar arquivos distintos para utilizações distintas, ou seja, uma página para o formulário de "login" e uma outra página para autenticar o login e fazer o devido redirecionamento.

6 - Separação HTML/Código

Uma das vantagens que tanto o PHP, quanto o ASP ou o JSP apresenta é justamente a possibilidade de embutir scripts no HTML, mas eu considero isso uma desvantagem, pois também gera um código confuso e de difícil manutenção.

7 - Diagramas

Embora não estejamos abordando aqui, diretamente, o uso de bancos de dados, vale citar que sempre que for usá-los é recomendável desenhar o diagrama das tabelas e seus relacionamentos.

2 - Comparação PHP-ASP-JSP

As principais diferenças entre as linguagens que abordaremos, veremos na tabela a seguir:

	PHP	ASP	JSP
Multi-Plataformas	Sim	Não, somente windows	Sim
Open Source	Sim	Não	Sim
Multiplos Servidores Web	Sim	Não	Sim
Preço	Free	Free	Free
Habilidade para separar a geração do conteúdo e da apresentação.	Sim	Sim	Sim
Linguagens de programação suportadas	PHP	VBScript, JScript	Java, JavaScript
Geração dinâmica de HTML	Sim	Sim	Sim
Escalabilidade desde pequenas até grandes aplicações Web.	Sim	Sim	Sim
Compatibilidade com Banco de Dados legados	Sim	Sim	Sim
Capaz de integrar com diversas fontes de dados	Sim	Sim, através de ODBC	Sim através de ODBC e JDBC
Componentes	COM, Beans e outros	COM	Beans ou Tags

3 - A Melhor Linguagem

Muitos perguntam sobre a melhor linguagem, a mais rápida, a mais eficiente, a que possui os maiores e melhores recursos disponíveis.

Penso que a melhor linguagem é a aquela que melhor atenda a sua necessidade e o importante é dominar o desenvolvimento de bons algoritmos, pois, com isso não importará a linguagem que você poderá desenvolver suas aplicações e atingir seus objetivos.

No mundo de hoje, são poucos os que podem se dar ao luxo de escolher a linguagem em que trabalharão.

PHP

Personal Home pages

1- Introdução

1 - O que é?

PHP é uma linguagem que permite criar sites dinâmicos, possibilitando uma interação com o usuário através de formulários, parâmetros da URL e links.

A diferença de PHP com relação a linguagens semelhantes a Javascript é que o código PHP é executado no servidor, sendo enviado para o cliente apenas html puro.

Desta maneira é possível interagir com bancos de dados e aplicações existentes no servidor, com a vantagem de não expor o código fonte para o cliente.

O que diferencia PHP de um script CGI escrito em C ou Perl é que o código PHP pode ficar embutido no próprio HTML, enquanto no outro caso é necessário que o script CGI obrigatoriamente gere todo o código HTML, ou leia de um outro arquivo.

2 - O que pode ser feito?

Basicamente, qualquer coisa que pode ser feita por algum programa CGI pode ser feita também com PHP, como coletar dados de um formulário, gerar páginas dinamicamente ou enviar e receber *cookies*.

PHP também tem como uma das características mais importantes o suporte a um grande número de bancos de dados, como dBase, Interbase, mSQL, mySQL, Oracle, Sybase, PostgreSQL, MSSQL, Fontes ODBC e vários outros.

Além disso, PHP tem suporte a outros serviços através de protocolos como IMAP, SNMP, NNTP, POP3 e, logicamente, HTTP. Ainda é possível abrir *sockets* e interagir com outros protocolos e também criar arquivos de diferentes formatos como GIF, JPG, FLASH, etc.

3 - Como surgiu?

A linguagem PHP foi concebida durante o outono de 1994 por **Rasmus Lerdorf**. As primeiras versões não foram disponibilizadas, tendo sido utilizadas em sua *home-page* apenas para que ele pudesse ter informações sobre as visitas que estavam sendo feitas.

A primeira versão utilizada por outras pessoas foi disponibilizada em 1995, e ficou conhecida como "**Personal Home Page Tools**" (ferramentas para página pessoal). Esta ferramenta era composta por um sistema bastante simples que interpretava algumas *macros* e alguns utilitários que rodavam "por trás" das *homepages* um livro de visitas, um contador e algumas outras coisas.

Em meados de 1995 o interpretador foi reescrito, e ganhou o nome de **PHP/FI**, o "FI" veio de um outro pacote escrito por Rasmus que interpretava dados de formulários HTML (**F**orm **I**nterpreter). Ele combinou os scripts do pacote *Personal Home Page Tools* com o FI e adicionou suporte a *mSQL*, nascendo assim o PHP/FI, que cresceu bastante, e as pessoas passaram a contribuir com o projeto.

Estima-se que em 1996 PHP/FI estava sendo usado por cerca de 15.000 *sites* pelo mundo, e em meados de 1997 esse número subiu para mais de 50.000. Nessa época houve uma mudança no desenvolvimento do PHP. Ele deixou de ser um projeto de Rasmus com contribuições de outras pessoas para ter uma equipe de desenvolvimento mais organizada.

O interpretador foi reescrito por **Zeev Suraski** e **Andi Gutmans**, e esse novo interpretador foi a base para a versão 3.

O lançamento do PHP4, ocorrido em 22/05/2000, trouxe muitas novidades aos programadores de PHP. Uma das principais foi o suporte a sessões, bastante útil pra identificar o cliente que solicitou determinada informação. Além das mudanças referentes a sintaxe e novos recursos de programação, o PHP4 trouxe como novidade um otimizador chamado Zend, que permite a execução muito mais rápida de scripts PHP.

A empresa que produz o Zend, A Zend Technologies, promete o lançamento de um compilador de PHP. Códigos compilados serão executados mais rapidamente, além de proteger o fonte da aplicação.

Atualmente o PHP está na versão 4.3.3, mas a versão 5.0.0 Beta 1 já foi disponibilizada no site oficial para testes, e promete trazer vários avanços em relação a Orientação a Objetos e tratamento de erros.

As principais novidades, em relação ao novo PHP 5, seriam:

- o O PHP 5 inclui a nova Zend Engine 2. Maiores detalhes sobre a Zend Engine visite o endereço abaixo:

<http://www.php.net/zend-engine-2.php>

- o O suporte a XML foi completamente reestruturado, todas extensões são agora baseadas na excelente libxml2, que você pode saber mais a respeito em:

<http://www.xmlsoft.org/>

- o O SQLite agora é nativo no PHP 5. Maiores detalhes você encontra em:

<http://www.hwaci.com/sw/sqlite/>

- o O tratamento de Streams foi melhorado extremamente, incluindo a habilidade de acesso a sockets de baixo nível em operações de Stream.

Para maiores detalhes e novidades sobre PHP visite sempre o site oficial do PHP:

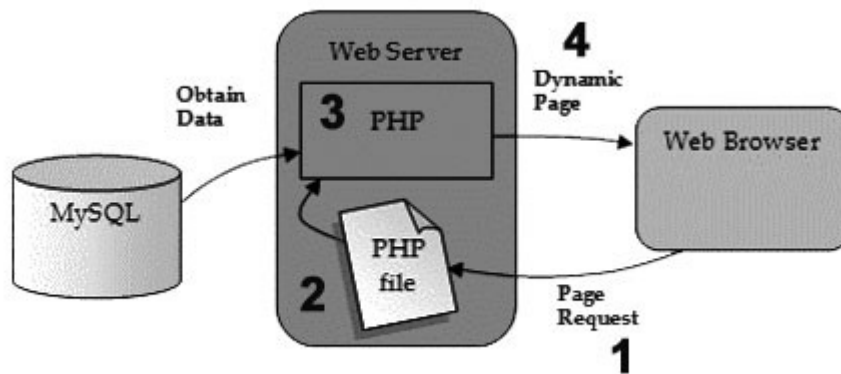
<http://www.php.net>

4 - Diagrama

Uma requisição para o PHP funciona assim:

1. O **Browser** faz uma requisição ao **Servidor Web**;
2. O **Servidor Web** detecta que trata-se de uma página PHP e encaminha a requisição ao interpretador **PHP**;
3. O interpretador **PHP** faz os processamentos necessários, inclusive acessos a bancos de dados e outros recursos e devolve o HTML para o **Servidor Web**;
4. O **Servidor Web** devolve O HTML ao **Browser**.

No diagrama abaixo você visualiza estas etapas:



2 - Sintaxe Básica

1 - Delimitando o código

O código PHP fica embutido no próprio HTML. O interpretador identifica quando um código é PHP pelas seguintes tags:

```
<?php  
comandos  
?>
```

ou

```
<script language="php">  
comandos  
</script>
```

ou

```
<?  
comandos  
?>
```

ou

```
<%  
comandos  
%>
```

O tipo de *tags* mais utilizado é o terceiro, que consiste em uma "abreviação" do primeiro. O último tipo serve para facilitar o uso por programadores acostumados à sintaxe de ASP. Para utilizá-lo é necessário habilitá-lo através do arquivo de configuração *php.ini*.

2 - Separando instruções

Entre cada instrução em PHP é preciso utilizar o ponto-e-vírgula, assim como em C, Perl e outras linguagens mais conhecidas. Na última instrução do bloco de script não é necessário o uso do ponto-e-vírgula, mas por questões estéticas recomenda-se o uso sempre.

3 - Nomes de variáveis

Toda variável em PHP tem seu nome composto pelo caracter \$ e uma string, que deve iniciar por uma letra ou o caracter "_".

PHP é case sensitive, ou seja, as variáveis \$teste e \$TESTE são diferentes. Por isso é preciso ter muito cuidado ao definir os nomes das variáveis. É bom evitar os nomes em maiúsculas, pois como veremos mais adiante, o PHP já possui algumas variáveis pré-definidas cujos nomes são formados por letras maiúsculas.

Além disso, existem algumas palavras reservadas como break e class, por exemplo, que não podem ser usadas para nomear variáveis.

4 - Comentários

Os comentários são porções de informações que podemos adicionar ao código para fins de documentação, sendo que o interpretador os ignorará. Existem dois tipos de comentários em PHP:

Comentários de uma linha:

Marca como comentário até o final da linha ou até o final do bloco de código PHP o que vier antes. Pode ser delimitado pelo caracter "#" ou por duas barras (//).

```
<?
print "teste"; #isto é um teste
print "teste"; //este teste é similar ao anterior
?>
```

Comentários de mais de uma linha:

Tem como delimitadores os caracteres "/*" para o início do bloco e "*/" para o final do comentário. Se o delimitador de final de código PHP (?>) estiver dentro de um comentário, não será reconhecido pelo interpretador.

```
<?
print "teste";      /* Isto é um comentário com mais
                    de uma linha */
?>
```

5 - Imprimindo código html

Um script php geralmente tem como resultado uma página html, ou algum outro texto. Para gerar esse resultado, deve ser utilizada uma das funções de impressão, echo e print.

Para utilizá-las deve-se utilizar um dos seguintes formatos:

```
<?
print(texto);
?>
```

ou

```
<?
print texto;
?>
```

ou

```
<?
echo (texto1, texto2, ... );
?>
```

ou

```
<?
echo texto;
?>
```

3 - Tipos

1 - Tipos Suportados

PHP utiliza checagem de tipos dinâmica, ou seja, uma variável pode conter valores de diferentes tipos em diferentes momentos da execução do script. Por este motivo não é necessário declarar o tipo de uma variável para usá-la. O interpretador PHP decidirá qual o tipo daquela variável, verificando o conteúdo em tempo de execução.

Ainda assim, é permitido converter os valores de um tipo para outro desejado, utilizando o *typecasting* ou a função *settype*.

Inteiros (integer ou long)

Uma variável pode conter os seguintes valores inteiros.

```
$teste = 1234;    # inteiro positivo na base decimal
$teste = -234;    # inteiro negativo na base decimal
$teste = 0234;    # inteiro na base octal-simbolizado pelo 0
                  # equivale a 156 decimal
$teste = 0x34;    # inteiro na base hexadecimal(simbolizado
                  # pelo 0x) - equivale a 52 decimal.
```

Números em Ponto Flutuante (double ou float)

Uma variável pode conter os seguintes valores de ponto flutuante:

```
$teste = 1.234;
$teste = 23e4;    # equivale a 230.000
```

Strings

As Strings armazenam uma sequência de Chars e podem ser atribuídas de duas maneiras:

Aspas Simples (')

Desta maneira, o valor da variável será exatamente o texto contido entre as aspas (com exceção de `\\` e `\'` - ver tabela abaixo)

Aspas Duplas (")

Desta maneira, qualquer variável ou caracter de escape será expandido antes de ser atribuído.

```
<?
$teste = "PHP";
$teste = '---$teste--\n';
print "$teste";
?>
```

A saída desse script será **---\$teste--\n**.

```
<?
$teste = "PHP";
$teste = "----$teste----\n";
print "$teste";
?>
```

A saída desse script será **----PHP--** (com uma quebra de linha no final).

Arrays

Arrays em PHP podem ser observados como mapeamentos ou como vetores indexados. Mais precisamente, um valor do tipo array é um dicionário onde os índices são as chaves de acesso. Vale ressaltar que os índices podem ser valores de qualquer tipo e não somente inteiros. Inclusive, se os índices forem todos inteiros, estes não precisam formar um intervalo contínuo.

Como a checagem de tipos em PHP é dinâmica, valores de tipos diferentes podem ser usados como índices de array, assim como os valores mapeados também podem ser de diversos tipos.

```
<?
$cor[1] =      "vermelho";
$cor[2]       = "verde";
$cor[3]       = "azul";
$cor["teste"] = 1;
?>
```

Equivalentemente, pode-se escrever:

```
<?
$cor = array(1 => "vermelho", 2 => "verde", 3 => "azul", "teste" => 1);
?>
```

Listas

As listas são utilizadas em PHP para realizar atribuições múltiplas. Através de listas é possível atribuir valores que estão num array para variáveis. Conforme o exemplo:

```
list($a, $b, $c) = array("a", "b", "c");
```

O comando acima atribui valores às três variáveis simultaneamente. É bom notar que só são atribuídos às variáveis da lista os elementos do array que possuem índices inteiros e não negativos. No exemplo acima as três atribuições foram bem sucedidas porque ao inicializar um array sem especificar os índices eles passam a ser inteiros, a partir do zero.

Objetos

Um objeto pode ser inicializado utilizando o comando *new* para instanciar uma classe para uma variável.

```
class teste {
    function nada() {
        print "nada";
    }
}

$teste = new teste();
$teste->nada();
```

Booleanos

O PHP não possui um tipo booleano, mas é capaz de avaliar expressões e retornar *true* ou *false*, através do tipo integer: é usado o valor 0 (zero) para representar o estado *false*, e qualquer valor diferente de zero (geralmente 1) para representar o estado *true*.

2 - Transformação de Tipos

Existem três maneiras para fazer a transformação de tipos em PHP.

Coerções

Quando ocorrem determinadas operações (adição, por exemplo) entre dois valores de tipos diferentes, o interpretador PHP converte o valor de um deles automaticamente (coerção).

É interessante notar que se o operando for uma variável, seu valor não será alterado.

O tipo para o qual os valores dos operandos serão convertidos é determinado da seguinte forma: Se um dos operandos for float, o outro será convertido para float, senão, se um deles for integer, o outro será convertido para integer.

```
$teste = "1";           // $teste é a string "1"  
$teste = $teste + 1;    // $teste é o integer 2  
$teste = $teste + 3.7;  // $teste é o double 5.7  
$teste = 1 + 1.5        // $teste é o double 2.5
```

Como podemos notar, o PHP converte string para integer ou double mantendo o valor. O sistema utilizado pelo PHP para converter de *strings* para números é o seguinte:

1. É analisado o início da string. Se contiver um número, ele será avaliado. Senão, o valor será 0 (zero);
2. O número pode conter um sinal no início ("+" ou "-");
3. Se a string contiver um ponto em sua parte numérica a ser analisada, ele será considerado, e o valor obtido será double;
4. Se a string contiver um "e" ou "E" em sua parte numérica a ser analisada, o valor seguinte será considerado como expoente da base 10, e o valor obtido será double;

```
$teste = 1 + "10.5";      // $teste == 11.5  
$teste = 1 + "-1.3e3";    // $teste == -1299  
$teste = 1 + "teste10.5"; // $teste == 1  
$teste = 1 + "10testes";  // $teste == 11  
$teste = 1 + " 10testes"; // $teste == 11  
$teste = 1 + "+ 10testes"; // $teste == 1
```

Transformação explícita de tipos

A sintaxe do ***typecast*** de PHP é semelhante ao da linguagem C, basta escrever o tipo entre parênteses antes do valor

```
$teste = 15;           // $teste é integer (15)
$teste = (double)$teste // $teste é double (15.0)
$teste = 3.9           // $teste é double (3.9)
$teste = (int)$teste    // $teste é integer (3)
                        // o valor decimal é truncado
```

Os tipos de ***casting(conversões)*** permitidos são:

(int), (integer)	cast para integer;
(real), (double), (float)	cast para float;
(string)	cast para string;
(array)	cast para array;
(object)	cast para objeto.

Com a função `settype`

A função `settype` converte uma variável para o tipo especificado, que pode ser "integer", "double", "string", "array" ou "object".

```
$teste = 15;           // $teste é integer
settype($teste,double) // $teste é double
```

4 - Operadores

1 - Aritméticos

Só podem ser utilizados quando os operandos são números (integer ou float). Se forem de outro tipo, terão seus valores convertidos antes da realização da operação.

+	adição
-	subtração
*	multiplicação
/	divisão
%	módulo

2 - Strings

Só há um operador exclusivo para strings:

.	concatenação
---	--------------

3 - Atribuição

Existe um operador básico de atribuição e diversos derivados. Sempre retornam o valor atribuído. No caso dos operadores derivados de atribuição, a operação é feita entre os dois operandos, sendo atribuído o resultado para o primeiro. A atribuição é sempre por valor, e não por referência.

=	atribuição simples
+=	atribuição com adição
-=	atribuição com subtração
*=	atribuição com multiplicação
/=	atribuição com divisão
%=	atribuição com módulo
.=	atribuição com concatenação

4 - Bit a bit

Comparam dois números bit a bit.

&	"e" lógico
	"ou" lógico
^	ou exclusivo
~	não (inversão)
<<	shift left
>>	shift right

5 - Lógicos

Utilizados para inteiros representando valores booleanos

```
and  "e" lógico
or   "ou" lógico
xor  ou exclusivo
!    não (inversão)
&&  "e" lógico
||   "ou" lógico
```

Existem dois operadores para "e" e para "ou" porque eles têm diferentes posições na ordem de precedência.

6 - Comparação

As comparações são feitas entre os valores contidos nas variáveis, e não as referências. Sempre retornam um valor booleano.

```
==  igual a
!=  diferente de
<   Menor que
>   maior que
<=  Menor ou igual a
>=  maior ou igual a
```

7 - Expressão Condicional

Existe um operador para seleção que é ternário. Funciona como se fosse um **IF**:

```
(expressao1)?(expressao2):(expressao3)
```

O interpretador PHP avalia a primeira expressão. Se ela for verdadeira, a expressão retorna o valor de expressão2. Senão, retorna o valor de expressão3.

8 - Incremento e Decremento

```
++  incremento
--  decremento
```

Podem ser utilizados de duas formas: antes ou depois da variável. Quando utilizado antes, retorna o valor da variável antes de incrementá-la ou decrementá-la. Quando utilizado depois, retorna o valor da variável já incrementado ou decrementado.

5 - Estruturas de Controle

1 - Blocos

Um bloco consiste de vários comandos agrupados com o objetivo de relacioná-los com determinado comando ou função. Em comandos como `if`, `for`, `while`, `switch` e em declarações de funções, os blocos podem ser utilizados para permitir que um comando faça parte do contexto desejado.

Blocos em PHP são delimitados pelos caracteres `"{"` e `"}"`. A utilização dos delimitadores de bloco em uma parte qualquer do código não relacionada com os comandos citados ou funções não produzirá efeito algum, e será tratada normalmente pelo interpretador.

```
if ( $x == $y )  
    comando1;  
    comando2;
```

Para que `comando2` esteja relacionado ao `if` é preciso delimitar onde começa e onde termina o bloco:

```
if ( $x == $y ){  
    comando1;  
    comando2;  
}
```

2 - Condicionais

IF

O mais comum dos comandos condicionais é o **if**. Ele testa a condição e executa o comando indicado se o resultado for `true` (valor diferente de zero). A sintaxe mais usada é:

```
if ( expressão )  
    comando;
```

Para incluir mais de um comando no `if`, é preciso utilizar um bloco, demarcado por chaves.

O **else** é um complemento opcional para o **if**. Se utilizado, o comando será executado se a expressão retornar o valor false (zero). A sintaxe mais usada é:

```
if (expressão)
    comando;
else
    comando;
```

SWITCH

O comando switch atua de maneira semelhante a uma série de comandos if na mesma expressão. O programador pode querer comparar uma variável com diversos valores, e executar um código diferente a depender de qual valor é igual ao da variável.

Quando isso for necessário, deve-se usar o comando **switch**. O exemplo seguinte mostra dois trechos de código que fazem a mesma coisa, sendo que o primeiro utiliza uma série de **if's** e o segundo utiliza **switch**:

```
if ( $i == 0 )
    print "i é igual a zero";
elseif ( $i == 1 )
    print "i é igual a um";
elseif ( $i == 2 )
    print "i é igual a dois";

switch ( $i ) {
case 0:
    print "i é igual a zero";
    break;
case 1:
    print "i é igual a um";
    break;
case 2:
    print "i é igual a dois";
    break;
}
```

É importante compreender o funcionamento do **switch** para não cometer enganos.

O comando **switch** testa linha a linha os cases encontrados, e a partir do momento que encontra um valor igual ao da variável testada, passa a executar todos os comandos seguintes, mesmo os que fazem parte de outro teste, até o fim do bloco.

Por isso usa-se o comando **break**, quebrando o fluxo e fazendo com que o código seja executado da maneira desejada. Veremos mais sobre o **break** mais adiante.

3 - Repetição

WHILE

O **while** é o comando de repetição(laço) mais simples. Ele testa uma condição e executa um comando, ou um bloco de comandos, até que a condição testada seja falsa.

```
while ( condição )  
    comando;
```

DO

O laço **do..while** funciona de maneira bastante semelhante ao **while**, com a simples diferença que a expressão é testada ao final do bloco de comandos.

```
do {  
    comando;  
    comando;  
} while ( condição );
```

FOR

O tipo de laço mais complexo é o **for**. Para os que programam em C, C++ ou Java, a assimilação do funcionamento do **for** é natural.

A sintaxe mais usada é:

```
for ( inicialização; condição; incremento )  
    comando;
```

Os três parâmetros que ficam entre parênteses têm as seguintes finalidades:

Inicialização - Comando ou sequencia de comandos a serem realizados antes do início do laço. Serve para inicializar as variáveis.

Condição - Expressão booleana que define se os comandos que estão dentro do laço serão executados ou não. Enquanto a expressão for verdadeira (valor diferente de zero) os comandos serão executados.

Incremento - Comando executado ao final de cada execução do laço.

Quebra de fluxo

Em todos os loops (for, while e do), o loop termina quando a condição que você está testando é atingida. Porém, em alguma situação, você desejará sair do loop antes do seu término normal. Para isso, você pode usar as palavras chaves *break* e *continue*.

Break

O comando break pode ser utilizado em laços de **do**, **for** e **while**, além do uso já visto no comando **switch**. Ao encontrar um **break** dentro de um desses laços, o interpretador PHP para imediatamente a execução do laço, seguindo normalmente o fluxo do script.

```
while ( $x > 0 ) {  
    if ( $x == 20 ) {  
        print "erro! x = 20";  
        break;  
    }  
}
```

Continue

O comando continue também deve ser utilizado no interior de laços, e funciona de maneira semelhante ao break, com a diferença que o fluxo ao invés de sair do laço volta para o início dele. Conforme o exemplo:

```
for ( $i = 0; $i < 100; $i++ ) {  
    if ( $i % 2 ) continue;  
    print " $i ";  
}
```

6 - Funções

1 - Definição

A sintaxe básica para definição de uma função é:

```
function nome_da_função(arg1, arg2, arg3) {  
    Comandos;  
    return valor;  
}
```

2 - Retorno

Toda função pode opcionalmente retornar um valor, ou simplesmente executar os comandos e não retornar valor algum.

Não é possível que uma função retorne mais de um valor, mas é permitido fazer com que uma função retorne um valor composto, como **listas** ou **arrays**.

3 - Argumentos

É possível passar argumentos ou parâmetros para uma função. Eles devem ser declarados logo após o nome da função, entre parênteses, e tornam-se variáveis pertencentes ao escopo local da função.

```
function imprime($texto){  
    print $texto;  
}  
imprime("teste de funções");
```

Passagem de parâmetros por referência

Normalmente, a passagem de parâmetros em PHP é feita por valor, ou seja, se o conteúdo da variável for alterado, essa alteração não afeta a variável original.

```
function soma5($numero) {  
    $numero += 5;  
}  
$a = 3;  
soma5($a); //$a continua valendo 3
```

No exemplo acima, como a passagem de parâmetros é por valor, a função **mais5** é inútil, já que após a execução sair da função o valor anterior da variável é recuperado.

Se a passagem de valor fosse feita por referência, a variável **\$a** teria 8 como valor. O que ocorre normalmente é que ao ser chamada uma função, o interpretador salva todo o escopo atual, ou seja, os conteúdos das variáveis. Se uma dessas variáveis for passada como parâmetro, seu conteúdo fica preservado, pois a função irá trabalhar na verdade com uma cópia da variável.

Se a passagem de parâmetros for feita por referência, toda alteração que a função realizar no valor passado como parâmetro afetará a variável que o contém.

Há duas maneiras de fazer com que uma função tenha parâmetros passados por referência:

1. Indicando isso na declaração da função, o que faz com que a passagem de parâmetros sempre seja assim;
2. Na própria chamada da função.

Nos dois casos utiliza-se o modificador "&":

```
function soma5(&$num1, $num2) {  
    $num1 += 5;  
    $num2 += 5;  
}  
  
$a = $b = 1;  
soma5($a, $b); /* Neste caso, só $num1 terá seu valor alterado, pois a passagem  
por referência está definida na declaração da função. */  
  
soma5($a, &$b); /* Aqui as duas variáveis terão seus valores alterados. */
```

Argumentos com valores padrão (default)

Em PHP é possível ter valores *default* para argumentos de funções, ou seja, valores que serão assumidos em caso de nenhum valor ser passado no lugar do argumento.

Quando algum parâmetro é declarado desta maneira, a passagem do mesmo na chamada da função torna-se opcional.

```
function teste( $teste = "testando" ) {  
    print $teste;  
}  
  
teste();           // imprime "testando"  
teste("outro teste"); // imprime "outro teste"
```

Vale lembrar que quando a função tem mais de um parâmetro, o que tem valor *default* deve ser declarado por último:

```
function teste( $figura = "circulo", $cor ) {  
    print "a figura é um $figura de cor $cor";  
}  
  
teste("azul");  
/* A função não vai funcionar da maneira esperada, ocorrendo um erro no  
interpretador. A declaração correta seria: */  
  
function teste2( $cor, $figura = "circulo" ) {  
    print "a figura é um $figura de cor $cor";  
}  
  
teste2("azul");  
/* Aqui a função funciona da maneira esperada, ou seja, imprime o texto: "a  
figura é um círculo de cor azul" */
```

4 - Contexto

O contexto é o conjunto de variáveis e seus respectivos valores num determinado ponto do programa. Na chamada de uma função, ao iniciar a execução do bloco que contém a implementação da mesma é criado um novo contexto, contendo as variáveis declaradas dentro do bloco, ou seja, todas as variáveis utilizadas dentro daquele bloco serão eliminadas ao término da execução da função.

5 - Escopo

O escopo de uma variável em PHP define a porção do programa onde ela pode ser utilizada. Na maioria dos casos todas as variáveis têm escopo global. Entretanto, em funções definidas pelo usuário um escopo local é criado. Uma variável de escopo global não pode ser utilizada no interior de uma função sem que haja uma declaração explícita.

```
$teste = "Testando";

function teste() {
    print $teste;
}

teste();
```

O trecho acima não produzirá saída alguma, pois a variável \$teste é de escopo global, e não pode ser referida num escopo local, mesmo que não haja outra com nome igual que cubra a sua visibilidade.

Para que o script funcione da forma desejada, a variável global a ser utilizada deve ser declarada.

```
$teste = "Testando";

function teste() {
    global $teste;
    print $teste;
}

teste();
```

Uma declaração "global" pode conter várias variáveis, separadas por vírgulas. Uma outra maneira de acessar variáveis de escopo global dentro de uma função é utilizando um array pré-definido pelo PHP cujo nome é \$GLOBALS.

O índice para a variável referida é o próprio nome da variável, sem o caracter \$. O exemplo acima e o abaixo produzem o mesmo resultado:

```
$teste = "Testando";

function teste() {
    print $GLOBALS["teste"]; // imprime $teste
    print $teste; // não imprime nada
}

teste();
```

7 - Variáveis e Constantes

1 - Declaração

Como a tipagem em PHP é dinâmica, as variáveis não precisam ser declaradas. Uma variável é inicializada no momento em que é feita a primeira atribuição. O tipo da variável será definido de acordo com o valor atribuído.

2 - O modificador `static`

Uma variável estática é visível num escopo local, mas ela é inicializada apenas uma vez e seu valor não é perdido quando a execução do script deixa esse escopo:

```
function teste() {  
    $a = 0;  
    print $a;  
    $a++;  
}
```

O último comando da função é inútil, pois assim que for encerrada a execução da função a variável **\$a** perde seu valor. Mas, no exemplo seguinte, a cada chamada da função a variável \$a terá seu valor impresso e será incrementada:

```
function teste() {  
    static $a = 0;  
    print $a;  
    $a++;  
}
```

O modificador **static** é muito utilizado em funções recursivas, já que o valor de algumas variáveis precisa ser mantido. Ele funciona da seguinte forma: O valor das variáveis declaradas como estáticas é mantido ao terminar a execução da função. Na próxima execução da função, ao encontrar novamente a declaração com **static**, o valor da variável é recuperado.

Resumindo, uma variável declarada como **static** tem o mesmo "tempo de vida" que uma variável global, porém sua visibilidade é restrita ao escopo local em que foi declarada e só é recuperada após a declaração.

3 - Variáveis variáveis

O PHP tem um recurso conhecido como variáveis variáveis, que consiste em variáveis cujos nomes também são variáveis. Sua utilização é feita através do duplo cifrão (\$\$).

```
<?php
$a = "hello";
$$a = "world";
print $hello;    //Imprime "world"
?>
```

4 - Variáveis enviadas pelo navegador

Para interagir com a navegação do usuário, é necessário que o PHP possa enviar e receber informações para o browser.

A maneira de enviar informações é através de um comando de impressão, como o **print**. Para receber informações vindas do navegador, através de um *link* ou um formulário html, o PHP utiliza as informações enviadas através da URL.

Se seu script PHP está localizado em "<http://localhost/teste.php>" e você o chama com a url "<http://localhost/teste.php?nome=paulo>", automaticamente o PHP criará uma variável com o nome **\$nome** contendo a string "**paulo**".

Note que o conteúdo da variável está no formato **urlencode**. Os formulários html já enviam informações automaticamente nesse formato, e o PHP decodifica sem necessitar de tratamento.

URLencode

O formato urlencode é obtido substituindo os espaços pelo caracter "+" e todos os outros caracteres não alfanuméricos (com exceção de "_") pelo caracter "%" seguido do código ASCII em hexadecimal.

Por exemplo: o texto "Testando 1 2 3 !!" em urlencode fica "Testando+1+2+3+%21%21"

O PHP possui duas funções para tratar informações em urlencode:

```
urlencode($texto);
```

ou

```
urldecode($texto);
```

Essas funções servem respectivamente para codificar ou decodificar um texto passado como argumento.

5 - Variáveis de Ambiente

O PHP possui diversas variáveis de ambiente, como a **\$PHP_SELF**, por exemplo, que contém o **nome** e o **path** do próprio arquivo.

Algumas outras variáveis contém informações sobre o browser do usuário, o servidor http, a versão do PHP e diversas outras informações.

Para obter uma listagem de todas as variáveis e constantes de ambiente e seus respectivos conteúdos, deve-se utilizar a função **phpinfo()**:

```
<?
phpinfo();
?>
```

6 - Verificando o tipo de uma variável

Por causa da tipagem dinâmica utilizada pelo PHP, nem sempre será possível saber qual o tipo de uma variável, em determinado instante, se não contar com a ajuda de algumas funções que ajudem a verificar isso. A verificação pode ser feita de duas maneiras:

Função que retorna o tipo da variável

A função para retorna o tipo da variável chama-se **gettype**:

```
gettype( $variavel );
```

A função **gettype** pode retornar as seguintes strings: "integer", "double", "string", "array", "object" e "unknown type".

Funções que testam o tipo da variável

São as funções **is_int()**, **is_integer()**, **is_real()**, **is_long()**, **is_float()**, **is_string()**, **is_array()** e **is_object()**. Todas possuem o mesmo formato:

```
is_integer( $variavel);
```

Todas essas funções retornam **true** se a variável for daquele tipo, e **false** em caso contrário.

7 - Destruindo uma variável

É possível desalocar uma variável se ela não for usada posteriormente através da função **unset**:

```
unset( $variavel );
```

A função destrói a variável, ou seja, libera a memória ocupada por ela, fazendo com que ela deixe de existir. Se mais na frente for feita uma chamada á variável, será criada uma nova variável de mesmo nome e de conteúdo vazio, a não ser que a chamada seja pela função **isset()**. Se a operação for bem sucedida, retorna **true**.

8 - Verificando se uma variável possui um valor

Existem dois tipos de teste que podem ser feitos para verificar se uma variável está setada: com a função **isset** e com a função **empty**.

isset()

Ela retorna **true** se a variável estiver setada (ainda que com uma string vazia ou o valor zero), e **false** em caso contrário.

```
isset( $variavel );
```

empty()

Ela retorna **true** se a variável não contiver um valor (não estiver setada) ou possuir valor 0 (zero) ou uma string vazia. Caso contrário, retorna **false**.

```
empty( $variavel );
```

9 - Constantes

Para definir constantes utiliza-se a função `define`. Uma vez definido, o valor de uma constante não poderá mais ser alterado. Uma constante só pode conter valores escalares, ou seja, não pode conter um **array** ou **objeto**:

```
define("nome_da_constante", valor_da_constante);
```

A função retorna **true** se for bem-sucedida:

Para utilizar o valor da constante basta chamar o nome da constante(sem \$).

```
define ("VALOR", 15);  
print VALOR;
```

8 - Classes & Objetos

1 - Classes

Uma classe é um conjunto de variáveis e funções relacionadas a essas variáveis. Uma vantagem da utilização de programação orientada a objetos é poder usufruir do recurso de encapsulamento de informação. Com o encapsulamento o usuário de uma classe não precisa saber como ela é implementada, bastando para a utilização conhecer a interface, ou seja, as funções disponíveis.

Uma classe é um tipo, e portanto não pode ser atribuída a uma variável. Para definir uma classe, deve-se utilizar a seguinte sintaxe:

```
class nome_da_classe {  
    var $variavel1;  
    var $variavel2;  
    function funcao1( $parametro ) {  
        comandos;  
    }  
}
```

2 - Objetos

Variáveis do tipo de uma classe são chamadas de objetos, e devem ser criadas utilizando o operador **new**:

```
$variavel = new $nome_da_classe;
```

Para utilizar as funções definidas na classe, deve ser utilizado o operador **"->"**:

```
$variavel->funcao1();
```

3 - A variável \$this

Na definição de uma classe, pode-se utilizar a variável **\$this**, que é a representação do próprio objeto. Assim, quando uma classe é instanciada em um objeto, e uma função desse objeto na definição da classe utiliza a variável **\$this**, essa variável significa o objeto que estamos utilizando.

Como exemplo da utilização de classes e objetos, podemos utilizar a classe `carrinho_de_compra`, que define um carrinho de compras de comércio eletrônico simples, com funções para adicionar e subtrair produtos do carrinho de compras:

```
class carrinho_de_compra {
    var $produtos;
    function adiciona( $valor ) {
        return $this->produtos + $valor;
    }

    function subtrai( $valor ) {
        return $this->produtos - $valor;
    }
}

$meu_carrinho = new carrinho_de_compra;
$meu_carrinho->adiciona(1);
$meu_carrinho ->subtrai(1);
```

4 - SubClasses

Uma classe pode ser uma extensão de outra. Isso significa que ela herdará todas as variáveis e funções da outra classe, e ainda terá as que forem adicionadas. Em PHP não é permitido utilizar herança múltipla, ou seja, uma classe pode ser extensão de apenas uma outra.

Para criar uma classe estendida, ou derivada de outra, deve ser utilizada a palavra reservada **extends**:

```
class novo_carrinho extends carrinho_de_compra {
    var $status;
    function status() {
        return $this->status;
    }
}
```

A classe acima é derivada da classe `carrinho_de_compra`, tendo as mesmas funções e variáveis, com a adição da variável `$status` e a função `status()`.

5 - Construtores

Um construtor é uma função definida na classe que é automaticamente chamada no momento em que a classe é instanciada (através do operador **new**). O construtor deve ter o mesmo nome que a classe a que pertence:

```
class conta {
    var $saldo;

    function conta () {
        $this->saldo = 0;
    }

    function saldo() {
        return $this->saldo;
    }

    function credito($valor) {
        $this->saldo += $valor;
    }
}
```

Podemos perceber que a classe `conta` agora possui um construtor, que inicializa a variável **\$saldo** com o valor 0.

Um construtor pode conter argumentos, que são opcionais, o que torna esta ferramenta mais poderosa. No exemplo acima, o construtor da classe `conta` pode receber como argumento um valor, que seria o valor inicial da conta.

Vale observar que para classes derivadas, o construtor da classe pai não é automaticamente herdado quando o construtor da classe derivada é chamado.

9 - Bancos de Dados

Como já foi mencionado, o PHP oferece funções de acesso aos mais diversos bancos de dados.

Abordaremos, a seguir, os bancos de dados mais usados no mercado que são o MySQL, Microsoft SQL Server e ODBC.

1 - MySQL

Conectando

Para acessar bases de dados MySQL, é necessário antes estabelecer uma conexão. Para isso, deve ser utilizado o comando **mysql_connect**, ou o **mysql_pconnect**.

A diferença entre os dois comandos é que o **mysql_pconnect** estabelece uma conexão permanente, ou seja, que não é encerrada ao final da execução do script.

```
mysql_connect("host" , "usuario" , "senha" );
```

ou

```
mysql_pconnect("host" , "usuario" , "senha");
```

O valor de retorno é um inteiro que identifica a conexão, ou falso se a conexão falhar. Antes de tentar estabelecer uma conexão, o interpretador PHP verifica se já existe uma conexão estabelecida com o mesmo host, o mesmo login e a mesma senha. Se existir, o identificador desta conexão é retornado. Senão, uma nova conexão é criada.

Uma conexão estabelecida com o comando **mysql_connect** é encerrada ao final da execução do script. Para encerrá-la antes disso deve ser utilizado o comando **mysql_close**:

```
mysql_close(identificador da conexão);
```

Se o identificador não for fornecido, a última conexão estabelecida será encerrada.

Selecionando a base de dados

Depois de estabelecida a conexão, é preciso selecionar a base de dados a ser utilizada, através do comando **mysql_select_db**:

```
mysql_select_db(nome da base, identificador da conexão );
```

Novamente, se o identificador da conexão não for fornecido, a última conexão estabelecida será utilizada.

Selecionando dados

Para executar consultas SQL no MySQL, utiliza-se o comando **mysql_query**:

```
mysql_query("select * from mytable", conexão );
```

No caso de uma expressão **SELECT**, as linhas retornadas são armazenadas numa memória de resultados, o **resultset**, e o valor de retorno é o identificador deste **resultset**.

Inserindo/Atualizando/Apagando dados

Para executar estas operações SQL no MySQL, também utiliza-se o comando **mysql_query**:

```
mysql_query("insert into mytable(1,2,3)", conexão );
```

No caso de uma expressão **INSERT/UPDATE/DELETE**, o comando **mysql_query** retorna 1 se obtiver sucesso e 0 em caso contrário.

Apagando o Resultset

```
mysql_free_result( resultset );
```

O comando **mysql_free_result** deve ser utilizado para apagar da memória o **resultset** indicado.

Selecionado o número de linhas

```
mysql_num_rows( resultset );
```

O comando **mysql_num_rows** retorna o número de linhas contidas num determinado **resultset**.

Utilizando os Resultados

Existem diversas maneiras de ler os resultados de um **SELECT**, ou seja, um **resultset**. As mais comuns são:

```
mysql_fetch_array( resultset );
```

Este comando lê uma linha do **resultset** e devolve na forma de um array, cujos índices podem ser numéricos iniciando por 0(zero) ou podem ser os **nomes** dos campos ou aliases das tabelas.

```
mysql_fetch_row( resultset );
```

Funcionamento idêntico ao comando anterior, porém, neste caso os índices do array são somente **numéricos**, iniciando pelo 0 (zero).

```
mysql_fetch_assoc( resultset );
```

Funcionamento identico ao comando anterior, porém, neste caso os índices do array são somente os **nomes** dos campos ou aliases das tabelas.

2 - Microsoft SQL Server(MSSQL)

Conectando

Para acessar bases de dados num servidor MSSQL(Microsoft SQL Server), é necessário antes estabelecer uma conexão. Para isso, deve ser utilizado o comando **mssql_connect**, ou o **mssql_pconnect**.

A diferença entre os dois comandos é que o **mssql_pconnect** estabelece uma conexão permanente, ou seja, que não é encerrada ao final da execução do script:

```
mssql_connect("host" , "usuario" , "senha");
```

ou

```
mssql_pconnect("host" , "usuario" , "senha");
```

O valor de retorno é um inteiro que identifica a conexão, ou falso se a conexão falhar. Antes de tentar estabelecer uma conexão, o interpretador PHP verifica se já existe uma conexão estabelecida com o mesmo host, o mesmo login e a mesma senha. Se existir, o identificador desta conexão é retornado. Senão, uma nova conexão é criada.

Uma conexão estabelecida com o comando **mssql_connect** é encerrada ao final da execução do script. Para encerrá-la antes disso deve ser utilizado o comando **mssql_close**:

```
mssql_close( conexão );
```

Se o identificador não for fornecido, a última conexão estabelecida será encerrada.

Selecionando a base de dados

Depois de estabelecida a conexão, é preciso selecionar a base de dados a ser utilizada, através do comando **mssql_select_db**:

```
mssql_select_db(nome da base, conexão);
```

Novamente, se o identificador da conexão não for fornecido, a última conexão estabelecida será utilizada.

Selecionando dados

Para executar consultas SQL no MSSQL, utiliza-se o comando **mssql_query**:

```
mssql_query("select * from mytable", conexão );
```

No caso de uma expressão SELECT, as linhas retornadas são armazenadas numa memória de resultados, o **resultset**, e o valor de retorno é o identificador deste **resultset**.

Inserindo/Atualizando/Apagando dados

Para executar estas operações SQL no MSSQL, também utiliza-se o comando **mssql_query**:

```
mssql_query("insert into mytable(1,2,3)", conexão );
```

No caso de uma expressão **INSERT/UPDATE/DELETE**, o comando **mssql_query** retorna 1 se obtiver sucesso e 0 em caso contrário.

Apagando o Resultset

```
mssql_free_result( resultset );
```

O comando **mssql_free_result** deve ser utilizado para apagar da memória o **resultset** indicado.

Selecionando o número de linhas

```
mssql_num_rows( resultset );
```

O comando **mssql_num_rows** retorna o número de linhas contidas num determinado **resultset**.

Utilizando os Resultados

Existem diversas maneiras de ler os resultados de um **SELECT**, ou seja, um **resultset**. As mais comuns são:

```
mssql_fetch_array( resultset );
```

Este comando lê uma linha do **resultset** e devolve na forma de um array, cujos índices podem ser numéricos iniciando por 0(zero) ou podem ser os **nomes** dos campos ou aliases das tabelas.

```
mssql_fetch_row( resultset );
```

Funcionamento idêntico ao comando anterior, porém, neste caso os índices do array são somente **numéricos**, iniciando pelo 0 (zero).

```
mssql_fetch_assoc( resultset );
```

Funcionamento idêntico ao comando anterior, porém, neste caso os índices do array são somente os **nomes** dos campos ou aliases das tabelas.

3 - Fontes de Dados ODBC

Antes que possamos fazer qualquer coisa numa fonte de dados ODBC (como conectar, inserir, selecionar, etc.), precisamos configurá-la, a menos que isto já tenha sido feito. Veremos como configurar, passo a passo, a fonte de dados **ODBC** no capítulo 7 dos apêndices.

Conectando

Para acessar Fontes de Dados **ODBC**, é necessário antes estabelecer uma conexão. Para isso, deve ser utilizado o comando `odbc_connect`:

```
odbc_connect( datasource, usuario , senha );
```

O valor de retorno é um inteiro que identifica a conexão, ou falso se a conexão falhar. Antes de tentar estabelecer uma conexão, o interpretador PHP verifica se já existe uma conexão estabelecida com o mesmo **datasource**, o mesmo **usuario** e a mesma **senha**. Se existir, o identificador desta conexão é retornado. Senão, uma nova conexão é criada.

Uma conexão estabelecida com o comando `odbc_connect`, é encerrada ao final da execução do script. Para encerrá-la antes disso deve ser utilizado o comando `odbc_close`:

```
odbc_close( conexão );
```

Se o identificador não for fornecido, a última conexão estabelecida será encerrada.

Selecionando dados

Para executar consultas SQL através de ODBC, utiliza-se o comando `odbc_exec`:

```
odbc_exec( conexão, ""select * from mytable"" );
```

No caso de uma expressão **SELECT**, as linhas retornadas são armazenadas numa memória de resultados, o **resultset**, e o valor de retorno é o identificador deste **resultset**.

Inserindo/Atualizando/Apagando dados

Para executar estas operações SQL em ODBC, também utiliza-se o comando **odbc_exec**:

```
odbc_exec(conexão, "insert into mytable(1,2,3)");
```

No caso de uma expressão **INSERT/UPDATE/DELETE**, o comando **odbc_exec** retorna 1 se obtiver sucesso e 0 em caso contrário.

Apagando o Resultset

```
odbc_free_result( resultset );
```

O comando **odbc_free_result** deve ser utilizado para apagar da memória o **resultset** indicado.

Selecionando o número de linhas

```
odbc_num_rows( resultset );
```

O comando **odbc_num_rows** retorna o número de linhas contidas num determinado **resultset**.

Utilizando os resultados

Existem diversas maneiras de ler os resultados de um **SELECT**, ou seja, um **resultset**. As mais comuns são:

```
odbc_fetch_array( resultset );
```

Este comando lê uma linha do **resultset** e devolve na forma de um array, cujos índices podem ser numéricos iniciando por 0(zero) ou podem ser os **nomes** dos campos ou aliases das tabelas.

```
odbc_fetch_row( resultset );
```

Funcionamento idêntico ao comando anterior, porém, neste caso os índices do array são somente **numéricos**, iniciando pelo 0 (zero).

10 - Headers

O comando `header`, como o próprio nome sugere, permite enviar cabeçalhos html para o cliente. O principal cuidado que deve-se ter na sua utilização é que o mesmo não pode ser enviado depois de qualquer tipo de texto:

```
header("tipo de cabeçalho");
```

Alguns exemplos de utilizações deste comando:

```
header("Location: http://www.naredemundial.com.br");
```

O comando acima direciona o usuário para a URL indicada.

```
header("http/1.0 404 Not Found");
```

O comando acima define o script atual como uma mensagem de erro do tipo 404.

```
header("Cache-Control: no-cache, must-revalidate"); //HTTP/1.1  
header("Pragma: no-cache"); //HTTP/1.0
```

O comando acima define a expiração da página.

Estes são somente os mais utilizados e para obter uma lista completa dos headers **HTTP** possíveis, visite o seguinte endereço na web:

```
http://www.w3.org/Protocols/rfc2068/rfc2068
```

11 - Cookies

Cookies são pequenos arquivos gravados no cliente por um determinado site, sendo que, somente este poderá recuperar as informações do mesmo.

Este recurso é muito útil para que determinadas informações sejam fornecidas pelo usuário apenas uma vez. Exemplos de utilização de cookies são os sites que informam a quantidade de vezes que você já visitou, ou alguma informação fornecida numa visita anterior.

1 - Criando

Para gravar cookies no cliente, deve ser utilizada a função **setcookie**:

```
setcookie(nome, valor, expiração, path, dominio, secure);
```

Sendo que:

nome	nome do cookie;
valor	valor armazenado no cookie;
exp	data de expiração do cookie(opcional), no formato Unix. Se não for definida, o cookie expirará quando o browser for fechado;
path	path do script que gravou o cookie;
domínio	domínio responsável pelo cookie;
secure	valor 1, indica que o cookie só pode ser transmitido por uma conexão segura (https).

Somente o primeiro e o segundo parametros(nome e valor) são obrigatórios para gravar o cookie.

Os Cookies, assim com os **headers**, só podem ser gravados antes do envio de qualquer informação para o cliente, inclusive os próprios **headers**.

2 - Recuperando informações

Os cookies lidos ficam armazenados no array `$_COOKIE[]`, tendo como índice o nome do cookie, ou ainda numa variável cujo nome é o mesmo do cookie, precedido pelo símbolo `$`.

Um cookie gravado numa página anterior assim:

```
setcookie("nome", "Joãozinho da Silva");
```

Poderá ser lido assim:

```
$_COOKIE["nome"];
```

ou ainda, diretamente pela variável:

```
$nome;
```

12 - Manipulação de Arquivos

Através do PHP é possível manipular todos arquivos e pastas do sistema e inclusive arquivo remotos.

1 - Copiando

Para fazer uma cópia de arquivo basta utilizar a função **copy**, desde que o usuário tenha as permissões necessárias para isso:

```
copy( origem, destino);
```

2 - Verificando tamanho

A função **filesize** pode ser bastante útil na criação de um script que liste o conteúdo de um diretório, por exemplo, mas também é utilizada em combinação com a função **fread**, como veremos:

```
filesize(arquivo);
```

Esta função retorna um inteiro com o tamanho do arquivo, em bytes, ou false em caso de erro.

3 - Verificando se existe

Para evitar erros em tratamento de arquivos, é aconselhável verificar se o determinado arquivo existe antes de tentar a manipulação do mesmo. Utiliza-se para isso a função **file_exists**:

```
file_exists(arquivo);
```

Esta função tem um comportamento booleano: retorna apenas true ou false, não informando mais nada sobre o arquivo.

4 - Limpando o cache

Algumas funções que verificam o estado de arquivos utilizam um **cache**, ou seja, chamadas sucessivas da mesma função com relação ao mesmo arquivo não verificam se houve mudança no mesmo, retornando sempre o mesmo valor.

Para eliminar esse **cache**, obrigando o PHP a reavaliar o valor de retorno de uma função, deve ser utilizada a seguinte função:

```
clearstatcache();
```

As funções **filesize** e **file_exists** utilizam este **cache**.

5 - Abrindo arquivos

Para manipular arquivos é necessário abri-lo antes. Para isso deve ser utilizada a função **fopen**.

Um arquivo pode ser aberto em modos de leitura e/ou escrita:

Leitura

```
fopen("c:\teste.txt", r);
```

Escrita

```
fopen("c:\teste.txt", w);
```

A função **fopen** retorna **false** em caso de erro, e um identificador do arquivo em caso de sucesso. Esse identificador será utilizado em outras funções que manipulam o conteúdo do arquivo.

O primeiro argumento é uma string contendo o nome do arquivo; o segundo, o modo como o arquivo será aberto, que pode ser um dos seguintes:

r

Abre o arquivo com permissão apenas para leitura.

R+

Abre o arquivo com permissão para escrita e leitura, posicionando o ponteiro no início do mesmo.

w

Abre o arquivo com permissão apenas para escrita. Se o arquivo existir, todo o conteúdo é apagado. Se não existir, o PHP tenta criá-lo. O ponteiro é posicionado no início do arquivo

W+

Abre o arquivo com permissão para escrita e leitura. Se o arquivo existir, todo o conteúdo é apagado. Se não existir, o PHP tenta criá-lo. O ponteiro é posicionado no início do arquivo

a

Abre o arquivo com permissão apenas para escrita. Se o arquivo não existir, o PHP tenta criá-lo. O ponteiro é posicionado no final do arquivo.

A+

Abre o arquivo com permissão para escrita e leitura. Se o arquivo não existir, o PHP tenta criá-lo. O ponteiro é posicionado no final do arquivo.

Arquivos Remotos

Além da manipulação de arquivos locais, a função **fopen** também permite abrir arquivos remotos, utilizando os protocolos http ou ftp, da seguinte maneira:

Se a string como o nome do arquivo iniciar por "**http://**" (maiúsculas e minúsculas são iguais), uma conexão é aberta com o servidor e o arquivo contendo o texto de retorno será aberto para leitura.

Se a string como o nome do arquivo iniciar por "**ftp://**" (maiúsculas e minúsculas são iguais), uma conexão é aberta com o servidor e o arquivo será aberto. utilizando ftp o arquivo poderá ser aberto para leitura ou escrita, mas não simultaneamente.

6 - Fechando Arquivos

Para encerrar a manipulação de um arquivo deve-se utilizar a função **fclose**:

```
fclose(arquivo);
```

7 - Lendo arquivos

```
fread( arquivo, tamanho);
```

Esta função retorna uma string com o conteúdo do arquivo. O segundo parâmetro determina até onde o arquivo será lido, em bytes.

Se o tamanho determinado for maior que o arquivo, não ocorre erro, tendo como retorno apenas o arquivo. Na maioria dos casos a função **fread** é bastante útil, como no exemplo abaixo:

```
$meuarquivo = "c:/autoexec.bat";  
$id          = fopen($meuarquivo, "r");  
$conteudo    = fread($id, filesize($meuarquivo));
```

A função **fread** é "binary-safe", ou seja, pode ser usada para ler o conteúdo de um arquivo binário. Obviamente nesse caso é preciso saber exatamente onde utilizar o valor lido, para não obter resultados indesejados.

8 - Escrevendo em arquivos

```
fwrite( arquivo, conteudo, tamanho);
```

Esta função grava num arquivo o conteúdo do segundo parâmetro. Se o tamanho é fornecido e for menor que o tamanho da string, será feita a gravação apenas de uma parte da mesma, determinada pelo terceiro parâmetro.

13 - Email

PHP oferece o mais simples método para envio de emails entre as linguagens que estamos abordando, basta utilizar a função **mail** com seus parâmetros básicos:

```
mail(destinatario, assunto, mensagem, headers);
```

onde:

destinatario

Esta é a string contendo o endereço de email do destinatário.

assunto

Este é o assunto do email.

mensagem

Aqui vai o corpo do email.

headers

Este campo é opcional, e contém outras informações de cabeçalho, como por exemplo "**from**", "**reply-to**", "**bcc**", etc.

Para facilitar a compreensão dos scripts, os argumentos (como geralmente são strings grandes) devem ser atribuídos a variáveis antes da chamada da função mail.

Exemplo de email em formato HTML:

```
<?
$destinatario = teste@teste.com;
$assunto      = "email em html";
$mensagem     = "este é meu email em <b>HTML</b>";
$from         = "From: teste@teste.com\n";
$headers      = "MIME-Version: 1.0\r\n";
$headers      .= "Content-type: text/html; charset=iso-8859-1\r\n";
$headers      .= $from;
```

```
if ( mail( $destinatario, $assunto, $mensagem, $headers ) ) {  
    print "email enviado para $destinatario";  
} else {  
    print "email não enviado para $destinatario";  
}  
?>
```

14 - PHP & Java

Uma das grandes vantagens do PHP é sua possibilidade de interação com praticamente todas as demais tecnologias disponíveis atualmente, como Flash, PDF, Imagens, Bancos de Dados diversos, Java e até a novíssima tecnologia .NET.

Veremos aqui como acessar funções e métodos oriundos de Classes Java. A vantagem disso é que uma classe Java é um arquivo compilado, o que torna sua utilização mais segura, além do que o Java possui alguns recursos bem avançados e interessantes.

Para usar Classes Java, deve se proceder da mesma maneira que se faria para usar uma Classe PHP normal, ou seja:

```
<?
/* Instanciando uma classe Java que está num arquivo chamado
   noticias.class
*/
$myjava = new Java("noticias");

//Lendo uma função vinda da classe Java
$resultado = myjava->selectNoticias();

//Imprimindo o conteúdo recebido da classe Java
print $resultado;
?>
```

Para usar este recurso, é preciso preparar o ambiente Java/PHP, maiores informações sobre isso você encontra no capítulo 6 dos apêndices.

15 - Sessions

Podemos usar variáveis de sessão e gerenciá-las facilmente com PHP. Sessions são semelhantes aos Cookies, porém, as Sessions ficam gravadas no servidor e não no computador do usuário.

1 - O Objeto Session

O PHP cria uma nova Session para cada novo usuário e destrói a respectiva Session quando a sessão termina.

2 - Quando a sessão começa?

A sessão começará quando:

1. Quando um valor é armazenado numa variável de sessão;
2. Quando encontra a instrução **session_start()** no script.

3 - Quando a sessão termina?

A sessão terminará se um usuário não requisitar ou atualizar uma página da aplicação por um determinado período.

Se você quiser alterar este intervalo de timeout, poderá fazer isso através do PHP.INI.

Para encerrar uma sessão imediatamente ou forçar que esta termine, poderá fazer isso através da função **session_destroy()**.

```
<?
session_destroy();
?>
```

O principal problema com sessões é saber quando elas devem ser encerradas. Não sabemos se a ultima requisição do usuario foi o final da sessão ou não. Sendo assim, não sabemos por quanto tempo manter a sessão ativa. Mantendo a sessão ativa por um longo tempo, esta estará consumindo recursos do servidor. Mas, se a sessão for deletada muito rapida corremos o risco de quando o usuario retornar ele ter que começar tudo de novo. Encontrar o **Timeout** adequado pode ser uma tarefa muito dificil.

4 - Gravando e Recuperando Variáveis de Sessão

```
<?
session_start();                //inicia a sessão
$contador = $_SESSION['CONTADOR']; //recupera valores da sessão

if (( $contador != 0 )||( $contador != "" )) {
    $contador++;
} else {
    $contador = 1;
}

print "você esteve aqui $contador vezes";

$_SESSION['CONTADOR'] = $contador; //Grava valores na sessão
?>
```

Com o exemplo acima, a cada visita que você fizer à esta página, a variável `$_SESSION['CONTADOR']` será incrementada. Este valor somente será perdido se a sua janela do browser for fechada ou se expirar o tempo de existência da mesma, valor este que está configurado no arquivo de configurações do PHP, o **PHP.INI**.

16 - Includes/Requires

Outra grande vantagem do PHP é o uso de **"includes"** e/ou **"requires"**.

Esses dois comandos são idênticos, exceto de como eles manipulam os erros. **include()** produz um erro do tipo **Warning**, enquanto **require()** produzirá um **Fatal Error**.

Utilizamos **require()** se desejamos que um arquivo faltando interrompa o processamento da página.

O comando **include()** não se comporta da mesma maneira, permitindo que o script continue nessas situações.

Quando um arquivo é incluído, seu código entra no escopo de variável da linha onde a inclusão ocorre. Qualquer variável disponível da linha onde a chamada da inclusão ocorre estará disponível para o arquivo incluído, daquele ponto em diante.

Também existem as funções **include_once()** e **require_once()**, que funcionam de maneira semelhante a **include()** e **require()**, respectivamente, porém antes, elas testam se o arquivo já não foi incluído na página atual, muito útil em páginas com vários includes.

```
include("funcoes.php");
```

ou

```
require("funcoes.php");
```

17 - Tratamento de Erros

Os tratamentos de erros são muito importantes para que evitemos que o usuário receba mensagens indesejáveis ou confusas.

1 - Erros de Função

Para sumprimir e/ou tratar mensagens de erro de funções, basta preceder a chamada da função com o sinal de @.

```
<?
    @fopen('/tmp/f7g6b6','r');
?>
```

2 - Erros Globais

Podemos usar a função **error_reporting()** para configurar o nível de erros que desejamos que sejam exibidos na execução do script.

```
<?
//Desabilita todas as mensagens de erro
error_reporting(0);

//Reporta erros simples de funcionamento do script
error_reporting (E_ERROR | E_WARNING | E_PARSE);

//Reporta E_NOTICE
error_reporting (E_ERROR | E_WARNING | E_PARSE | E_NOTICE);

//Reporta todos erros exceto E_NOTICE
//Este é o valor padrão do php.ini
error_reporting (E_ALL ^ E_NOTICE);

//Reporta todos erros
error_reporting (E_ALL);

//Idêntico ao error_reporting(E_ALL);
ini_set ('error_reporting', E_ALL);
?>
```

Tabela de Códigos de Erro

E_ERROR	Fatal run-time errors
E_WARNING	Run-time warnings
E_PARSE	Compile-time parse errors
E_NOTICE	Run-time notices
E_CORE_ERROR	Fatal startup errors
E_CORE_WARNING	Startup warnings

E_COMPILE_ERROR	Fatal compile-time errors
E_COMPILE_WARNING	Compile-time warnings
E_USER_ERROR	User-generated errors
E_USER_WARNING	User-generated warnings
E_USER_NOTICE	User-generated notices
E_ALL	All of the above

3 - Die/Exit

A função **die()** é um apelido para a função **exit()** e serve para terminar a execução dos scripts em caso de erros. A função exibe uma mensagem de erro personalizada antes de encerrar a execução do script.

```
<?
$filename = '/caminho/para/arquivo';
$file = fopen ($filename, 'r') or exit("Não pude abrir o arquivo $filename");
?>
```

ASP

Active Server Pages

1 - Introdução

1 - O que é?

Active Server Pages, o ASP, é a tecnologia criada pela Microsoft para o desenvolvimento de aplicações web.

2 - O que pode ser feito?

Basicamente, qualquer coisa pode ser feita com ASP, desde coletar dados de formulários até gerar páginas dinâmicas oriundas de bancos de dados, XML, arquivos texto, etc.

3 - Como Surgiu?

Após um tempo de existência do Internet Information Server 2.0, o servidor web da Microsoft, a empresa começou a publicar o beta-teste de uma tecnologia cujo o nome de código era Denali.

Esta tecnologia passou a chamar-se Active Server Pages e atualmente é um aspecto muito importante na estratégia do Internet Information Server.

A tecnologia ASP está encapsulada em um único e pequeno arquivo, do tipo DLL, chamado ASP.DLL, sendo que este é um filtro ISAPI (Internet Server Application Programming Interface) que reside no mesmo espaço de memória do IIS.

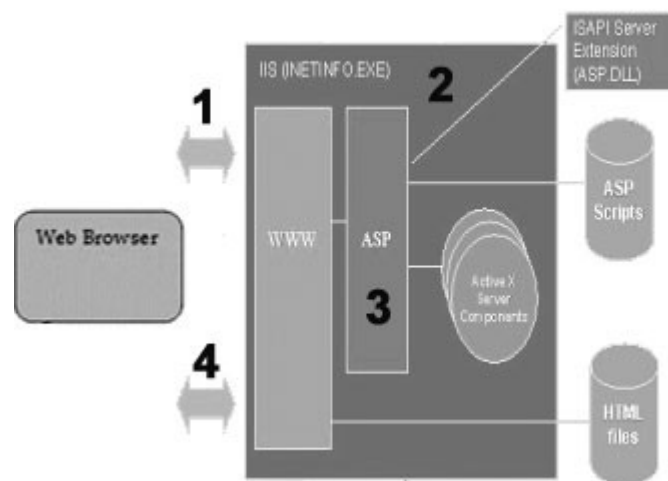
A novíssima tecnologia .Net é a evolução do ASP e a nova aposta da Microsoft em tecnologia web. As principais diferenças da tecnologia .Net são as várias linguagens que podem ser usadas (J# e c#) além da possibilidade de ter scripts compilados, o que melhora a performance.

4 - Diagrama

Uma requisição para o ASP funciona assim:

5. O **Browser** faz uma requisição ao **Servidor Web**;
6. O **Servidor Web** detecta que trata-se de uma página ASP e encaminha a requisição ao interpretador **ASP**;
7. O interpretador **ASP** faz os processamentos necessários, inclusive acessos a bancos de dados e outros recursos e devolve o HTML para o **Servidor Web**;
8. O **Servidor Web** devolve O HTML ao **Browser**.

No diagrama abaixo você visualiza estas etapas:



2 - Sintaxe básica

1 - Delimitando o código

O código ASP, uma vez que, fica embutido em meio ao código HTML, para que o interpretador possa delimitar o que é ASP e o que é HTML precisa de tags que indiquem isso:

```
<%  
comandos  
%>
```

2 - Separando Instruções

O ASP não possui o conceito de separação de instruções com o ";" ou outro caracter, como no PHP e JSP.

3 - Nomes de variáveis

O nome de uma variável pode ser uma sequência de um ou mais caracteres alfabéticos, sendo que, esses nomes só podem ser iniciados por caracteres alfabéticos. Além disso, existem algumas palavras reservadas como break e class, por exemplo, que não podem ser usadas para nomear variáveis.

É preciso ressaltar, também, que o ASP não faz distinção entre letras maiúsculas e minúsculas.

4 - Comentários

Existe a tag de comentários para ASP que é o sinal de apóstrofe conforme abaixo:

```
<%  
'comentários  
%>
```

5 - Imprimindo código HTML

```
<%  
Response.write("ASP imprimindo...")
```

```
Ou  
Response.write "ASP imprimindo..."  
%>
```

3 - Tipos

1 - Tipos Suportados

O ASP, possui um único tipo de dados chamado "Variant". Um dado Variant é uma tipo especial de dados que pode conter espécies diferentes de informações, dependendo de como seja usado.

Sendo assim uma variável declarada como "Variant" pode conter tanto um valor numérico quanto uma cadeia de caracteres.

SubTipos da Variant

Variant

Contém intencionalmente nenhum valor válido.

Inteiros

Integer - inteiros de -32768 a 32.767.

Ponto Flutuante

Currency - 922.337.203.685.477.4808 a 922.337.203.685.477.5807.

Strings

String - variáveis alfanuméricas que podem ter comprimento de 0 até 2 bilhões de caracteres.

Objetos

Object - qualquer referência de objeto.

Booleanos

Boolean - Contém true ou false.

Byte

Byte - inteiros de 0 a 255.

Data

Date - valores de data.

Erro

Error - contém um número de erro.

2 - Transformação de Tipos

CBool(expressão)

Converte uma expressão numérica ou string em um valor booleano.

CByte(expressão)

Converte uma expressão numerica ou string em um valor do tipo byte.

CCur(expressão)

Converte uma expressão numérica ou string em um valor do tipo currency.

CDate(expressão)

Converte uma expressão numérica ou string em um valor do tipo date.

CDbl(expressão)

Converte uma expressão numerica ou string em um valor double.

CInt(expressão)

Converte uma expressão numerica ou string em um valor inteiro.

CLng(expressão)

Converte uma expressão numerica ou string em um valor long.

CSng(expressão)

Converte uma expressão numerica ou string em um valor single.

CStr(expressão)

Converte uma expressão numerica ou string em um valor string.

4 - Operadores

1 - Aritméticos

Exponenciação	^
Multiplicação	*
Divisão inteira	/
Adição	+
Subtração	-
Módulo	MOD

2 - Strings

Concatenação	&
--------------	---

3 - Atribuição

=

4 - Lógicos

não	not
e	and
ou	or

5 - Comparação

Igualdade	=
Diferença	<>
Menor que	<
Maior que	>
Menor ou igual que	<=
Maior ou igual que	>=
Comparação de Strings	LIKE

6 - Incremento e Decremento

ASP não possui um operador específico para este fim, usa-se isso então:

```
variavel = variavel + 1
ou
variavel = variavel - 1
```

5 - Estruturas de Controle

1 - Blocos

O ASP não possui o conceito de delimitação de blocos com a "{" e a "}" ou outro caracter, como no PHP e JSP.

2 - Condicionais

IF

Este é o comando mais trivial dos condicionais. Ele testa uma expressão e executa determinado comando se esta for verdadeira.

```
if expressão1 then comando
```

Existe um complemento para o IF, no caso da expressão 1 não for verdadeira, ele executa os comando do ELSE então:

```
If expressão1 then  
    comando  
else  
    comando alternativo  
end if
```

SELECT...CASE

Este condicional avalia uma única expressão no topo da estrutura. O resultado da expressão é então comparada com os valores para cada Case da estrutura.

Se algum dos Cases retorna como verdadeiro então o respectivo bloco de código passa a ser executado.

```
Select Case  
    minha_variável  
Case "1"  
    Response.write("um")  
Case "2"  
    Response.write("dois")  
Case "3"  
    Response.write("tres")  
Case else  
    Response.write("nenhum deles...")  
End select
```

3 - Repetição

WHILE...WEND

O *loop while* é usado para repetir uma instrução ou bloco de instruções até que uma condição particular seja verdadeira:

```
while condição
    comando
wend
```

DO...

O loop *DO* é exatamente como o loop *WHILE*, exceto pelo fato de que ele executará uma dada instrução ou bloco até que uma condição seja falsa. A diferença principal entre os dois é que ele possui algumas variações pois pode ser usado em conjunto com o *while* para testar uma determinada condição antes de entrar no laço ou depois do laço e ainda pode ser usado em conjunto com o *Until*.

Aqui o **Do** com **while** depois:

```
Do
    comando
loop while condicao
```

Aqui usando o **while** antes do comando **Do**:

```
Do while condicao
    comando
Loop
```

Aqui o **Do** em conjunto com o **Until**

```
Do until condicao
    Commandos
Loop
```

Para sair de um laço **Do** pode-se usar a declaração **Exit Do**.

FOR...NEXT

O *loop for*, repete uma instrução ou um bloco de instruções algum número de vezes até que a condição seja satisfeita.

Loops *for* são freqüentemente usados para simples iterações na qual você repete um bloco de instruções um certo número de vezes e então para:

```
For x = 1 To 50
    comando
Next
```

Pode sair de um laço For usando a declaração Exit For.

O valor da interação pode ser alterado usando a declaração Step assim:

```
For x = 1 To 50 Step 2
    comando
Next
```

FOR EACH...NEXT

Um laço For Each... Next é muito parecido com os laços For...Next, sendo que a diferença é como ocorre o número de repetições.

Ele repete um grupo de declarações para cada item em uma coleção de objetos ou ainda para cada elemento de um array passado. Tornando-se muito útil quando não se sabe qual o número exato de iterações que se necessita.

Quebras de Fluxo

Em todos os loops (for, while e do), o loop termina quando a condição que você está testando é atingida. Porém, em alguma situação, você desejará sair do loop antes do seu término normal. Para isso, você pode usar as palavras chaves *break* e *continue*.

BREAK

A palavra reservada **break** pára imediatamente a execução do loop corrente. Se você tiver loops aninhados dentro de loop mais externo; caso contrário, o programa simplesmente continua a execução da próxima instrução após o loop.

CONTINUE

A palavra reservada **continue** pára o loop e começa novamente na próxima iteração. **Continue** é útil quando você quer ter situações de casos especiais dentro de loop.

6 - Funções

1 - Definição

No ASP existem duas espécies de Funções ou Procedures: as **Subs** e as **Functions**.

Subs

Este é o tipo de procedimento que executa determinada rotina sem retornar um valor, sendo que é possível passar parâmetros para este.

```
Sub EscreveNome( NOME )  
    Response.Write("Seu nome é: " & NOME)  
End Sub
```

Functions

Este é o tipo de procedimento que executa determinada rotina e retorna um valor, sendo que também é possível passar parâmetros para este.

```
Function RetornaNome( NOME )  
    RetornaNome = NOME  
End Function
```

Para chamar as procedures basta apenas referenciar o nome da procedure passando ou não seus respectivos parâmetros.

Poderá ser usada a declaração **"Call"** para chamar as procedures, sendo que isso não é necessário. Caso decida usar esta declaração os parametros deverão, obrigatoriamente, estar em parênteses.

2 - Retorno

Como já vimos acima, as **subs** não retornam valores, enquanto as **functions** retornam. Para o retorno de valores, em **functions**, atribui-se o valor que se deseja retornar à uma variável com o mesmo nome da função, sem os parênteses.

3 - Argumentos

É possível passar argumentos ou parâmetros para uma **function** ou **sub**. Eles devem ser declarados logo após o nome da função, entre parênteses, e tornam-se variáveis pertencentes ao escopo local da função.

Referência

Normalmente, a passagem de parâmetros em ASP é feita por valor, ou seja, se o conteúdo da variável for alterado, essa alteração não afeta a variável original.

```
Function mais5( NUMERO )  
    NUMERO = NUMERO + 5  
    Mais5 = NUMERO  
End Function  
  
NUMERO = 3  
Call mais5( NUMERO ); 'NUMERO continua valendo 3
```

No exemplo acima, como a passagem de parâmetros é por valor, a função **mais5** pode ser inútil, já que após a execução sair da função o valor anterior da variável é recuperado.

Caso a passagem de valor fosse feita por referência, a variável **NUMERO** teria 8 como valor. O que ocorre normalmente é que ao ser chamada uma função, o interpretador salva todo o escopo atual, ou seja, os conteúdos das variáveis.

Se uma dessas variáveis for passada como parâmetro, seu conteúdo fica preservado, pois a função irá trabalhar na verdade com uma cópia da variável.

Porém, se a passagem de parâmetros for feita por referência, toda alteração que a função realizar no valor passado como parâmetro afetará a variável que o contém.

A maneira de fazer com que uma função tenha parâmetros passados por referência e explicitando isso na declaração da mesma, o que faz com que a passagem de parâmetros seja assim.

Para isso utiliza-se o modificador **"byRef"**:

```
Function mais5( byRef NUMERO )  
    NUMERO = NUMERO + 5  
    Mais5 = NUMERO  
End Function  
  
NUMERO = 3  
Call mais5( NUMERO ); 'NUMERO passa a valer 8
```

4 - Contexto

O contexto é o conjunto de variáveis e seus respectivos valores num determinado ponto do programa. Na chamada de uma função, ao iniciar a execução do bloco que contém a implementação da mesma é criado um novo contexto, contendo as variáveis declaradas dentro do bloco, ou seja, todas as variáveis utilizadas dentro daquele bloco serão eliminadas ao término da execução da função.

5 - Escopo

O escopo de uma variável em ASP define a porção do programa onde ela pode ser utilizada. Na maioria dos casos todas as variáveis têm escopo global. Entretanto, em funções definidas pelo usuário um escopo local é criado. Uma variável de escopo global não pode ser utilizada no interior de uma função sem que haja uma declaração explícita.

```
teste = "Testando"  
  
function teste()  
    response.wtite(teste)  
end function
```

```
call teste()
```

O trecho acima não produzirá saída alguma, pois a variável **teste** é de escopo global, e não pode ser referida num escopo local, mesmo que não haja outra com nome igual que cubra a sua visibilidade.

Para que o script funcione da forma desejada, a variável global a ser utilizada deve ser declarada antecedida pela palavra reservada **Dim**.

```
Dim teste = "Testando"

function teste() {
    response.write(teste)
end function

call teste()
```

7 - Variáveis e Constantes

1 - Declaração

Você declara variáveis explicitamente em seu script usando a declaração Dim, a declaração Public e a declaração Private:

```
Dim nome
```

Pode-se também declarar variáveis múltiplas numa mesma linha separando-as por uma vírgula:

```
Dim nome, sobrenome, idade
```

Entretanto, não é necessária a declaração explícita em ASP.

2 - Variáveis enviadas pelo Navegador

Para interagir com a navegação do usuário, é necessário que o ASP possa enviar e receber informações para o browser.

A maneira de enviar informações é através de um comando de impressão, como o **`response.write()`**. Para receber informações vindas do navegador, através de um *link* ou um formulário html, o ASP utiliza as informações enviadas através da URL.

Se seu script ASP está localizado em **`http://localhost/teste.asp`** e você o chama com a url **`http://localhost/teste.asp?nome=paulo`**, você poderá recuperar automaticamente o valor do parametro **`"nome"`**, da url, através do comando:

```
request.querystring("nome")
```

...e se for um parametro vindo de um formulário HTML:

```
request.form("nome")
```

... onde **`"nome"`** será o nome ou id, do campo do formulario.

Note que o conteúdo da variável está no formato **urlencode**. Os formulários html já enviam informações automaticamente nesse formato, e o PHP decodifica sem necessitar de tratamento.

URLEncode

Este método aplica regras de codificação de URL, incluindo caracteres de escape.

```
Server.URLEncode(string)
```

HTMLEncode

Este método aplica codificação HTML.

```
Server.HTMLEncode(string)
```

3 - Verificando o Tipo

Estas funções pré-definidas permitem determinar o subtipo de uma variável ou mesmo expressão.

VarType (valor)

Retorna o código inteiro que corresponde ao seu tipo de dados, de acordo com a tabela abaixo:

Código	Tipo de Dados
0	Empty
1	Null
2	Integer
3	Long
4	Single
5	Double
6	Currency
7	Date
8	String
9	Object
10	Error
11	Boolean
12	Variant
13	Objeto de Acesso de Dados
14	Decimal
17	Byte
8192	Array

TypeName (valor)

Retorna uma string com o nome correspondente ao seu tipo de dados.

IsNumeric (valor)

Retorna um valor Booleano True se a expressão for um dado numerico e False se for um outro tipo.

IsArray (valor)

Retorna um valor Booleano True se a expressão for um Array e False se for outro tipo.

IsDate(valor)

Retorna um valor Booleano True se a expressão for um dado do tipo Date/Time e False se for outro tipo.

IsEmpty (valor)

Retorna um valor Booleano True se a expressão for um valor vazio ou variavel não inicializada e False em caso contrário.

IsNull (valor)

Retorna um valor Booleano True se a expressao não contiver um tipo de dado valido e False em caso contrário.

IsObject (valor)

Retorna um valor Booleano True se a expressão for um Objeto e False se for um outro tipo.

4 - Destruindo

As variáveis públicas(declaradas como Public) são automaticamente destruídas, ou desalocadas na memória do sistema, quando ocorre o término do script que as contém, enquanto as Privadas(declaradas como Private) são destruídas após a execução do procedimento que as contém.

5 - Definindo Constantes

Para declarar constantes usa-se a palavra reservada **Const**, conforme o exemplo:

```
Const Idade = 30           'Para numeros
Const Titulo = "Aprendendo Constantes" 'Para Strings
Const Aniversario = #29-5-1982#      'Para Datas
```

8 - Classes & Objetos

1 - Classes

Desde a versão 5.0 do VBScript, o ASP suporta o uso de classes através da palavra reservada **Class** oferecendo todo o poder da Orientação a Objeto:

```
Class myClass
    'Define a variavel valor como acesso privado a classe
    Private valor

    'Define uma Sub Publica que atribui a variável valor acima
    Public Sub setaValor( ByVal novoValor )
        valor = novoValor
    End Function

    'Define uma Function Publica que retorna o conteúdo da variavel valor
    Public Function displayValor()
        displayValor = "Valor: " & valor
    End Function
End Class
```

2 - Objetos

A Classe é a abstração do objeto. Cada instancia da classe será chamada de objeto, então, instanciar uma classe é referir-se a mesma. Cada objeto é declarado com a palavra reservada **New**.

```
Dim objeto
Set objeto = New myClass
```

É recomendável criar as classes em arquivos separados e instância-las de dentro dos arquivos onde você as usará, implementando com isso o conceito de reusabilidade, que é o sentido maior da orientação a objetos, conforme o exemplo abaixo:

```
<!-- #include file="myClass.asp" -->'Arquivo que contem a classe
<%
    Dim objeto
    Set objeto = New myClass 'Instancia a classe/Criando o objeto

    objeto.setaValor 10 'Usando o método setaValor do objeto
    Response.Write(objeto.displayValor) 'Usando o método displayValor
    set objeto = Nothing 'Destruindo o objeto
%>
```

Para usar a classe não é necessário saber como ela foi implementada ou que técnicas utiliza para oferecer as informações ou processamentos que utilizamos, basta saber quais os métodos que podem ser utilizados e quais parâmetros devem ser passados para os mesmos.

3 - Objetos Implícitos

Objeto Request

Este objeto é responsável pela obtenção de informações do cliente.

Este objeto possui algumas coleções, métodos e propriedades que nos possibilitam a manipulação de dados vindos do cliente:

Coleções

ClientCertificate

Mostra informações sobre o certificado de segurança do usuário e só pode ser usado em sites acessados com transmissão segura de dados(https).

```
Response.write "Data de expiração de seu certificado: "  
Response.write Request.ClientCertificate("ValidUntil")
```

A lista de parâmetros possíveis:

Subject	Informações sobre o certificado
Issuer	Informações sobre quem emite o certificado
ValidFrom	Data que o certificado tornou-se válido.
ValidUntil	Data de expiração do certificado
SerialNumber	Número serial do certificado
Certificate	Código binário completo
Flags	Informações Adicionais sobre o certificado

Cookies

Recupera informações gravadas em cookies, que são usados para persistência de informações do cliente.

```
Request.Cookies("USUARIO")("ID")
```

Onde USUÁRIO é o nome do cookie e ID é o campo do cookie a ser lido.

Form

Recupera dados vindos de formulários HTML com métodos de envio POST ou GET.

```
Request.Form("CIDADE")
```

Onde CIDADE é o nome do campo do formulário que está sendo recuperado.

QueryString

Recupera dados de strings URL ou de formulários HTML com metodos de envio GET.

```
Request.QueryString("CIDADE")
```

Onde CIDADE é o nome do campo do formulário ou do campo da URL a ser recuperado.

ServerVariables

Recupera dados do cabeçalho HTTP enviados tanto pelo cliente quanto pelo servidor.

```
Request.ServerVariables("PARAMETRO")
```

Onde o PARAMETRO pode ser:

Path_info

Caminho da URL requisitada no navegador.

Server_Name

Nome do servidor da URL da página.

Remote_Addr

IP do host remoto solicitante.

Remote_Host

Host do solicitante.

Logon_User

Conta que o usuário está logado no Windows NT.

http_Ua_OS

Nome e versão do software do servidor.

Server_Port

Número da porta na qual a requisição foi enviada.

Server_Protocol

Nome e versão do protocolo de informação.

Request_Addr

IP do host remoto que responde ao solicitante.

Request_Method

Método utilizado na requisição (GET ou POST).

Propriedades

TotalBytes

Recupera o número de bytes relacionado a solicitação atual.

```
Request.TotalBytes()
```

Métodos

BinaryRead

Conteúdo completo da solicitação do cliente ao servidor.

```
Request.BinaryRead();
```

Objeto Response

Este objeto é responsável pelo controle dos dados que serão enviados para o cliente.

Este objeto possui uma coleção que nos possibilita a criação de cookies no cliente:

Coleções

Cookies

Os Cookies são pequenos arquivos que são gravados no cliente para persistência de dados.

Os cookies são de dois tipos:

Cookies Permanentes

Duram até a data determinada para sua expiração.

Cookies de Sessão

Duram enquanto o navegador estiver aberto.

```
Response.Cookies("USUARIO")("NOME")
```

Onde USUÁRIO é o nome do arquivo a ser criado e NOME é o campo do cookie.

Propriedades

Expires

Permite especificar em minutos o tempo que durará o cookie, sendo que se este não for estipulado, o cookie deixará de existir assim que o navegador for encerrado.

```
Response.Cookies.Expires = "10"
```

ExpiresAbsolute

Permite especificar a data e hora absoluta que o cookie expirará.

```
Response.Cookies.ExpiresAbsolute = "December 18, 2003 8:00:00"
```

Status

Permite especificar o valor do status e a mensagem que é enviada ao navegador nos cabeçalhos http que indicam o resultado processamento das requisições.

```
Response.Status = "404 Page Not Found"
```

ContentType

Permite especificar o tipo de conteúdo que deverá ser enviado como resposta ao cliente.

```
Response.ContentType = "text/html"
```

Buffer

Permite especificar a forma como sera enviado ao navegador o conteudo gerado pelo seu script.

```
Response.Buffer = True
```

Onde "True" significa que o servidor somente enviará mensagens ao cliente após todo o script ASP ter sido processado. É importante que este comando seja escrito sempre antes de qualquer tag HTML e/ou qualquer saída gerada por Response.Write.

"False" é o valor padrão deste atributo.

Métodos

Write

Este é o método utilizado para o envio de textos ao cliente.

```
Response.Write "Este é meu texto..."
```

Redirect

Este método permite redirecionar o cliente para outra URL.

```
Response.Redirect "http://www.naredemundial.com.br"
```

É importante que este comando seja escrito sempre antes de qualquer tag HTML e/ou qualquer saída gerada por Response.Write.

AddHeader

Permite criar um cabeçalho personalizado usando valores especificados.

```
Response.AddHeader nome, conteúdo
```

Onde "nome" é a variável que será usada e conteúdo é o conteúdo que será gravado na variável.

Clear

Permite apagar o conteúdo do Buffer da página se este foi definido como true.

```
Response.Clear
```

End

Permite encerrar o processamento do script da página devolvendo o conteúdo criado.

`Response.End`

9 - Bancos de Dados

o ASP acessa bancos de dados através da tecnologia chamada de **ADO**, ActiveX Data Objects, que através do **OLEDB** fornece provedores que permitem o acesso a diversas fontes de dados, inclusive por **ODBC**, sendo que no caso de não existir um provedor **OLEDB** específico, para sua base de dados, ainda é possível através do **ODBC**, obter um driver que forneça o acesso à esta base de dados.

Com isso, na maioria das vezes será possível acessar os vários tipos de base de dados tanto através de **OLEDB** quanto através de **ODBC**, sendo que o ideal é usar a primeira visto que, através de **ODBC** ocorrerá sempre uma perda de performance.

Em termos de código de programação, a diferença entre **ODBC** E **OLEDB** é apenas no método de conexão como veremos a seguir, pois para realizar os SELECT'S os métodos não mudam.

Para acessar arquivos do **Access**, além de **ODBC**, também é possível usar o **Microsoft Jet**, que é um driver específico para este fim.

1 - MySQL

O ASP não possui um provedor de acesso direto para as bases de dados MySQL. A maneira de solucionar isto, é acessá-lo através de uma fonte de dados ODBC, pois para esta existe um driver chamado MySQL Connector que pode ser baixado em:

<http://www.mysql.com/downloads/api-myodbc-3.51.html>

2 - Microsoft SQL Server

Conectando

OLE DB

```
Set db = Server.CreateObject("ADODB.Connection")
db.Open "Provider=SQLOLEDB; Data Source=SERVER; Initial Catalog=database; User
ID=username; Password=password"
```

ODBC

```
Set db = Server.CreateObject("ADODB.Connection")
db.Open "DSN=mydsn;UID=username;PWD=password"
```

Seleccionando dados

```
set rs = db.execute("SELECT * FROM MyTable")
rs.MoveFirst
Do Until rs.EOF
    Response.Write rs("NAME")
    rs.MoveNext
Loop
```

Inserindo/Atualizando/Apagando

```
sql = "delete from clientes"
db.execute(sql)
```

10 - Cookies

Os cookies são usados para identificar um usuário. O cookie é um pequeno arquivo que o servidor grava no computador do usuário. Cada vez que o mesmo computador requisita uma página, o browser envia o cookie também. Com ASP, podemos gravar e recuperar dados de cookies.

1 - Criando

O comando **Response.Cookies** é usado para criar cookies, sendo que este comando deve vir antes de qualquer tag HTML.

```
<%  
Response.Cookies("name") = "João"  
%>
```

É possível setar as propriedades de um cookie também, como por exemplo a data que o mesmo expirará.

```
<%  
Response.Cookies("nome") = "João"  
Response.Cookies("nome").Expires = #May 10, 2002#  
%>
```

2 - Recuperando informações

O comando **Request.Cookies** é usado para recuperar informações de cookies.

```
<%  
nome = Request.Cookies("nome")  
response.write("Nome=" & nome)  
%>
```

3 - Cookies com chaves

Quando um cookie contiver uma coleção de múltiplos valores, podemos dizer que o Cookie possui chaves.

```
<%  
Response.Cookies("usuario")("nome") = "João"  
Response.Cookies("usuario ")("sobrenome")="da Silva"  
Response.Cookies("usuario ")("pais")="Brasil"  
Response.Cookies("usuario ")("idade")="25"
```


11 - Manipulação de Arquivos

Para manipular arquivos, pastas e drives o ASP utiliza o **File System Object**(FSO) que fornece todos os métodos necessários:

CopyFile	copiar arquivos.
CopyFolder	copiar pastas.
CreateFolder	criar pastas.
CreateTextFile	criar arquivos texto.
DeleteFile	deletar arquivos.
DeleteFolder	deletar pastas.
DriveExists	testar se drive existe.
FileExists	testar se arquivo existe.
FolderExists	testar se pasta existe.
GetAbsolutePathName	retornar o caminho completo.
GetDrive	retornar o drive.
GetDriveName	retornar o nome do drive.
GetFile	retornar o arquivo.
GetFileName	retornar o nome do arquivo.
GetFolder	retornar o folder.
GetParentFolderName	retornar o nome do folder pai.
GetTempName	criar e retornar o nome de um arquivo.
MoveFile	mover arquivos.
MoveFolder	mover pastas.
OpenTextFile	abrir um arquivo texto existente.

1 - Copiando

```
dim fs
set fs=Server.CreateObject("Scripting.FileSystemObject")
fs.CopyFile "c:\documentos\web\*.htm","c:\web\"
set fs=nothing
```

2 - Verificando Tamanho

```
Dim ObjFso, ObjArq

Set ObjFso = Server.CreateObject("Scripting.FileSystemObject")
Set ObjArq = ObjFso.GetFile(Server.MapPath("teste.asp"))

Response.Write "Tamanho do arquivo: " & ObjArq.Size

Set ObjFso = nothing
Set ObjArq = nothing
```

3 - Verificando se existe

```
dim fs
set fs=Server.CreateObject("Scripting.FileSystemObject")
if fs.FileExists("c:\documentos\teste.asp")=true then
    response.write("File c:\documentos\teste.asp existe!")
else
    response.write("File c:\documentos\teste.asp não existe!")
end if
```

```
set fs=nothing
```

4 - Abrindo Arquivos

Leitura

```
Const openForReading = 1
Const openForWriting = 2
Const openForAppending = 8

Dim fso, f, txtStream
Dim fileName, strText

fileName = Server.MapPath("piadas.txt")

Set fso = Server.CreateObject("Scripting.FileSystemObject")
Set f = fso.GetFile(fileName)
Set txtStream = f.OpenAsTextStream(openForReading)

Do While Not txtStream.AtEndOfStream
    strText = txtStream.ReadLine
    Response.Write strText & "<br>"
Loop

txtStream.Close
Set txtStream = Nothing
Set f = Nothing
Set fso = Nothing
```

Escrita

```
Dim fso
Dim txtStream, fileName

fileName = Server.MapPath("junk.txt")

Set fso = Server.CreateObject("Scripting.FileSystemObject")
Set txtStream = fso.CreateTextFile(fileName, True)

' Escrevendo no arquivo
txtStream.WriteLine "First line in the text file."
txtStream.WriteLine " "
txtStream.WriteLine "Line after two blank lines."
txtStream.Close

Set txtStream = Nothing
Set fso = Nothing
```

12 - Email

Para utilizarmos este recurso, em ASP, é necessário a utilização de componentes de terceiros instalados no servidor web, sendo que os mais conhecidos e/ou utilizados são o **AspMail** e o **CDonts**.

1 - AspMail

O exemplo abaixo serve como referência para a utilização padrão do componente AspMail:

```
<%  
Set Mailer = Server.CreateObject("SMTPsvg.Mailer")  
Mailer.FromName = "João da Silva"
```

Na linha acima vai o nome do remetente.

```
Mailer.FromAddress = "eu@dominio.com.br"
```

Na linha acima vai o endereço de email do remetente.

```
Mailer.RemoteHost = "servidor smtp"
```

Aqui vai o nome do seu servidor SMTP, normalmente fornecido pelo administrador de sua rede ou provedor de acesso. Caso o nome do servidor estiver errado nada funcionará.

```
Mailer.AddRecipient "José da Silva", "joao@provedor.com.br"
```

Aqui vai o Nome e Email do destinatário do email.

```
Mailer.ContentType = "text/html"
```

Na linha acima vai é configurado o formato que será enviado o email, pode ser HTML ou TEXTO.

```
Mailer.Subject = "Seu pedido foi recebido"
```

Aqui vai o Assunto do Email.

```
Mailer.BodyText = "Prezado José, este é um teste."
```

Aqui é o conteúdo, propriamente dito do email ou seja, o texto do mesmo.

```
If Mailer.SendMail Then
    Response.Write "Mensagem enviada com sucesso"
Else
    Response.Write "Erro " & Mailer.Response
End If
Set Mailer = Nothing
%>
```

2 - CDONTS

O exemplo abaixo serve como referência para a utilização padrão do componente CDONTS:

```
<%
Set Mailer = CreateObject("CDONTS.NewMail")
Mailer.From = "eu@dominio.com.br"
```

Na linha acima vai o email do remetente.

```
Mailer.To = "conta@destinatario.com.br"
```

Aqui vai o Email do destinatário.

```
Mailer.BodyFormat = 1
Mailer.MailFormat = cdoMailFormatText
```

Na linha acima vai é configurado o formato que será enviado o email, pode ser HTML ou TEXTO.

```
Mailer.Subject = "Assunto do E-mail"
```

Aqui vai o Assunto do Email.

```
Mailer.Body = "Corpo da Mensagem"
```

Aqui é o conteúdo, propriamente dito do email ou seja, o texto do mesmo.

```
Mailer.Send
Set Mailer = Nothing
%>
```

13 - ASP & Java

Através do ASP também podemos acessar classes Java e aproveitar suas características.

Para acessar classes Java o procedimento é bem semelhante como faríamos para instanciar objetos nativos.

```
<%  
'Instanciando uma classe Java que está num arquivo chamado noticias.class  
set javaObject = GetObject("java:noticias")  
  
'Lendo uma função vinda da classe Java  
resultado = javaObject.selectNoticias()  
  
'Imprimindo o conteúdo recebido da classe Java  
Response.Write(resultado)  
set javaObject = nothing  
%>
```

Para usar este recurso é preciso preparar o ambiente Java/ASP.

14 - Sessions

Em ASP, o objeto Session é usado para armazenar informações sobre a sessão do usuário. As variáveis armazenadas no Objeto Session mantem informações sobre um usuário e a torna disponível para todas páginas de uma aplicação.

1 - O Objeto Session

O ASP cria um novo Objeto Session para cada novo usuário e destroi o respectivo objeto quando a sessão termina.

2 - Quando a sessão começa?

A sessão começará quando:

3. Quando um novo usuário requisita um arquivo ASP e o arquivo Global.asa incluir uma Procedure Session_OnStart;
4. Quando um valor é armazenado numa variável Session;
5. Quando um novo usuário requisita um arquivo ASP e o arquivo Global.asa incluir a tag <object> para instanciar um objeto com o escopo de Sessão.

3 - Quando a sessão termina?

A sessão terminará se um usuário não requisitar ou atualizar uma pagina da aplicação por um determinado período. Por padrão, este período é de 20 minutos.

Se você quiser alterar este intervalo de timeout, poderá fazer isso através da propriedade **Timeout..**

```
<%  
Session.Timeout = 5
```

```
%>
```

Para encerrar uma sessão imediatamente ou forçar que esta termine, poderá fazer isso através do método **Abandon**.

```
<%  
Session.Abandon  
%>
```

O principal problema com sessões é saber quando elas devem ser encerradas. Não sabemos se a ultima requisição do usuario foi o final da sessão ou não. Sendo assim, não sabemos por quanto tempo manter a sessão ativa. Mantendo a sessão ativa por um longo tempo, esta estará consumindo recursos do servidor. Mas, se a sessão for deletada muito rapida corremos o risco de quando o usuario retornar ele ter que começar tudo de novo. Encontrar o **Timeout** adequado pode ser uma tarefa muito dificil.

4 - Gravando e Recuperando Variáveis de Sessão

O mais importante sobre o objeto Session é que podemos armazenar variaveis nele.

No exemplo abaixo setamos a variavel de sessão "usuario" com o nome "João da Silva" e "idade" para 30:

```
<%  
Session("usuario") = "João da Silva"  
Session("idade") = 30  
%>
```

Quando temos valores armazenados em variaveis de sessão podemos acessa-los de qualquer página da aplicação ASP.

```
<%  
Response.Write("Bem vindo " & Session("usuario"))  
%>
```

5 - Removendo Variáveis de Sessão

A coleção **Contents** contém todas as variaveis de sessão.

È possivel remover uma variavel de sessão com o método **Remove**.

No exemplo abaixo removemos a variavel de sessão "bebidas" se o valor da variavel de sessão "idade" for menor que 18:

```
<%
if Session.Contents("idade") < 18 then
    Session.Contents.Remove("bebidas")
end if
%>
```

Para remover todas as variáveis de uma sessão use o método

RemoveAll:

```
<%
Session.Contents.RemoveAll()
%>
```

6 - Acessando toda coleção Contents

A coleção **Contents** possui todas variáveis de sessão. Podemos fazer um **loop** na coleção **Contents** e ver tudo que ela contém:

```
<%
Session("usuario") = "João da Silva"
Session("age") = 30
dim i
for each i in Session.Contents
    Response.Write(i & "<br>")
next
%>
```

O resultado seria:

```
usuario
idade
```

Se você não conhecer o número de itens da coleção **Contents**, podemos usar a propriedade **Count**.

```
<%
dim i
dim j
j = Session.Contents.Count
Response.Write("Variáveis de Sessão: " & j)
for i=1 to j
    Response.Write(Session.Contents(i) & "<br>")
Next
%>
```

E o resultado seria:

```
Variaveis de Sessão: 2
João da Silva
30
```

7 - Acessando a coleção StaticObjects

Podemos fazer um **loop** na coleção **StaticObjects** para ver todos os objetos armazenados no objeto **Session**.

```
<%  
dim i  
for each i in Session.StaticObjects  
    Response.Write(i & "<br>")  
next  
>%
```

15 - Includes

Os includes em ASP funcionam da seguinte maneira:

```
<!--#include virtual="funcoes.asp"-->
```

ou

```
<!--#include file ="funcoes.asp"-->
```

Usa-se a palavra reservada **virtual** para indicar um arquivo a ser incluído, cujo caminho inicie por um diretório virtual do servidor.

```
<!-- #include virtual ="/mensagens/header.asp" -->
```

Usa-se a palavra reservada **file** para indicar um arquivo a ser incluído, cujo caminho seja relativo.

```
<!-- #include file ="mensagens\header.asp" -->
```

Usa-se a palavra reservada **file** com a sintaxe (`..\`) para incluir um arquivo de uma pasta localizada em nível superior ao mesmo.

16 - Tratamento de Erros

Os tratamentos de erros são muito importantes para que evitemos que o usuário receba mensagens indesejáveis ou confusas.

1 - ERR

No ASP existe um objeto interno utilizado para indicar situações de erro em tempo de interpretação: o objeto **ERR**. O objeto possui duas propriedades.

Description	Contém a mensagem original do erro
Number	Contém o número associado ao erro

Quando não fazemos uso desse objeto, os usuários do nosso site podem receber mensagens de erro "indecifráveis". A idéia então é escrever um código capaz de detectar os erros e enviar mensagens amigáveis aos usuários.

Para usar este objeto devemos adicionar o seguinte comando no início do nosso script: **ON ERROR RESUME NEXT**.

Esse comando informará ao ASP que, caso ocorra algum erro na interpretação de algum comando do script, deve-se descartar esse comando, atualizar o objeto **ERR** e executar a próxima linha.

Basta agora escrever código em qualquer lugar do script para verificar a ocorrência do erro para posterior tratamento.

```
<HTML>
<HEAD><TITLE>Curso de ASP</TITLE></HEAD>
<BODY>
<% On Error Resume Next
    Dim i
    For i=10 to 0 step -1    %>
        100 Dividido por <%=i%> = <%=(100/i)%><BR>
<% next
    if err then    %>
        <B>OCORREU UM ERRO [<%=Err.Number%>] : <%=Err.Description%>
<%end if%>
</BODY>
</HTML>
```

2 - ASPError

O objeto **ASPError** foi implementado no ASP 3.0 e somente está disponível no IIS5.

O objeto **ASPError** é usado para exibir informações detalhadas de quaisquer erros que venham a ocorrer nos scripts de páginas ASP. Este objeto é criado quando **Server.GetLastError** é chamado, assim estas informações podem somente ser acessadas usando o método **Server.GetLastError**.

```
<%  
'Esta cria um erro  
dim i for i=1 to 1 next  
  
'Chama o método GetLastError() para pegar o erro  
dim objerr  
set objerr=Server.GetLastError()  
  
'A variavel objerr agora possui o objeto ASPError  
response.write("ASP Code=" & objerr.ASPCode)  
response.write("<br>")  
response.write("Number=" & objerr.Number)  
response.write("<br>")  
response.write("Source=" & objerr.Source)  
response.write("<br>")  
response.write("Filename=" & objerr.File)  
response.write("<br>")  
response.write("LineNumber=" & objerr.Line)  
%>
```

As propriedades do objeto **ASPError** estão descritas abaixo, sendo que as mesmas são somente para leitura, pois estas não podem ser alteradas.

ASPCode

Retorna o código do erro gerado pelo IIS.

ASPDscription

Retorna a descrição detalhada do erro.

Category

Retorna a origem do erro.

Column

Retorna a posição da coluna do arquivo em que foi gerado o erro.

Description

Retorna uma breve descrição do erro.

File

Retorna o nome do arquivo ASP que gerou o erro.

Line

Retorna o número da linha onde o erro foi detectado.

Number

Retorna código de erro padrão COM.

Source

Retorna o código-fonte da linha onde o erro ocorreu.

17 - Global.asa

O arquivo Global.asa é um arquivo opcional, numa aplicação, que pode conter declarações de objetos, variáveis e métodos que podem ser acessados por cada página de sua aplicação ASP.

O arquivo **Global.asa** deve ser gravado no diretório **root** da aplicação ASP, sendo que cada aplicação pode ter somente um **Global.asa**.

1 - Eventos

No Global.asa podemos dizer para a aplicação e objetos da sessão o que fazer quando a aplicação e/ou sessão inicia e quando ela termina. O código para isto é colocado em **Eventos**. O arquivo Global.asa possui 4 tipos de eventos:

Application_OnStart

Este evento ocorre quando o primeiro usuário chama a primeira página da aplicação ASP. Este evento ocorre após o webserver ser restartado ou depois do **Global.asa** ser editado. O evento **"Session_OnStart"** ocorre imediatamente após este evento.

Session_OnStart

Este evento ocorre a cada vez que um novo usuário requisita a primeira página da aplicação ASP.

Session_OnEnd

Este evento ocorre a cada vez que o usuário finaliza uma sessão. O usuário termina a sessão após a página deixar de ser requisitada por um determinado tempo.

Application_OnEnd

Ese evento ocorre após o último usuário finalizar sua sessão. Tipicamente, este evento ocorre quando o webserver para. Este procedimento é usado para limpar configurações após a aplicação parar.

Abaixo um exemplo do arquivo Global.asa:

```
<script language="vbscript" runat="server">

sub Application_OnStart
'''some code
end sub

sub Application_OnEnd
'''some code
end sub

sub Session_OnStart
'''some code
end sub

sub Session_OnEnd
'''some code
end sub
</script>
```

JSP

Java Server pages

1 - Introdução

1 - O que é?

JSP é uma combinação de HTML e Java dentro de uma mesma página.

Através de Tags especiais podemos introduzir código Java em qualquer parte da página, e todo o código, fora destas tags especiais, é HTML puro.

2 - O que pode ser feito?

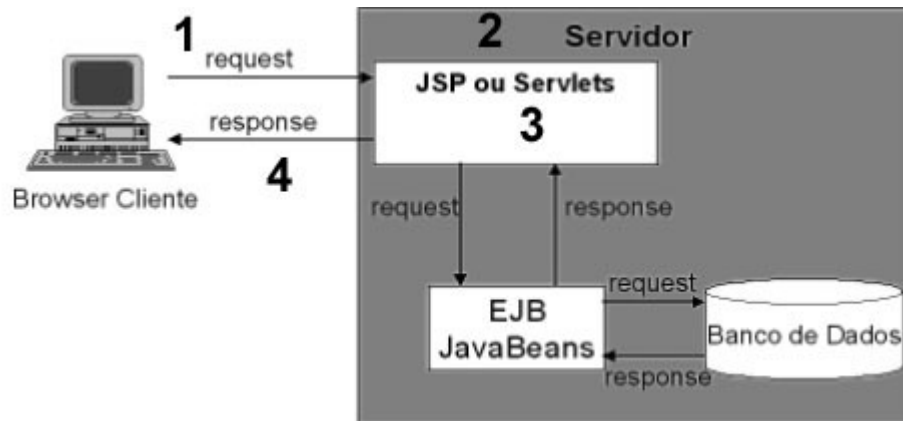
Basicamente, qualquer coisa pode ser feita com JSP desde coletar dados de formulários até gerar páginas dinâmicas oriundas de fontes de dados quaisquer.

4 - Diagrama

Uma requisição para o JSP funciona assim:

1. O **Browser** faz uma requisição ao **Servidor Web**;
2. O **Servidor Web** detecta que trata-se de uma página JSP e encaminha a requisição a Engine **JSP**;
3. A engine **JSP** faz os processamentos necessários, inclusive acessos a bancos de dados e outros recursos e devolve o HTML para o **Servidor Web**;
4. O **Servidor Web** devolve O HTML ao **Browser**.

No diagrama abaixo você visualiza estas etapas:



2 - Sintaxe Básica

1 - Delimitando o código

O código JSP, uma vez que, fica embutido em meio ao código HTML, para que o interpretador possa delimitar o que é JSP e o que é HTML precisa de tags que indiquem isso:

```
<%  
comandos  
%>
```

2 - Separando Instruções

Assim como na linguagem C, javascript e outras mais conhecidas, é necessário utilizar o "ponto-e-virgula" para sinalizar o fim de uma instrução ou comando.

3 - Nomes de Variáveis

É recomendável seguir as seguintes regras na codificação de programas em Java:

1. Não colocar mais do que uma instrução por linha. Uma instrução deve espalhar-se por múltiplas linhas para que nenhuma linha ocupe mais do 70 caracteres por linha para facilitar o entendimento.
2. Os nomes das classes devem começar por uma maiúscula. As letras seguintes devem ser minúsculas, exceto no início de novas palavras. Evitar o uso de abreviaturas e acrônimos.
3. Os nomes dos métodos devem ser verbos e começar com letras minúsculas. As letras subsequentes devem também ser minúsculas, exceto se forem a primeira letra de novas palavras.

4. Os nomes das variáveis também devem começar com letras minúsculas. As letras subsequentes devem também ser minúsculas, exceto se forem a primeira letra de novas palavras. Os nomes das variáveis devem ser curtos mas com maior significado possível. Nomes de variáveis com uma letra devem ser evitados sempre.
5. Variáveis declaradas como **final** e usadas como constantes devem ter nomes constituídos por letras maiúsculas e traços de união.
6. Ser consistente na colocação das chavetas, indentação, e espaços. Em caso de dúvida, siga o exemplo da codificação disponibilizada ou consulte:

<http://java.sun.com/docs/codeconv/>.
7. Usar bons nomes para as variáveis. Apesar desta directiva ser subjectiva, evite ser muito verboso ou sintético.
8. Não use métodos marcados como "deprecated" na documentação.
9. Remova o código que está obsoleto e não é usado, não se limite simplesmente a comentá-lo ou mesmo apagá-lo.

É preciso ressaltar, também, que o Java faz distinção entre letras maiúsculas e minúsculas, assim como no PHP.

Além disso, existem algumas palavras reservadas como `break` e `class`, por exemplo, que não podem ser usadas para nomear variáveis.

4 - Comentários

Existem dois tipos de comentários em JSP:

Comentários de uma linha

```
<% // comentários %>
```

ou

```
<!-- comentários --%>
```

Comentários de mais de uma linha

```
<%  
  /*  
    comentários  
  */  
%>
```

5 - Imprimindo código HTML

```
<%  
out.println("JSP imprimindo...");  
%>
```

3 - Tipos

O JSP, como é derivado do Java é fortemente tipado, ou seja, é necessário a declaração do tipo de dados que cada variável poderá manipular. Os tipos suportados veremos logo abaixo.

As declarações de variáveis consistem de um tipo e um nome de variável, como segue o exemplo:

```
int idade;           //Declara "idade" como inteiro.
String nome;         //Declara "nome" como sequência chars
boolean existe;      //Declara "existe" como true ou false
```

É possível que seja feita a inicialização do valor do variável no mesmo momento da declaração:

```
int idade = 10;      //Declara "idade" e inicializa com valor 10
```

1 - Tipos Suportados

As variáveis em Java podem ser definidas para aceitar tipos de dados primitivos ou referências a objetos.

Tipos de dados primitivos

Estes tipos são chamados primitivos porque não são objetos e são nativos no Java.

Estes tipos de dados primitivos são:

Integer

Armazena os números inteiros e possui 4 subtipos, sendo que destes o mais usado é o "int":

Byte	Armazena valores entre -128 e +127
Short	Armazena valores entre -32.768 e +32.767
int	Armazena valores entre -2147483648 e +2147483647
long	Armazena valores entre -9223372036854775808 e +9223372036854775807

Floating Point

São os valores numéricos com partes decimais e existem dois subtipos:

float	Armazena valores numéricos com 32 bits de precisão.
double	Armazena valores numéricos com 34 bits de precisão.

Character

Este é o tipo usado para os caracteres individuais:

char	Armazena caracteres.
------	----------------------

True/False

Boolean	Armazena valores binários "true, false" ou "0,1".
---------	---

Strings

Uma String é uma sequência de caracteres(char) e a classe "String" representa todas as strings em Java.

```
String nome = "Carlos Alberto";
```

Para comparar strings existem métodos especiais:

```
String senha = "teste";
if ( senha.equals("teste") ) {
    out.println("OK...");
}
```

ou assim, considerando "case sensitives":

```
String senha = "teSte";
if ( senha.equalsIgnoreCase("teste") ) {
    out.println("ERRO...");
}
```

Produzindo Strings a partir de primitivas:

```
String dez = String.valueOf(10);
```

Produzindo primitivos a partir de Strings

Existe uma wrapper class para cada tipo primitivo

```
String valor = "10";
int quantidade = Integer.parseInt(valor);
```

Arrays

Armazenam coleções de dados de um mesmo tipo sendo que o tamanho do mesmo deve ser definido durante a sua criação e este não poderá mais ser alterado.

Quando se declara um array ele terá valores "null" até que o mesmo seja inicializado usando a palavra reservada "new".

Um array pode armazenar tanto primitivas quanto objetos.

Declarando um array de primitivas:

```
int[] cores;  
int cores[];
```

Declarando um array de objetos apartir de um array de primitivas:

```
int[] cores;  
cores new int[4];
```

Inicializando:

```
cores[0] = "vermelho";
```

ou ainda:

```
int[] cores = {"vermelho","verde","azul"};
```

Vector

Armazenam coleções de dados de um mesmo tipo sendo que o tamanho do mesmo, diferente dos arrays, pode ser definido e/ou modificado a qualquer momento.

A classe Vector implementa um array redimensionavel de um tipo de objeto.

Criando um vector vazio:

```
Vector alunos = new Vector();
```

Criando um vector com um tamanho inicial:

```
Vector alunos = new Vector(10);
```

Adicionando elementos no fim do vector:

```
String nome = "Pedro";  
alunos.addElement(nome);
```

Adicionando elementos numa posição determinada do vector:

```
alunos.insertElement(name,0);
```

Removendo elementos numa posição determinada do vector:

```
alunos.removeElement(0);
```

Acessando o primeiro elemento de um vector:

```
String aluno = (String)alunos.firstElement();
```

Acessando um elemento em uma determinada posição:

```
String aluno = (String)alunos.elementAt(2);
```

Procura um objeto num vector:

```
int indice = alunos.indexOf(nome);
```

Pegando o tamanho de um vector:

```
int tamanho = alunos.size();
```

4 - Operadores

1 - Aritméticos

+	e	-	Operações de adição e subtração.
*	e	/	Operadores de multiplicação e divisão.
%			O módulo da divisão.

2 - Strings

+	Concatenação.
---	---------------

3 - Atribuição

=	Atribuição Simples
=+	Atribuição composta, atribuição e adição.
=-	Atribuição composta, atribuição e subtração.
=/	Atribuição composta, atribuição e divisão.
=*	Atribuição composta, atribuição e multiplicação.
=%	Atribuição composta, atribuição e módulo da divisão.

4 - Lógicos

&&	"E" lógico.
	"OU" lógico.
!	"NOT" lógico.

5 - Comparação

==	Igual
!=	Diferente
<	Menor que
>	Maior que
<=	Menor ou igual
>=	maior ou igual

6 - Expressão Condicional

Existe um operador de seleção que é ternário:

```
(expressão1)?(expressão2):(expressão3)
```

A primeira expressão é avaliada, se for verdadeira é executada a expressão 2, senão é executada a expressão 3.

7 - Incremento e Decremento

++	Incremento
--	Decremento

5 - Estruturas de Controle

1 - Blocos

Blocos em JSP são delimitados pelos caracteres "{" e "}". A utilização dos delimitadores de bloco em uma parte qualquer do código não relacionada com os comandos citados ou funções não produzirá efeito algum, e será tratada normalmente pelo interpretador.

2 - Condicionais

IF

Este é o comando mais trivial dos condicionais. Ele testa uma expressão e executa determinado comando se esta for verdadeira.

```
if ( expressão 1 ) {  
    comando;  
}
```

Existe um complemento para o IF, no caso da expressão 1 não for verdadeira, ele executa os comando do ELSE então:

```
if ( expressão 1 ) {  
    comando;  
} else {  
    comando alternativo;  
}
```

SWITCH

Neste comando condicional, o teste (um tipo primitivo de byte, char, short ou int) é comparado com cada valor em questão. Se um valor coincidente é achado, a instrução (ou instruções) depois do teste é executada. Se nenhum valor for encontrado, a instrução default é executada.

```
switch(idade) {  
    case 15:  
        out.println("quinze");  
    case 20:  
        out.println("vinte");  
}
```

```
case 30:
    out.println("trinte");
default:
    out.println("não informado...");
}
```

TRY...CATCH

Este condicional tenta primeiro executar os comandos do bloco "TRY", se por algum motivo ocorre alguma falha ou erro então é executado o comando do bloco "CATCH".

Este condicional é usado principalmente para a captura de erros.

```
try {
    comando 1;
} catch (Exception e) {
    comando 2;
}
```

3 - Repetição

WHILE

O *loop while* é usado para repetir uma instrução ou bloco de instruções até que uma condição particular seja verdadeira:

```
while(condição) {
    comando;
}
```

DO... WHILE

O loop *DO...WHILE* é exatamente como o loop *WHILE*, exceto pelo fato de que ele executará uma dada instrução ou bloco até que uma condição seja falsa. A diferença principal entre os dois é que os loops while testam a condição antes de iniciar o loop, tornando possível que o corpo do loop nunca seja executado caso a condição seja inicialmente falsa. Os *loop do while* executam pelo menos uma vez antes de testar a condição:

```
do {
    comando;
}
while(condição);
```

FOR

O *loop for*, repete uma instrução ou um bloco de instruções algum número de vezes até que a condição seja satisfeita.

Loops *for* são freqüentemente usados para simples iterações na qual você repete um bloco de instruções um certo número de vezes e então pára:

```
for(inicialização; teste ; incremento) {  
    comando;  
}
```

Quebras de Fluxo

Em todos os loops (*for*, *while* e *do*), o loop termina quando a condição que você está testando é atingida. Porém, em alguma situação, você desejará sair do loop antes do seu término normal. Para isso, você pode usar as palavras chaves *break* e *continue*.

BREAK

A palavra reservada ***break*** pára imediatamente a execução do loop corrente. Se você tiver loops aninhados dentro de loop mais externo; caso contrário, o programa simplesmente continua a execução da próxima instrução após o loop.

CONTINUE

A palavra reservada ***continue*** pára o loop e começa novamente na próxima iteração. ***Continue*** é útil quando você quer ter situações de casos especiais dentro de loop.

6 - Funções

1 - Definição

As funções em JSP, possuem algumas peculiaridades, ou seja, elas funcionam de maneira idêntica aos servlets.

A nomenclatura das funções obedecem as mesmas regras de nomenclatura das variáveis.

```
public String saudacao( String nome ){  
    return "Seja bem vindo " + nome;  
}
```

2 - Retorno

Todas as funções devem obrigatoriamente ser declaradas os tipos de dados que retornarão, mesmo que elas não retornem nada, o que no caso seria do tipo "void".

2 - Argumentos

Os argumentos que estas recebem também devem ser declarados com seus respectivos tipos.

3 - Escopo

As funções, junto a declaração de tipo também devem ter o escopo de acesso das mesmas, ou seja, "public" ou "private".

7 - Variáveis e Constantes

1 - Declaração

As variáveis em Java devem, obrigatoriamente, ser declaradas antes de serem usadas.

A declaração é feita com o tipo precedendo o nome da variável sendo que no momento de declaração também, poderá ser feito a inicialização dos valores.

```
String nome;  
int idade;
```

ou:

```
String nome = "Pedro";  
int idade = 25;
```

2 - Variáveis enviadas pelo Navegador

Para interagir com a navegação do usuário, é necessário que o JSP possa enviar e receber informações para o browser.

A maneira de enviar informações é através de um comando de impressão, como o **out.println()**. Para receber informações vindas do navegador, através de um *link* ou um formulário html, o JSP utiliza as informações enviadas através da URL.

Se seu script JSP está localizado em **"http://localhost/teste.jsp"** e você o chama com a url **http://localhost/teste.jsp?nome=paulo"**, você poderá recuperar automaticamente o valor do parametro **"nome"**, da url, através do comando:

```
request.getParameter("nome")
```

Note que o conteúdo da variável está no formato **urlencode**. Os formulários html já enviam informações automaticamente nesse formato, e o JSP decodifica sem necessitar de tratamento.

URLEncoder

Este método aplica regras de codificação de URL, incluindo caracteres de escape.

```
java.net.URLEncoder.encode(String)
```

HTMLEncoder

Este método aplica codificação HTML.

```
java.net.URLDecoder.decode(String)
```

3 - Variáveis de Ambiente

A classe **System** contém uma série de propriedades que definem os atributos do ambiente de trabalho.

Abaixo segue a lista deste atributos:

file.separator	Separador de arquivos (por exemplo, "/")
java.class.path	Caminho das classes Java
java.class.version	versão da classe Java
java.home	Diretório de instalação do Java
java.vendor	String Java vendor
java.vendor.url	URL Java vendor
java.version	Versão Java
line.separator	Separador de linhas
os.arch	Arquitetura do Sistema Operacional
os.name	Nome do Sistema Operacional
os.version	Versão do Sistema Operacional
path.separator	Separador de Caminhos (por exemplo, ":")
user.dir	Pasta de trabalho do usuário atual
user.home	Pasta de trabalho "home" do usuário atual
user.name	Nome do usuário

Os programas Java podem ler e gravar as propriedades do sistema através dos métodos da classe **System**.

4 - Definindo Constantes

Para declarar constantes em JSP usa-se o modificador **final** precedendo o nome da mesma. Recomenda-se usar os nomes de constantes em caixa alta para diferenciar das variáveis.

```
final int VALOR = 10;
```

8 - Classes e Objetos

1 - Classes

Uma classe é um conjunto de variáveis e funções relacionadas a essas variáveis. Uma vantagem da utilização de programação orientada a objetos é poder usufruir do recurso de encapsulamento de informação. Com o encapsulamento o usuário de uma classe não precisa saber como ela é implementada, bastando para a utilização conhecer a interface, ou seja, as funções disponíveis.

Uma classe é um tipo, e portanto não pode ser atribuída a uma variável. Para definir uma classe, deve-se utilizar a seguinte sintaxe:

```
public class frutas {  
    private string cor;  
    public boolean ok;  
}
```

1 - Usando uma variavel ou metodo de classe

Para referir-se a um metodo ou variavel de classe, junta-se o nome da classe e o nome do método ou variável com o símbolo de "ponto" (".").

2 - Usando uma variavel ou metodo de instância

Os métodos e variáveis que não são de classe são conhecidos como métodos e variáveis de instância.

Para referenciar os métodos e às variáveis de instancia, você deve referenciar os métodos e variáveis de objeto.

Você referencia os métodos e variáveis de instancia similarmente à maneira que você consulta aos métodos e variáveis da classe.

Você junta uma referência do objeto e o nome do método da instância ou da variável junto com o ponto(".").

3 - Classes Abstratas

Uma classe abstrata é uma classe que não pode ser instanciada diretamente sendo que os métodos abstratos devem ser implementados em suas subclasses.

A classe abstrata define dados e comportamentos que serão comuns a todas as subclasses que a herdarem.

Um método abstrato definido em uma classe abstrata deverá ser implementado por todas as suas subclasses sendo que esta técnica permite que ao modelar um objeto o projetista possa definir exatamente quais comportamentos serão implementados, somente que serão implementados

4 - Definindo classes abstratas

Usamos a palavra reservada ***abstract*** para declarar uma classe como abstrata.

```
public abstract class frutas {  
    private string cor;  
    public boolean ok;  
}
```

usando a classe abstrata acima:

```
public class pera extends frutas {  
    private string nome;  
}
```

5 - Definindo métodos abstratos

Para que os métodos de uma classe abstrata sejam obrigatoriamente implementados nas suas subclasses estes também devem ser declarados usando a palavra reservada ***abstract***, caso contrário suas subclasses não serão forçadas a isto.

```
public abstract class frutas {  
    public abstract boolean ok();  
}
```

6 - Interfaces

Uma interface é a especificação de uma coleção de métodos, semelhante a uma classe abstrata. Adicionalmente ao que a classe abstrata oferece, uma interface pode efetivamente implementar múltipla herança. Uma classe poderá implementar um número infinito de interfaces mas poderá implementar somente uma superclasse.

Numa interface todos seus métodos serão abstratos e todas suas variáveis serão *public static final*.

7 - Definindo interfaces

Usamos a palavra reservada *interface* para declarar uma interface.

```
public interface Ordenavel {  
    int comparar(Object valor);  
}
```

e utilizando:

```
public class numeros implements Ordenavel {  
    public int comparar(Object valor){  
        return valor;  
    }  
}
```

2 - this

Dentro do corpo do método de um objeto você pode apenas consultar diretamente às variáveis do próprio objeto.

Porém, às vezes você necessita referenciar o nome da variável membro, se um dos argumentos ao método tiver o mesmo nome.

Para o exemplo, o seguinte construtor para a classe de HSBCColor inicializa algumas de variáveis do membro de um objeto de acordo com os argumentos passados no construtor.

Cada argumento, do construtor, tem o mesmo nome que a variável do membro do objeto, cujo valor inicial o argumento contém.

```
class HSBCColor {
    int hue, saturation, brightness;
    HSBCColor (int hue, int saturation, int brightness) {
        this.hue = hue;
        this.saturation = saturation;
        this.brightness = brightness;
    }
}
```

Os nomes do argumento fazem exame da precedência e escondem variáveis do membro com o mesmo nome.

3 - SubClasses

Uma classe pode ser uma extensão de outra. Isso significa que ela herdará todas as variáveis e funções da outra classe, e ainda terá as que forem adicionadas. Em JSP não é permitido utilizar herança múltipla, ou seja, uma classe pode ser extensão de apenas uma outra.

Para criar uma classe extendida, ou derivada de outra, deve ser utilizada a palavra reservada **extends**:

```
public class Employee {

private String name;
private Date hireDate;
private Date dateOfBirth;
private String jobTitle;
private int grade;

}

public class Manager extends Employee {

private String departament;
private Employee [ ] subordinates;
}
}
```

Essa técnica é conhecida como herança e polimorfismo.

6 - Construtores

O mecanismo de inicialização do Java é automático, ou seja se não inicializarmos um construtor, o Java o inicializará automaticamente.

Mas existem casos que se faz necessário a declaração explícita dos construtores.

Para escrever um método que chama um construtor, você deve seguir duas regras:

1. O nome do método precisa ser igual ao nome da classe.
2. Não deve retornar um tipo declarado para o método.

9 - Bancos de Dados

JDBC é a interface padrão usada pelo Java para acessar bancos de dados relacionais.

Cada fabricante de bancos de dados fornece um ou mais drivers JDBC que devem ser instalados no servidor para que o Java possa fazer o respectivo acesso. Um driver JDBC implementa todas interfaces do pacote **Java.sql**, fornecendo assim, todo código necessário para acessar e manipular os dados.

1 - MySQL

Para acessar bases de dados MySQL você deve baixar o driver JDBC oficial para MySQL e instalá-lo.

<http://www.mysql.com/downloads/api-jdbc-stable.html>

Conectando

```
Class.forName("com.mysql.jdbc.Driver").newInstance();
java.sql.Connection conn;
conn = DriverManager.getConnection(
"jdbc:mysql://localhost/test?user=blah&password=blah");
```

Selecionando dados

```
String sql = "select * from clientes";
Statement stmt = conn.createStatement();
ResultSet rset = stmt.executeQuery(sql);
```

Inserindo/Atualizando/Apagando

```
String sql      = "delete from clientes";
Statement stmt  = conn.createStatement();
int linhas     = stmt.executeUpdate(sql);
```

2 - ODBC

Através de ODBC é possível acessar vários bancos de dados que possam não existir drivers para o Java. Alguns tipos de dados que podemos acessar através do ODBC é Access, arquivos texto, excel, mssql e outros.

Para usar este recurso o Java possui nativamente um driver chamado de ponte JDBC-ODBC.

Conectando

```
Connection conn = null;
try {
    Driver d = (Driver)Class.forName
("sun.jdbc.odbc.JdbcOdbcDriver").newInstance();
    String URL = "jdbc:odbc:" + db;
    conn = DriverManager.getConnection(URL, user, passw);
} catch (Exception e) {
    e.printStackTrace();
}
```

Selecionando dados

```
String sql = "select * from clientes";
Statement stmt = conn.createStatement();
ResultSet rset = stmt.executeQuery(sql);
```

Inserindo/Atualizando/Apagando

```
String sql      = "delete from clientes";
Statement stmt  = conn.createStatement();
int linhas      = stmt.executeUpdate(sql);
```

10 – Headers

Os comandos que enviam cabeçalhos de HTML para o cliente são implementados pelo objeto response, alguns deles são:

```
response.addHeader("tipo de cabeçalho");
```

Alguns exemplos de utilizações deste comando:

```
response.sendRedirect("http://www.naredemundial.com.br");
```

O comando acima direciona o usuário para a URL indicada.

```
response.addHeader("Pragma", "no-cache");  
response.addHeader("Cache-Control", "no-store");
```

O comando acima define a expiração da página.

Estes são somente os mais utilizados e para obter uma lista completa dos headers **HTTP** possíveis, visite o seguinte endereço na web:

```
http://www.w3.org/Protocols/rfc2068/rfc2068
```

11 - Cookies

Cookies são pequenos arquivos gravados no cliente por um determinado site, sendo que, somente este poderá recuperar as informações do mesmo.

Este recurso é muito útil para que determinadas informações sejam fornecidas pelo usuário apenas uma vez. Exemplos de utilização de cookies são os sites que informam a quantidade de vezes que você já visitou, ou alguma informação fornecida numa visita anterior.

1 - Criando

Este método cria o cookie.

```
Cookie c = new cookie( nome, valor);
```

Este método configura a idade máxima para o cookie. Nesse exemplo, o cookie que existe por 120 segundos(2 minutos). O argumento para **setMaxAge** é um valor inteiro. Isso permite que um cookie tenha uma idade máxima de até 2.147.483.647 segundos (ou aproximadamente 24.855 dias).

```
c.setMaxAge( 120 );
```

Este método adiciona o cookie à resposta para o cliente.

As informações de cabeçalho sempre são fornecidas primeiro ao cliente, então os cookies devem ser adicionados ao *response* com *addCookie* antes de quaisquer outras informações. Depois que o cookie é adicionado, o servlet envia um documento HTML para o cliente.

```
response.addCookie( c );
```

1 - Métodos

setComment(String)

O comentário que descreve o propósito do cookie que é apresentado pelo navegador para o usuário (alguns navegadores permitem que o usuário aceite cookies selecionando individualmente cada cookie).

setDomain(String)

Este método determina quais servidores podem receber o cookie. Por default os cookies são enviados para o servidor que originalmente enviou o cookie para o cliente. O domínio é especificado na forma ".nome.com" (onde nome é o nome do servidor, por exemplo: ".sun.com"), indicando que todos os servidores com finais ".nome.com" pode receber este cookie.

setMaxAge(int)

Configura a idade máxima do cookie em segundos.

setPath(String)

Configura o URL - "alvo" indicando os diretórios no servidor que levam os serviços que podem receber este cookie.

setSecure(boolean)

Um valor true indica que o cookie deve ser enviado apenas utilizando um protocolo seguro.

setValue(String)

Configura o valor de um cookie.

setVersion(int)

Configura o protocolo de cookie para esse cookie.

2 - Recuperando informações

recupera os cookies do cliente utilizando o método *getCookies* de **HttpServletRequest**, que retorna um *array* de *Cookies*.

Quando uma operação GET ou POST é realizada para invocar o JSP, os cookies associados a este servidor são automaticamente retornados para o JSP.

```
Cookies = request.getCookies();
```

Vários métodos de **Cookie** são fornecidos para manipular os membros de um **Cookie**. Alguns destes métodos estão listados no quadro a seguir:

1 - Métodos

getComment()

Retorna um String descrevendo o propósito do cookie (null se nenhum comentário foi configurado com setComment)

getDomain()

Retorna um String contendo o domínio do cookie. Isso determina quais servidores podem receber o cookie. Por default, os cookies são enviados para o servidor que originalmente enviou o cookie para o cliente.

getMaxAge()

Retorna um int representando a idade máxima do cookie em segundos.

getName()

Retorna um String contendo o nome do cookie como configurado pelo construtor.

getPath()

Retorna um String contendo o prefixo da URL para o cookie. Os cookies podem ser "destinados" para URLs específicos que incluem diretórios no servidor da Web. Por default, um cookie é retornado para serviços que operam no mesmo diretório que o serviço que enviou o cookie ou um subdiretório desse diretório.

getSecure()

Retorna um valor boolean indicando se o cookie deve ser transmitido utilizando um protocolo seguro (true).

getValue()

Retorna um String contendo o valor do cookie como configurado com setValue ou com o construtor.

getVersion()

Retorna um int contendo a versão do protocolo de cookie utilizada para criar o cookie. Atualmente os cookies estão sofrendo padronização. Um valor 0 (por default) indica o protocolo original de cookie como definido pelo Netscape. Um valor 1 indica a versão que atualmente está sendo padronizada.

12 - Manipulação de Arquivos

1 - Copiando

```
public boolean copyFile(String inFile, String outFile) {
    InputStream is = null;
    OutputStream os = null;
    byte[] buffer;
    boolean success = true;
    try {
        is = new FileInputStream(inFile);
        os = new FileOutputStream(outFile);
        buffer = new byte[is.available()];
        is.read(buffer);
        os.write(buffer);
    } catch (IOException e) {
        success = false;
    } catch (OutOfMemoryError e) {
        success = false;
    } finally {
        try {
            if (is != null) {
                is.close();
            }
            if (os != null) {
                os.close();
            }
        } catch (IOException e) {}
    }
    return success;
}
```

2 - Verificando Tamanho

```
File file = new File("meu_arquivo.txt");
long length = file.length();
```

3 - Verificando se existe

```
boolean exists = (new File("meu_arquivo.txt")).exists();
if (exists) {
    out.println("Arquivo existe!");
} else {
    out.println("Arquivo não existe!");
}
```

4 - Abrindo Arquivos

Leitura

```
String arquivo = "c:\\clientes\\cidades.txt";
File Arquivo = new File(arquivo);
if (Arquivo.exists()) {
    String c;
```

```
FileReader      txt      = new FileReader(arquivo);
BufferedReader textoTxt = new BufferedReader(txt);
while ( ( c = textoTxtTef.readLine() ) != null ){
    out.println("CIDADE : " + c + "<br>");
}
txt.close();
}
%>
```

Escrita

```
try {
    BufferedWriter out = new BufferedWriter(new FileWriter("arquivo.txt"));
    out.write("isto vai dentro do arquivo novo!");
    out.close();
} catch (IOException e) {
}
```

13 - Email

A manipulação de emails, com JSP, também não é uma tarefa muito simples, como acontece no PHP.

Mas, a função abaixo desempenha bem essa função, bastando apenas configurar a linha que diz **String mailServer**, com o seu servidor de envio de emails(smtp).

```
public boolean enviarMail(String from, String to, String subject, String body) {
    try {
        String mailServer = "smtp.server.com";
        Properties mailProps = new Properties();
        mailProps.put("mail.smtp.host", mailServer);
        Session mailSession = Session.getDefaultInstance(mailProps, null);
        int toCount = 1;
        InternetAddress[] toAddrs = new InternetAddress[toCount];
        for (int i=0; i<toCount; ++i) {
            toAddrs[i] = new InternetAddress (to);
        }
        InternetAddress fromAddr = new InternetAddress(from);
        Message message = new MimeMessage(mailSession);
        message.setFrom(fromAddr);
        message.setRecipients(Message.RecipientType.TO, toAddrs);
        message.setSubject(subject);
        message.setContent(body, "text/plain");
        Transport.send(message);
        return true;
    } catch (AddressException endereco) {
        return false;
    }
    catch (MessagingException mex) {
        return false;
    }
}
```

14 - JSP & Java (Servlets e Beans)

O JSP pode acessar diretamente classes Java em forma de servlets e/ou Beans.

Para exemplificar usaremos uma classe chamada **getSource.class** que implementa o recurso de ler arquivos **HTML** e devolver o código-fonte.

1 - A Classe

Este seria o código-fonte da classe getSource.class.

GetSource.java:

```
import javax.mail.*;
import javax.mail.internet.*;
import java.util.*;
import java.net.*;
import java.io.*;
import java.lang.*;
import javax.activation.*;
import java.lang.reflect.Array;
import java.lang.Class;

public class getSource {
    public String get_Source (String pagina) {
        DataInputStream dis;
        String source = "";

        try {
            URL u = new URL (pagina);
            dis = new DataInputStream
(u.openConnection().getInputStream());
            String s;
            while ( ( s = dis.readLine() ) != null ) {
                source += s;
            }

        } catch (Exception e){
            System.out.println(e);
        }
        return source;
    }
}
```

Ela deverá ser compilada e colocada no repositório de servlets e ou beans do seu servidor.

2 - O JSP

Este seria o JSP que instancia a classe Java como Bean.

GetSource.jsp:

```
<html>
<body>
<%@ page import="getSource" %>
<jsp:useBean id="getSource" scope="session" class="getSource"/>
<%
try {
    String pagina      = "modelo.html";
    String source_html = getSource.get_Source(pagina);
} catch (Exception e) {
    out.println("Erro: " + e);
}
%>
</body>
</html>
```

3 - O HTML

Este seria o arquivo HTML que será passado como parâmetro pelo JSP para que o Bean leia-o e devolva seu código-fonte.

modelo.html:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<html>
<head>
    <title>Untitled</title>
</head>
<body>

Este é meu arquivo de modelo...

</body>
</html>
```

15 - Sessions

Podemos usar variáveis de sessão e gerenciá-las facilmente com JSP. Sessions são semelhantes aos Cookies, porém, as Sessions ficam gravadas no servidor e não no computador do usuário.

1 - Métodos das Sessões

getId()

Retorna o valor do identificador único da sessão.

putValue()

Permite adicionar objetos com seus respectivos nomes na sessão (Certifique-se que o nome dado a um objeto é único, em todos os servlets que compartilham esta sessão).

getValue()

Permite recuperar os objetos contidos na sessão.

isNew()

Retorna true toda vez que existe uma nova sessão. Uma sessão é considerada nova (new) , quando ela foi criada pelo servidor mas o cliente ainda não tem conhecimento que ele foi associado a ela.

invalidate()

Faz com que a sessão seja imediatamente invalidada. Todas informações contidas no objeto serão perdidas. E o objeto será removido.

getCreationTime()

Retorna o tempo em que o objeto foi criado.

`getLastAccessedTime()`

Retorna o tempo no qual ocorreu o último pedido enviado pelo cliente associado a sessão.

Cada um destes métodos acima pode, em caso de sessão inválida, chamar a *`java.lang.IllegalStateException`*.

2 - Como invalidar a Sessão

Uma sessão do usuário pode ser invalidada manualmente ou, dependendo de onde o JSP seja executado, automaticamente.

Para que seja invalidada automaticamente uma sessão, utilize o método da sessão *`invalidate`*. Algumas aplicações têm um ponto natural no qual se invalida a sessão.

16 – Includes

Em JSP também podemos usufruir das vantagens de “incluir” arquivos em nosso scripts, sendo que isso pode ser feito de três maneiras:

```
<%@ include file="connectoracle.jsp" %>
```

ou assim:

```
<jsp:include page="ct_rs.jsp" flush="true"></jsp:include>
```

ou ainda assim, usando parâmetros:

```
<jsp:include page="scripts/login.jsp">  
    <jsp:param name="username" value="jsmith" />  
</jsp:include>
```

17 - Tratamento de Erros

Em JSP existem basicamente duas maneiras de fazer o tratamento dos erros que possam ocorrer no decorrer de nossos scripts.

1 - Diretiva Page

As páginas JSP fornecem um mecanismo elegante para a manipulação de exceções em tempo de execução.

É possível criar sua própria forma de tratamento de exceções dentro das páginas JSP, porém nem todas as situações podem ser previstas.

Ao fazer uso do atributo **errorPage** da diretiva *page*, é possível transferir uma exceção não tratada para uma página de erro JSP personalizada.

```
<%@ page isErrorPage="false" errorPage="meu_erro.jsp" %>
```

A linha acima informa ao JSP para transferir qualquer exceção que não tenha sido tratada para a página JSP **meu_erro.jsp**.

Para que isto funcione é necessário que a página **meu_erro.jsp** esteja com o parametro **isErrorPage** setado em **true**.

```
<%@ page isErrorPage="true" %>
```

2 - Exceptions

A classe de **Exceptions** define condições de erro moderados que seus programas podem encontrar.

Quando um erro acontece em seu programa, o código que acha o erro pode "disparar" uma exceção. Disparar uma exceção é o processo de sinalizar ao processo atual que um erro aconteceu.

Podemos capturar a exceção e quando possível recuperar a condução normal rotinas.

1 - Manipulação de Exceptions

Declarações try e catch

Uma maneira de manipular possíveis erros, é usando as declarações **try** e **catch**.

A declaração **try** indica a parte do código onde poderá ocorrer uma **exception**, sendo que para isso você deverá delimitar esta parte do código com o uso de chaves.

Na declaração **catch** você coloca o código a ser executado caso venha a ocorrer uma **exception**.

```
try {  
    // código que pode ocasionar uma exception  
} catch{  
    // tratamento do erro  
}
```

Declaração finally

A declaração **finally** é utilizada para definir o bloco que irá ser executado ocorrendo ou não uma **exception**, isto, após o uso da declaração **try** e **catch**.

```
try {  
    // código que pode ocasionar uma exception  
} catch{  
    // tratamento do erro  
}  
finally {  
    // código  
}
```

2 - Exceptions mais comuns

Os tipos de exceções mais comuns são as seguintes:

ArithmeticException

Ocorre quando tenta-se dividir qualquer numero por "zero".

```
int i = 12 / 0
```

NullPointerException

Ocorre quando tenta-se utilizar um objeto que não foi instanciado.

NegativeArraySizeException

Ocorre quando é atribuído um valor nulo para um array.

ArrayIndexOutOfBoundsException

Ocorre quando tenta-se acessar um elemento do array que não existe.

Aplicação de Exemplo

Agenda

Para exemplificarmos a aplicação prática do que foi abordado neste livro, desenvolveremos um simples Sistema de Agenda composto por 15 arquivos, descritos a seguir.

Como banco de dados usaremos uma fonte de dados ODBC que estará acessando um arquivo do Microsoft Access com uma tabela chamada **Agenda**, que também veremos.

1 - Arquivo Comum

Este é o arquivo que será usado como banco de dados pela fonte de dados ODBC que usaremos.

1 - agenda.mdb

Este é o arquivo do Access que usaremos como bancos de dados, convém lembrar que manipularemos os dados desta tabela através da fonte de dados ODBC.

Campo	Tipo (tamanho)
ID	auto-numeração
NOME	texto(100)
ENDERECO	texto(100)
DDD	texto(5)
FONE	texto(20)
EMAIL	texto(100)
OBSERVACOES	texto(255)
DATA	data/hora

2 - Menu

Este é o arquivo com o menu principal de nossa agenda e deve ser salvo como agenda.html, ele também será incluído em nossos scripts.

Em PHP seria assim:

```
<table width="100%" border=0 bgcolor=White>
  <tr align="center">
```

```
<td width=77 bgcolor="#EEE5DE">
  <a href="inserir.php">Inserir</a>
</td>
<td width=77 bgcolor="#EEE5DE">
  <a href="buscar.php">Buscar</a>
</td>
<td width=77 bgcolor="#EEE5DE">
  <a href="listar.php">Listar</a>
</td>
<td width=77 bgcolor="#EEE5DE">
  <a href="index.php">Sair</a>
</td>
</tr>
</table>
```

Em ASP seria assim:

```
<table width="100%" border=0 bgcolor=White>
<tr align="center">
  <td width=77 bgcolor="#EEE5DE">
    <a href="inserir.asp">Inserir</a>
  </td>
  <td width=77 bgcolor="#EEE5DE">
    <a href="buscar.asp">Buscar</a>
  </td>
  <td width=77 bgcolor="#EEE5DE">
    <a href="listar.asp">Listar</a>
  </td>
  <td width=77 bgcolor="#EEE5DE">
    <a href="index.asp">Sair</a>
  </td>
</tr>
</table>
```

Em JSP seria assim:

```
<table width="100%" border=0 bgcolor=White>
<tr align="center">
  <td width=77 bgcolor="#EEE5DE">
    <a href="inserir.jsp">Inserir</a>
  </td>
  <td width=77 bgcolor="#EEE5DE">
    <a href="buscar.jsp">Buscar</a>
  </td>
  <td width=77 bgcolor="#EEE5DE">
    <a href="listar.jsp">Listar</a>
  </td>
  <td width=77 bgcolor="#EEE5DE">
    <a href="index.jsp">Sair</a>
  </td>
</tr>
</table>
```

3 - Funções

Este é o arquivo que usaremos para armazenar as variáveis, constantes e funções comuns aos demais scripts que por sua vez será "incluído" nos mesmos.

funcoes.php:

```
<?
/*
Função      : Fornecer variáveis, constantes e funções comuns
Desenvolvedor : Cerli Rocha
Data       : 30/05/2003
Atualização : 30/06/2003
*/

define("BANCO","AGENDA"); //Constante com o nome da base de dados
define("USUARIO","agenda"); //Constante com o nome do usuário da base de dados
define("SENHA","agenda"); //Constante com a senha do usuário da base de dados

//Função que executa a conexão a base de dados
function conecta( $BANCO, $USER, $PASS ) {
    global $con;
    $con = @odbc_connect($BANCO,$USER,$PASS);
}
?>
```

funcoes.asp:

```
<%
'Função      : Fornecer variáveis, constantes e funções comuns
'Desenvolvedor : Cerli Rocha
'Data       : 30/05/2003
'Atualização : 30/06/2003

Const BANCO  = "AGENDA"
Const USUARIO = "agenda"
Const SENHA  = "agenda"
Dim con

function conecta( BANCO, USUARIO, SENHA, byRef con )
    Set con = Server.CreateObject("ADODB.Connection")
    con.Open "DSN=" & BANCO & ";UID=" & USUARIO & ";PWD=" & SENHA
end function
%>
```

funcoes.jsp:

```
<%!
/*
Função      : Fornecer variáveis, constantes e funções comuns
Desenvolvedor : Cerli Rocha
Data       : 30/05/2003
Atualização : 30/06/2003
*/

public final String USUARIO = "agenda";
public final String SENHA   = "agenda";

public boolean autentica_usuario( String usuario, String senha ) {

    boolean valor = false;

    if ( ( usuario.equals(USUARIO) ) && ( senha.equals(SENHA) ) ) {
        valor = true;
    }
}
```

```

        } else {
            valor = false;
        }
        return valor;
    }

    Connection con = null;

%>
<%

    String driverName = "sun.jdbc.odbc.JdbcOdbcDriver";
    String serverURLBase = "jdbc:odbc:";
    String dbName = "Agenda";

//Conecta
try {
    Class.forName(driverName);
    con = DriverManager.getConnection(serverURLBase+dbName);
} catch (Exception e){

    }
//Fim do conecta
%>

```

4 - Index

Este é a tela inicial da nossa agenda, apenas exibe a tela para login e envia para o script que faz a validação dos dados.

index.php:

```

<?
/*
Função : Exibe a tela para login
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

session_start();                //inicializa a sessão
$_SESSION['USUARIO'] = NULL;    //Limpa a sessão, caso exista

//Abaixo o HTML
?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899">
<form name="form1" action="login.php" method=post>
<table width="400" border="0" align="center">
    <tr>
        <td bgcolor=white>
            <table width="100%" border="0" align="center">
                <tr>

```

```

        <td align=right colspan=2>.: <b>Bem Vindo. Por favor informe
seus dados</b> :.</td>
        </tr>
        <tr>
        <td align=right width="30%"><b>Login</b>: &nbsp;</td>
        <td><input name="LOGIN" type=text id="LOGIN"
style="width:80px;" value="" size=5></td>
        </tr>
        <tr>
        </tr>
        <td align=right width="30%"><b>Senha</b>: &nbsp;</td>
        <td><input name="SENHA" type=password id="SENHA"
style="width:80px;" value="" size=5></td>
        </tr>
        <tr>
        <td>&nbsp;</td>
        <td><input type=submit name=submit value="Acessar
&gt;&gt;"></td>
        </tr>
    </table>
</td>
</tr>
</table>
</form>
</body>
</html>

```

index.asp

```

<!-- #include file="funcoes.asp" -->
<%
'Função : Exibe a tela para login
'Desenvolvedor : Cerli Rocha
'Data : 30/05/2003
'Atualização: 30/06/2003

Session.Abandon
%>
<html>
<head>
    <title>:: Agenda ASP ::</title>

</head>
<body bgcolor="#778899" >
<form name="form1" action="login.asp" method=post>
<table width="400" border="0" align="center">
    <tr>
        <td bgcolor=white>
            <table width="100%" border="0" align="center">
                <tr>
                    <td align=right colspan=2>.: <b>Bem Vindo. Por favor informe
seus dados</b> :.</td>
                    </tr>
                    <tr>
                        <td align=right width="30%"><b>Login</b>: &nbsp;</td>
                        <td><input name="LOGIN" type=text id="LOGIN"
style="width:80px" value="" size=5></td>
                    </tr>
                    <tr>
                        </tr>
            </table>
        </td>
    </tr>
</table>

```

```

                <td align=right width="30%"><b>Senha</b>: &nbsp;</td>
                <td><input name="SENHA" type=password id="SENHA"
style="width:80px" value="" size=5></td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td><input type=submit name=submit value="Acessar
&gt;&gt;"></td>
            </tr>
        </table>
    </td>
</tr>
</table>
</form>
</body>
</html>

```

index.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%
/*
Função : Exibe a tela para login
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

String usuario = (String)session.getAttribute("USUARIO");

if ( ( usuario != null ) ) {
%>
<html>
<head>
    <title>:: Agenda JSP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
            <%@ include file="menu.html" %>
        </td></tr>

    <td valign=top>
        <form name="form1" action="inserir_agenda.jsp" method=post>
            <table width="100%" border="0" align="center">
                <tr>
                    <td bgcolor=white> <table width="100%" border="0" align="center">
                        <tr>
                            <td align=right colspan=2>.: <b>Inserir Nomes
                                na Agenda</b> :.</td>
                        </tr>
                        <tr>
                            <td align=right width="30%"><b>Nome</b>: &nbsp;</td>
                            <td><input name="NOME" type=text id="NOME" value="" size=30
maxlength="50"></td>
                        </tr>
                    </table>
                </tr>
            </table>

```

```

                <td align=right width="30%"><b>Endere&ccedil;o</b>:
                &nbsp;</td>
                <td><input name="ENDERECO" type=text id="ENDERECO" value=""
size=30 maxlength="100"></td>
            </tr>
            <tr>
                <td><b>Fone</b>: </td>
                <td><input name="DDD" type=text id="DDD" value="" size=3
maxlength="3">
                -
                <input name="FONE" type=text id="FONE" value="" size=7
maxlength="10"></td>
            </tr>
            <tr>
                <td><b>Email</b>: </td>
                <td><input name="EMAIL" type=text id="EMAIL" value="" size=30
maxlength="50"></td>
            </tr>
            <tr>
                <td><b>Observa&ccedil;&otilde;es</b>: </td>
                <td><textarea name="OBSERVACOES" cols="22" rows="5"
id="OBSERVACOES"></textarea></td>
            </tr>
            <tr>
                <td>&nbsp;</td>
                <td><input type=submit name=submit value="Gravar &gt;&gt;";
                <input type=reset name=submit2 value="Limpar &gt;&gt;";></td>
            </tr>
        </table></td>
    </tr>
</table>
</body>
</html>
<%
    } else {
        response.sendRedirect("index.jsp");
    }
%>

```

5 - Login

Este é o script que autentica o usuário, ou seja, se o usuario é válido ele abre realmente a Agenda, senão direciona para a tela inicial(listar).

login.php:

```

<?
/*
Função : Autentica o usuario
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

```

```

session_start();          //inicializa a sessão
include("funcoes.php"); //inclui o arquivo de funções

//Função que autentica o usuario
function autentica_usuario( $USUARIO, $SENHA ) {
    global $con;
    $valor = false;

    //Compara o usuario e senha recebidos com as constantes definidas no
//funcoes.php
    if ( ( $USUARIO == USUARIO ) && ( $SENHA == SENHA ) ) {
        $valor = true;
    } else {
        $valor = false;
    }

    return $valor; //Retorna o resultado
}

//Corpo Principal do Script
$login = trim($_POST['LOGIN']); //Recebe o login
$senha = trim($_POST['SENHA']); //recebe a senha

if ( ( $login!="" ) && ( $login!=NULL ) ) { //Testa o login

    //chama a função autentica_usuario
    if ( autentica_usuario($login,$senha) ) {
        $_SESSION['USUARIO'] = $login; //Grava o login na sessão
        header("location: listar.php"); //Direciona para a agenda
    } else {
        header("location: index.php"); //Direciona para a tela inicial
    }

} else {
    header("location: index.php"); //Direciona para a tela inicial
}
?>

```

login.asp

```

<!-- #include file="funcoes.asp" -->
<%
'Função : Autentica o usuario
'Desenvolvedor : Cerli Rocha
'Data : 30/05/2003
'Atualização: 30/06/2003

function autentica_usuario( LOGIN, PASS )
    valor = false

    if ( ( LOGIN = USUARIO ) AND ( PASS = SENHA ) ) then
        valor = true
    else
        valor = false
    end if

    autentica_usuario = valor

```

```
end function

'Corpo Principal do Script
login = Request.Form("LOGIN")
pass = Request.Form("SENHA")

if ( ( login <> "" )AND( NOT(isNull(login)) ) ) then

    if ( autentica_usuario(login,pass) ) then
        Session("USUARIO") = login
        Response.Redirect("listar.asp")
    else
        Response.Redirect("index.asp")
    end if

else
    Response.Redirect("index.asp")
end if
%>
```

login.jsp

```
<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%
/*
Função : Autentica o usuario
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

//Corpo Principal do Script
String login = request.getParameter("LOGIN");
String senha = request.getParameter("SENHA");

if ( login != null ) {

    if ( autentica_usuario(login,senha) ) {
        session.setAttribute("USUARIO",login);
        response.sendRedirect("listar.jsp");
    } else {
        response.sendRedirect("index.jsp");
    }

} else {
    response.sendRedirect("index.jsp");
}
%>
```

6 - Listar

Esta na verdade é a tela principal de nossa aplicação, pois ela listará todos os registros da nossa agenda, exibindo as opções para inserir, buscar, editar, excluir e sair.

listar.php:

```
<?
/*
Função : Lista os nomes cadastrados na agenda
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//Lista os nomes da agenda
function listar_agenda() {
    global $con;
    $valor = "";
    $sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = 0;
    } else {
        while( odbc_fetch_row( $result ) ) {
            $valor .= "<tr>
                        <td class=\"default\"><a
ref=\"editar.php?ID=\".odbc_result( $result, 1 ).\">Editar</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=350,height=200')\">.odbc_result( $result, 2 ).\"</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=350,height=200')\">.odbc_result( $result, 6 ).\"</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=350,height=200')\">(\".odbc_result( $result, 4 )
.\")\".odbc_result( $result, 5 ).\"</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('excluir.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=1,height=1')\">Excluir</a></td>
                        </tr>";
        }
    }
    return $valor;
}

//Corpo principal do script
$usuario = trim($_SESSION['USUARIO']); //recbe os dados da sessão

if ( ( $usuario != "" ) && ( $usuario != "" ) ) { //Testa o usuário

    if ( !$con ) { //Testa se existe conexão
        //Conecta ao banco
        conecta(BANCO,USUARIO,SENHA); //Conectando ao banco
    }

    //Grava a saída da função na variável
    $linhas = listar_agenda();
}

//Abaixo o HTML
```

```

?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
<?
//Incluindo o Menu
include("menu.html");
?>

        </td></tr>
        <td valign=top>
            <table cellspacing=1 cellpadding=1 width="100%" border=0
bgcolor=White>
                <tr>
                    <td width="60" class="headers"><b><a href="#">Editar</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Nome</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Email</a></b></td>
                    <td width="100" class="headers"><b><a href="#">Fone</a></b></td>
                    <td width="60" class="headers"><b><a href="#">Excluir</a></b></td>
                </tr>
                <?=$linhas?>
            </table>
        </td>
    </tr>
</table>
</body>
</html>
<?
    } else {
        //Direciona para a tela inicial
        header("location: index.php");
    }
?>

```

listar.asp:

```

<!-- #include file="funcoes.asp" -->
<%
'Função : Lista os nomes cadastrados na agenda
'Desenvolvedor : Cerli Rocha
'Data : 30/05/2003
'Atualização: 30/06/2003

function listar_agenda()
    valor = ""
    sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
Data FROM Agenda"
    Set rs = Server.CreateObject("ADODB.Recordset")
    rs.Open sql, con
    Do While Not rs.EOF
        valor = valor & "<tr>" & _
            "<td class=default><a
href=editar.asp?ID=" & rs(0) & ">Editar</a></td>" & _

```

```

                                "<td class=default><a href=#
onClick=window.open('detalhes.asp?ID=" & rs(0) &
"', 'Detalhes', 'width=350,height=200')>" & rs(1) & "</a></td>" & _
                                "<td class=default><a href=#
onClick=window.open('detalhes.asp?ID=" & rs(0) &
"', 'Detalhes', 'width=350,height=200')>" & rs(5) & "</a></td>" & _
                                "<td class=default><a href=#
onClick=window.open('detalhes.asp?ID=" & rs(0) &
"', 'Detalhes', 'width=350,height=200')>(" & rs(3) & ")" & rs(4) & "</a></td>" & _
                                "<td class=default><a href=#
onClick=window.open('excluir.asp?ID=" & rs(0) &
"', 'Detalhes', 'width=1,height=1')>Excluir</a></td>" & _
                                "</tr>"

                                rs.MoveNext
                                Loop
                                rs.Close
                                set rs = Nothing
                                listar_agenda = valor
end function

login = Session("USUARIO")

if ( ( login <> "" ) AND ( NOT(isNull(login)) ) ) then

    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if

    linhas = listar_agenda()
%>
<html>
<head>
    <title>:: Agenda ASP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center><!-- #include
file="menu.html" --></td></tr>
        <td valign=top>
            <table cellspacing=1 cellpadding=1 width="100%" border=0
bgcolor=White>
                <tr>
                    <td width="60" class="headers"><b><a href="#">Editar</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Nome</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Email</a></b></td>
                    <td width="100" class="headers"><b><a href="#">Fone</a></b></td>
                    <td width="60" class="headers"><b><a href="#">Excluir</a></b></td>
                </tr>
                <%=linhas%>
            </table>
        </td>
    </tr>
</table>
</body>
</html>
<%
else

```

```
Response.Redirect("index.asp")
end if
%>
```

listar.jsp:

```
<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
Função : Lista os nomes cadastrados na agenda
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

public String listar_agenda() {

    String valor = "";
    String sql = "";

    String id = "";
    String nome = "";
    String endereco = "";
    String ddd = "";
    String fone = "";
    String email = "";
    String observacoes = "";

    Statement stmt;
    ResultSet rs;

    //Select
    try {
        sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda";
        stmt = con.createStatement();
        stmt.execute(sql);
        rs = stmt.getResultSet();
        while(rs.next()) {
            id = rs.getString(1);
            nome = rs.getString(2);
            endereco = rs.getString(3);
            ddd = rs.getString(4);
            fone = rs.getString(5);
            email = rs.getString(6);
            observacoes = rs.getString(7);

            valor += "<tr>" +
                "<td class=\"default\"><a"
href=\"editar.jsp?ID=\" + id + "\">Editar</a></td>" +
                "<td class=\"default\"><a"
href=\"#\" onClick=\"window.open('detalhes.jsp?ID=\" + id +
\", 'Detalhes', 'width=350,height=200')\">" + nome + "</a></td>" +
                "<td class=\"default\"><a"
href=\"#\" onClick=\"window.open('detalhes.jsp?ID=\" + id +
\", 'Detalhes', 'width=350,height=200')\">" + email + "</a></td>" +
                "<td class=\"default\"><a"
href=\"#\" onClick=\"window.open('detalhes.jsp?ID=\" + id +
\", 'Detalhes', 'width=350,height=200')\">(" + ddd + ")" + fone + "</a></td>" +
```

```

                                "<td class=\"default\"><a
href=\"#\" onClick=\"window.open('excluir.jsp?ID=" + id +
"', 'Detalhes', 'width=1,height=1')\">Excluir</a></td>" +
                                "</tr>";

        }
        stmt.close();
        if (!con.isClosed()) {
            con.close();
        }
    } catch (Exception e) {
        try {
            if (!con.isClosed()) {
            }
        } catch (Exception ex){
        }
    }
    return valor;
}
%>
<%

String usuario = (String)session.getAttribute("USUARIO");

if ( ( usuario != null ) ) {

    String linhas = listar_agenda();
%>
<html>
<head>
    <title>:: Agenda JSP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
<%@ include file="menu.html" %>

        </td></tr>
        <td valign=top>
            <table cellspacing=1 cellpadding=1 width="100%" border=0
bgcolor=White>
                <tr>
                    <td width="60" class="headers"><b><a href="#">Editar</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Nome</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Email</a></b></td>
                    <td width="100" class="headers"><b><a href="#">Fone</a></b></td>
                    <td width="60" class="headers"><b><a href="#">Excluir</a></b></td>
                </tr>
                <%=linhas%>
            </table>
        </td>
    </tr>
</table>
</body>
</html>
<%
} else {

```

```

    response.sendRedirect("index.jsp");
}
%>

```

7 - Inserir

Esta tela monta o formulário para inserir novos nomes na agenda e envia para o script de inserir.

inserir.php

```

<?
/*
Função : Monta a tela para inserir nomes na agenda
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

session_start(); //Inicializa a Sessão
include("funcoes.php"); //inclui o arquivo de funções

//Corpo principal do script
$usuario = trim($_SESSION['USUARIO']); //Recebe os valores da sessão

//Testa o usuário
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {
//Abaixo o HTML
?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
            <?
include("menu.html");
?>

        </td></tr>

        <td valign=top>
            <form name="form1" action="inserir_agenda.php" method=post>
                <table width="100%" border="0" align="center">
                    <tr>
                        <td bgcolor=white> <table width="100%" border="0" align="center">
                            <tr>
                                <td align=right colspan=2>.: <b>Inserir Nomes
                                    na Agenda</b> :.</td>
                            </tr>
                            <tr>
                                <td align=right width="30%"><b>Nome</b>: &nbsp;  </td>
                                <td><input name="NOME" type=text id="NOME" value="" size=30
maxlength="50"></td>
                            </tr>

```

```

        <tr> </tr>
        <td align=right width="30%"><b>Endere&ccedil;o</b>:
        &nbsp;</td>
        <td><input name="ENDereco" type=text id="ENDereco" value=""
size=30 maxlength="100"></td>
        </tr>
        <tr>
        <td><b>Fone</b>: </td>
        <td><input name="DDD" type=text id="DDD" value="" size=3
maxlength="3">
        -
        <input name="FONE" type=text id="FONE" value="" size=7
maxlength="10"></td>
        </tr>
        <tr>
        <td><b>Email</b>: </td>
        <td><input name="EMAIL" type=text id="EMAIL" value="" size=30
maxlength="50"></td>
        </tr>
        <tr>
        <td><b>Observa&ccedil;&otilde;es</b>: </td>
        <td><textarea name="OBSERVACOES" cols="22" rows="5"
id="OBSERVACOES"></textarea></td>
        </tr>
        <tr>
        <td>&nbsp;</td>
        <td><input type=submit name=submit value="Gravar &gt;&gt;";
        <input type=reset name=submit2 value="Limpar &gt;&gt;";</td>
        </tr>
        </table></td>
    </tr>
</table>
</body>
</html>
<?
    } else {
        header("location: index.php");
    }
?>

```

inserir.asp

```

<!-- #include file="funcoes.asp" -->
<%
'Função : Monta a tela para inserir nomes na agenda
'Desenvolvedor : Cerli Rocha
'Data : 30/05/2003
'Atualização: 30/06/2003

login = Session("USUARIO")

if ( ( login <> "" )AND( NOT(isNull(login)) ) ) then

%>
<html>
<head>
    <title>:: Agenda ASP ::</title>

```

```

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
  <tr>
    <td valign=top width="100%" align=center><!-- #include
file="menu.html" --></td></tr>

  <td valign=top>
    <form name="form1" action="inserir_agenda.asp" method=post>
      <table width="100%" border="0" align="center">
        <tr>
          <td bgcolor=white> <table width="100%" border="0" align="center">
            <tr>
              <td align=right colspan=2>.: <b>Inserir Nomes
na Agenda</b> :.</td>
            </tr>
            <tr>
              <td align=right width="30%"><b>Nome</b>: &nbsp;</td>
              <td><input name="NOME" type=text id="NOME" value="" size=30
maxlength="50"></td>
            </tr>
            <tr> </tr>
            <td align=right width="30%"><b>Endere&ccedilo</b>:
&nbsp;</td>
            <td><input name="ENDERECO" type=text id="ENDERECO" value=""
size=30 maxlength="100"></td>
            </tr>
            <tr>
              <td><b>Fone</b>: </td>
              <td><input name="DDD" type=text id="DDD" value="" size=3
maxlength="3">
-
              <input name="FONE" type=text id="FONE" value="" size=7
maxlength="10"></td>
            </tr>
            <tr>
              <td><b>Email</b>: </td>
              <td><input name="EMAIL" type=text id="EMAIL" value="" size=30
maxlength="50"></td>
            </tr>
            <tr>
              <td><b>Observa&ccedil&otildees</b>: </td>
              <td><textarea name="OBSERVACOES" cols="22" rows="5"
id="OBSERVACOES"></textarea></td>
            </tr>
            <tr>
              <td>&nbsp;</td>
              <td><input type=submit name=submit value="Gravar &gt;&gt;">
                <input type=reset name=submit2 value="Limpar &gt;&gt;"></td>
            </tr>
          </table></td>
        </tr>
      </table>
    </form></td>
  </tr>
</table>
</body>
</html>
<% else

```

```

        Response.Redirect("index.asp")
    end if

%>

```

inserir.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%
/*
Função : Monta a tela para inserir nomes na agenda
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

String usuario = (String)session.getAttribute("USUARIO");

if ( ( usuario != null ) ) {
%>
<html>
<head>
    <title>:: Agenda JSP ::</title>
    <link rel="stylesheet" href="estilos.css" type="text/css">
</head>
<body bgcolor="#778899" text="#FFFFFF" link="#FFFFFF" vlink="#FFFFFF"
alink="#FFFFFF">
<table cellpadding=2 cellspacing=0 border=0 align=center bgcolor=White
width="700">
    <tr>
        <td valign=top width="100%" align=center>
            <%@ include file="menu.html" %>
        </td></tr>

        <td valign=top>
            <form name="form1" action="inserir_agenda.jsp" method=post>
                <table width="100%" border="0" cellpadding="2"
align="center">
                    <tr>
                        <td bgcolor=white> <table width="100%" border="0" cellpadding="1"
cellpadding="1" align="center">
                            <tr>
                                <td align=right class="title" colspan=2>.: <b>Inserir Nomes
na Agenda</b> :.</td>
                            </tr>
                            <tr>
                                <td align=right class="right" width="30%"><b>Nome</b>:
<br></td>
                                <td class="default1"><input name="NOME" type=text
class="textbox" id="NOME" value="" size=30 maxlength="50"></td>
                            </tr>
                            <tr> </tr>
                                <td align=right class="right"
width="30%"><b>Endere&ccedil;o</b>:
                                <br></td>
                                <td class="default1"><input name="ENDERECO" type=text
class="textbox" id="ENDERECO" value="" size=30 maxlength="100"></td>
                            </tr>
                        </tr>
                    </table>
                </table>
            </form>
        </td>
    </tr>
</table>

```

```

        <td class="right"><b>Fone</b>: </td>
        <td class="default1"><input name="DDD" type="text"
class="textbox" id="DDD" value="" size=3 maxlength="3">
        -
        <input name="FONE" type="text" class="textbox" id="FONE"
value="" size=7 maxlength="10"></td>
    </tr>
    <tr>
        <td class="right"><b>Email</b>: </td>
        <td class="default1"><input name="EMAIL" type="text"
class="textbox" id="EMAIL" value="" size=30 maxlength="50"></td>
    </tr>
    <tr>
        <td class="right"><b>Observa&ccedil;&otilde;es</b>: </td>
        <td class="default1"><textarea name="OBSERVACOES" cols="22"
rows="5" class="textbox" id="OBSERVACOES"></textarea></td>
    </tr>
    <tr>
        <td class="right">&nbsp;</td>
        <td class="default1"><input type="submit" name="submit"
value="Gravar &gt;&gt;" class="button">
        <input type="reset" name="submit2" value="Limpar &gt;&gt;"
class="button"></td>
    </tr>
</table></td>
</tr>
</table>
</form></td>
</tr>
</table>
</body>
</html>
<%
    } else {
        response.sendRedirect("index.jsp");
    }
%>

```

8 - Inserir Agenda

Este é o script que recebe os dados do formulário e insere no banco de dados.

inserir_agenda.php

```

<?
/*
Função : Insere os dados na Agenda
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//Função que insere os dados na agenda

```

```
function inserir_agenda( $nome, $endereco, $ddd, $fone, $email, $observacoes ) {
    global $con;
    $valor = false;
    $sql = "INSERT INTO Agenda( NOME, ENDERECO, DDD, FONE, EMAIL,
OBSERVACOES ) VALUES( '$nome', '$endereco', '$ddd', '$fone', '$email',
'$observacoes' )";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = false;
    } else {
        $valor = true;
    }
    return $valor;
}

//Corpo Principal do script
$usuario = trim($_SESSION['USUARIO']); //Recebe dados da sessão
$nome = $_POST['NOME'];
$endereco = $_POST['ENDERECO'];
$ddd = $_POST['DDD'];
$fone = $_POST['FONE'];
$email = $_POST['EMAIL'];
$observacoes = $_POST['OBSERVACOES'];

//Testa o usuario
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {

    if ( !$con ) { //Testa a conexão ao banco
        conecta(BANCO,USUARIO,SENHA); //Conectando ao banco
    }

    //Chama a função que insere os dados na agenda
    if ( inserir_agenda( $nome, $endereco, $ddd, $fone, $email,
$observacoes ) ) {
        header("location: listar.php"); //Direciona pra tela de listar
    } else {
        header("location: index.php"); //Direciona pra tela inicial
    }

} else {
    header("location: index.php"); //Direciona pra tela inicial
}
?>
```

inserir_agenda.asp

```
<!-- #include file="funcoes.asp" -->
<%
`Função : Insere os dados na Agenda
`Desenvolvedor : Cerli Rocha
`Data : 30/05/2003
`Atualização: 30/06/2003

function inserir_agenda( nome, endereco, ddd, fone, email, observacoes )
    valor = true
    sql = "INSERT INTO Agenda( NOME, ENDERECO, DDD, FONE, EMAIL,
OBSERVACOES ) " &
        "VALUES( '" & nome & "', '" & endereco & "', '" & ddd &
        "', '" & fone & "', '" & email & "', '" & observacoes & "'" )"
    set rs = con.execute(sql)
```

```

        if rs is nothing then
        valor = false
        else
        valor = true
        end if

        set rs = nothing
        inserir_agenda = valor
end function

login      = Session("USUARIO")
nome       = Request.Form("NOME")
endereco   = Request.Form("ENDERECO")
ddd        = Request.Form("DDD")
fone       = Request.Form("FONE")
email      = Request.Form("EMAIL")
observacoes = Request.Form("OBSERVACOES")

if ( ( login <> "" ) AND( NOT(isNull(login)) ) ) then

    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if

    if ( inserir_agenda( nome, endereco, ddd, fone, email, observacoes ) )
then
        Response.Redirect("listar.asp")
        else
        Response.Redirect("index.asp")
    end if

    else
        Response.Redirect("index.asp")
    end if
end if
%>

```

inserir_agenda.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
/*
Função : Insere os dados na Agenda
Desenvolvedor : Cerli Rocha
Data : 30/05/2003
Atualização: 30/06/2003
*/

public boolean inserir_agenda( String nome, String endereco, String ddd, String
fone, String email, String observacoes ) {
    String sql      = "";
    boolean valor = false;

    Statement stmt;

    try {
        sql = "INSERT INTO Agenda( NOME, ENDERECO, DDD, FONE, EMAIL,
OBSERVACOES ) " +

```

```

        "VALUES( '" + nome + "', '" + endereco + "', '" + ddd +
        "', '" + fone + "', '" + email + "', '" + observacoes + "' )";
        stmt = con.createStatement ();
        int linhas = stmt.executeUpdate(sql);

        if (linhas > 0) {
            valor = true;
        } else {
            valor = false;
        }

    } catch(SQLException e) {
        e.printStackTrace();
    }
    return valor;
}
%>
<%
String usuario      = (String)session.getAttribute("USUARIO");
String nome          = request.getParameter("NOME");
String endereco      = request.getParameter("ENDERECO");
String ddd           = request.getParameter("DDD");
String fone          = request.getParameter("FONE");
String email         = request.getParameter("EMAIL");
String observacoes   = request.getParameter("OBSERVACOES");

if ( ( usuario != null ) ) {

    if ( inserir_agenda( nome, endereco, ddd, fone, email, observacoes ) )
    {
        response.sendRedirect("listar.jsp");
    } else {
        response.sendRedirect("index.jsp");
    }

    } else {
        response.sendRedirect("index.jsp");
    }
}
%>

```

9 - Buscar

Este é o script que monta a tela de buscas da agenda.

buscar.php

```

<?
/*
Função      : Tela de Busca
Desenvolvedor: Cerli Rocha
Data        : 20/06/2003
Atualização : 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//Lendo a Sessão

```

```

$usuario = trim($_SESSION['USUARIO']);

//Testando se o usuario está logado
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {
//Abaixo o HTML
?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
            <? //Inclui o menu
                include("menu.html");
            ?>
        </td></tr>

        <td valign=top>
            <form name="form1" action="search.php" method=post>
                <table width="100%" border="0" align="center">
                    <tr>
                        <td bgcolor=white> <table width="100%" border="0" align="center">
                            <tr>
                                <td align=right colspan=2>.: <b>Buscar na Agenda</b> :.</td>
                            </tr>
                            <tr>
                                <td align=right width="30%"><b>Palavra-chave</b>:
                                    &nbsp;&nbsp;&nbsp;</td>
                                <td><input name="KEY" type=text id="KEY" value="" size=30
maxlength="50"></td>
                            </tr>
                            <tr>
                                <td>&nbsp;&nbsp;&nbsp;</td>
                                <td><input type=submit name=submit value="Buscar &gt;&gt;";
                                    <input type=reset name=submit2 value="Limpar &gt;&gt;";></td>
                            </tr>
                        </table></td>
                    </tr>
                </table>
            </form>
        </td>
    </tr>
</table>
</body>
</html>
<?
    } else {
        header("location: index.php"); //Direciona para tela ininial
    }
?>

```

buscar.asp

```

<!-- #include file="funcoes.asp" -->
<%
'Função          : Tela de Busca
'Desenvolvedor: Cerli Rocha

```

```

'Data      : 20/06/2003
'Atualização : 30/06/2003

'Lendo a Sessão
login = Session("USUARIO")

'Tastando se o usuario está logado
if ( ( login <> "" )AND( NOT(isNull(login)) ) ) then
%>
<html>
<head>
    <title>:: Agenda ASP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center><!-- #include
file="menu.html" --></td></tr>

        <td valign=top>
            <form name="form1" action="search.asp" method=post>
                <table width="100%" border="0" align="center">
                    <tr>
                        <td bgcolor=white> <table width="100%" border="0" align="center">
                            <tr>
                                <td align=right colspan=2>.: <b>Buscar na Agenda</b> :.</td>
                            </tr>
                            <tr>
                                <td align=right width="30%"><b>Palavra-chave</b>:
                                    &nbsp;  </td>
                                <td><input name="KEY" type=text id="KEY" value="" size=30
maxlength="50"></td>
                            </tr>
                            <tr>
                                <td>&nbsp;  </td>
                                <td><input type=submit name=submit value="Buscar &gt;&gt;";
                                    <input type=reset name=submit2 value="Limpar &gt;&gt;";></td>
                            </tr>
                        </table></td>
                    </tr>
                </table>
            </form>
        </td>
    </tr>
</table>
</body>
</html>
<%
    else
        Response.Redirect("index.asp") 'Login
end if
%>

```

buscar.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%
/*

```

```

Função      : Tela de Busca
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/

//Lendo a Sessão
String usuario = (String)session.getAttribute("USUARIO");

//Tastando se o usuario está logado
if ( ( usuario != null ) ) {
%>
<html>
<head>
    <title>:: Agenda JSP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
            <%@ include file="menu.html" %>
        </td></tr>
    <td valign=top>
        <form name="form1" action="search.jsp" method=post>
            <table width="100%" border="0" align="center">
                <tr>
                    <td bgcolor=white> <table width="100%" border="0" align="center">
                        <tr>
                            <td align=right colspan=2>.: <b>Buscar na Agenda</b> :.</td>
                        </tr>
                        <tr>
                            <td align=right width="30%"><b>Palavra-chave</b>:
                                &nbsp;  </td>
                            <td><input name="KEY" type=text id="KEY" value="" size=30
maxlength="50"></td>
                        </tr>
                        <tr>
                            <td>&nbsp;</td>
                            <td><input type=submit name=submit value="Buscar &gt;&gt;";
                                <input type=reset name=submit2 value="Limpar &gt;&gt;";</td>
                        </tr>
                    </table></td>
                </tr>
            </table>
        </form>
    </td>
</tr>
</table>
</body>
</html>
<%
    } else {
        response.sendRedirect("index.jsp");//Login
    }
%>

```

10 - Search

Este é o script que faz a busca no banco de dados de acordo com a palavra-chave recebida e retorna uma lista idêntica a listar.

search.php

```
<?
/*
Função      : Buscar a palavra-chave na agenda
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo e funções

//Função que faz a busca na agenda
function search( $KEY ) {
    global $con;
    $valor = "";
    $sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda WHERE (NOME LIKE '%$KEY%') OR (ENDERECO LIKE '%$KEY%') OR (DDD
LIKE '%$KEY%') OR (FONE LIKE '%$KEY%') OR (EMAIL LIKE '%$KEY%')";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = 0;
    } else {
        while( odbc_fetch_row( $result ) ) {
            $valor .= "<tr>
                        <td class=\"default\"><a
href=\"editar.php?ID=\".odbc_result( $result, 1 ).\">Editar</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=350,height=200')\">.odbc_result( $result, 2 ).\"</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=350,height=200')\">.odbc_result( $result, 6 ).\"</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=350,height=200')\">(\".odbc_result( $result, 4 )
.\".odbc_result( $result, 5 ).\"</a></td>
                        <td class=\"default\"><a
href=\"#\" onClick=\"window.open('excluir.php?ID=\".odbc_result( $result, 1 )
.', 'Detalhes', 'width=1,height=1')\">Excluir</a></td>
                        </tr>";
        }
    }
    return $valor;
}

//Corpo principal do script
$usuario = trim($_SESSION['USUARIO']); //Recebe os dados da sessão
$key      = trim($_POST['KEY']);

//Testa se o usuário está logado
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {
```

```

        if ( !$con ) { //testa se existe conexão ao banco
            conecta(BANCO,USUARIO,SENHA); //Conectando ao banco
        }

        //Chama a função de busca e grava na variavel $linhas
        $linhas = search( $key );
//Abaixo o HTML
?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
<?
//Incluindo o menu
include("menu.html");
?>

        </td></tr>
        <td valign=top>
            <table cellspacing=1 cellpadding=1 width="100%" border=0
bgcolor=White>
                <tr>
                    <td width="60" class="headers"><b><a href="#">Editar</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Nome</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Email</a></b></td>
                    <td width="100" class="headers"><b><a href="#">Fone</a></b></td>
                    <td width="60" class="headers"><b><a href="#">Excluir</a></b></td>
                </tr>
                <?=$linhas?>
            </table>
        </td>
    </tr>
</table>
</body>
</html>
<?
    } else {
        header("location: index.php"); //Direciona pra tela inicial
    }
?>

```

search.asp

```

<!-- #include file="funcoes.asp" -->
<%
`Função          : Buscar a palavra-chave na agenda
`Desenvolvedor: Cerli Rocha
`Data            : 20/06/2003
`Atualização    : 30/06/2003

function search( KEY )
    valor = ""
    sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
Data FROM Agenda " & _

```

```

        "WHERE (NOME LIKE '%' & KEY & '%') OR (ENDereco LIKE '%' &
KEY & '%') OR (DDD LIKE '%' & KEY & '%') OR (Fone LIKE '%' & KEY & '%') OR
(EMAIL LIKE '%' & KEY & '%') "
        Set rs = Server.CreateObject("ADODB.Recordset")
        rs.Open sql, con
        Do While Not rs.EOF
            valor = valor & "<tr>" & _
                "<td class=default><a
href=editar.asp?ID=" & rs(0) & ">Editar</a></td>" & _
                "<td class=default><a href=#
onClick=window.open('detalhes.asp?ID=" & rs(0) &
'", 'Detalhes', 'width=350,height=200')>" & rs(1) & "</a></td>" & _
                "<td class=default><a href=#
onClick=window.open('detalhes.asp?ID=" & rs(0) &
'", 'Detalhes', 'width=350,height=200')>" & rs(5) & "</a></td>" & _
                "<td class=default><a href=#
onClick=window.open('detalhes.asp?ID=" & rs(0) &
'", 'Detalhes', 'width=350,height=200')>(" & rs(3) & ") & rs(4) & "</a></td>" & _
                "<td class=default><a href=#
onClick=window.open('excluir.asp?ID=" & rs(0) &
'", 'Detalhes', 'width=1,height=1')>Excluir</a></td>" & _
                "</tr>"
            rs.MoveNext
        Loop
        rs.Close
        set rs = Nothing
        search = valor
end function

login = Session("USUARIO")
key      = trim(Request.Form("KEY"))

if ( ( login <> "" ) AND ( NOT(isNull(login)) ) ) then

    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if

    linhas = search( key )
%>
<html>
<head>
    <title>:: Agenda ASP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center><!-- #include
file="menu.html" --></td></tr>
        <td valign=top>
            <table cellpadding=1 cellspacing=1 width="100%" border=0
bgcolor=White>
                <tr>
                    <td width="60" class="headers"><b><a href="#">Editar</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Nome</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Email</a></b></td>
                    <td width="100" class="headers"><b><a href="#">Fone</a></b></td>
                    <td width="60" class="headers"><b><a href="#">Excluir</a></b></td>
                </tr>

```

```

<%=linhas%>
    </table>
    </td>
  </tr>
</table>
</body>
</html>
<%
  else
    Response.Redirect("index.asp")
end if
%>

```

search.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
/*
Função      : Buscar a palavra-chave na agenda
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/
public String search( String KEY ) {
    String valor = "";
    String sql   = "";

    String id = "";
    String nome = "";
    String endereco = "";
    String ddd = "";
    String fone = "";
    String email = "";
    String observacoes = "";

    Statement stmt;
    ResultSet rs;

    //Select
    try {
        sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda " +
            " WHERE (NOME LIKE '%" + KEY + "%') OR (ENDERECO LIKE '%" + KEY + "%') OR (DDD LIKE '%" + KEY + "%') OR (FONE LIKE '%" + KEY + "%') OR (EMAIL LIKE '%" + KEY + "%')";
        stmt = con.createStatement();
        stmt.execute(sql);
        rs = stmt.getResultSet();
        while(rs.next()) {
            id          = rs.getString(1);
            nome        = rs.getString(2);
            endereco    = rs.getString(3);
            ddd         = rs.getString(4);
            fone        = rs.getString(5);
            email       = rs.getString(6);
            observacoes = rs.getString(7);

```

```

        valor += "<tr>" +
                                "<td class=\"default\"><a
href=\"editar.jsp?ID=" + id + "\">Editar</a></td>" +
                                "<td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.jsp?ID=" + id +
\"", 'Detalhes', 'width=350,height=200')\">" + nome + "</a></td>" +
                                "<td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.jsp?ID=" + id +
\"", 'Detalhes', 'width=350,height=200')\">" + email + "</a></td>" +
                                "<td class=\"default\"><a
href=\"#\" onClick=\"window.open('detalhes.jsp?ID=" + id +
\"", 'Detalhes', 'width=350,height=200')\">(" + ddd + ")" + fone + "</a></td>" +
                                "<td class=\"default\"><a
href=\"#\" onClick=\"window.open('excluir.jsp?ID=" + id +
\"", 'Detalhes', 'width=1,height=1')\">Excluir</a></td>" +
                                "</tr>";

    }
    stmt.close();
    if (!con.isClosed()) {
        con.close();
    }
} catch (Exception e) {
    try {
        if (!con.isClosed()) {
        }
    } catch (Exception ex) {
    }
}
return valor;
}
%>
<%
String usuario = (String)session.getAttribute("USUARIO");
String key      = request.getParameter("KEY");

if ( ( usuario != null ) ) {

    String linhas = search( key );
%>
<html>
<head>
    <title>:: Agenda JSP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>
<%@ include file="menu.html" %>

        </td></tr>
        <td valign=top>
            <table cellspacing=1 cellpadding=1 width="100%" border=0
bgcolor=White>
                <tr>
                    <td width="60" class="headers"><b><a href="#">Editar</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Nome</a></b></td>
                    <td width="210" class="headers"><b><a href="#">Email</a></b></td>

```

```

        <td width="100" class="headers"><b><a href="#">Fone</a></b></td>
        <td width="60" class="headers"><b><a href="#">Excluir</a></b></td>
    </tr>
    <%=linhas%>
</table>
    </td>
</tr>
</table>
</body>
</html>
<%
    } else {
        response.sendRedirect("index.jsp");
    }
%>

```

11 - Editar

Esta é a tela que abre algum dado que já foi inserido na agenda para que possa ser alterado.

editar.php

```

<?
/*
Função      : Abre a Agenda para edição
Desenvolvedor: Cerli Rocha
Data        : 20/06/2003
Atualização : 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//Função que abre a agenda de acordo com o código recebido
function editar_agenda( $ID ) {
    global $con;
    global $nome, $endereco, $ddd, $fone, $email, $observacoes;
    $valor = false;
    $sql    = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda WHERE ID = $ID";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = false;
    } else {
        while( odbc_fetch_row( $result ) ) {
            $nome      = odbc_result( $result, 2 );
            $endereco   = odbc_result( $result, 3 );
            $ddd        = odbc_result( $result, 4 );
            $fone       = odbc_result( $result, 5 );
            $email      = odbc_result( $result, 6 );
            $observacoes = odbc_result( $result, 7 );
        }
    }
    return $valor;
}

```

```
//Corpo principal do script
$usuario = trim($_SESSION['USUARIO']); //Recebe dados da sessão
$id       = trim($_GET['ID']);

//testa se o usuario está logado
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {

    if ( !$con ) { //Testa se existe conexão ao banco
        conecta(BANCO,USUARIO,SENHA); //Conectando ao banco
    }

    //Chama a função que abre os dados da agenda
    editar_agenda($id);
}

//Abaixo o HTML
?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center>

<?
//Inclui o Menu
include("menu.html");
?>

        </td></tr>

    <td valign=top>
        <form name="form1" action="atualizar_agenda.php" method=post>
            <table width="100%" border="0" align="center">
                <tr>
                    <td bgcolor=white> <table width="100%" border="0" align="center">
                        <tr>
                            <td align=right colspan=2>.: <b>Inserir Nomes
                                na Agenda</b> :.</td>
                        </tr>
                        <tr>
                            <td align=right width="30%"><b>Nome</b>: &nbsp;  </td>
                            <td><input name="NOME" type=text id="NOME" value="<?=$nome?>"
size=30 maxlength="50"></td>
                        </tr>
                        <tr>
                            <td align=right width="30%"><b>Endere&ccedil;o</b>:
                                &nbsp;  </td>
                            <td><input name="ENDERECO" type=text id="ENDERECO"
value="<?=$endereco?>" size=30 maxlength="100"></td>
                        </tr>
                        <tr>
                            <td><b>Fone</b>: </td>
                            <td><input name="DDD" type=text id="DDD" value="<?=$ddd?>"
size=3 maxlength="3">
                                -
                                <input name="FONE" type=text id="FONE" value="<?=$fone?>"
size=7 maxlength="10"></td>
                        </tr>
                        <tr>
                            <td><b>Email</b>: </td>
```

```

        <td><input name="EMAIL" type="text" id="EMAIL"
value="<?=$email?>" size=30 maxlength="50"></td>
        </tr>
        <tr>
            <td><b>Observa&ccedil;&otilde;es</b>: </td>
            <td><textarea name="OBSERVACOES" cols="22" rows="5"
id="OBSERVACOES"><?=$observacoes?></textarea></td>
        </tr>
        <tr>
            <td>&nbsp;</td>
            <td>
                <input type="hidden" name="ID" value="<?=$id?>">
                <input type="submit" name="submit" value="Gravar
&gt;&gt;">
                <input type="reset" name="submit2" value="Limpar &gt;&gt;"></td>
            </tr>
        </table></td>
    </tr>
</table>
</body>
</html>
<?
    } else {
        header("location: index.php"); //Redireciona para tela inicial
    }
?>

```

editar.asp

```

<!-- #include file="funcoes.asp" -->
<%
`Função      : Abre a Agenda para edição
`Desenvolvedor: Cerli Rocha
`Data        : 20/06/2003
`Atualização : 30/06/2003

function editar_agenda( ID )
    sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
Data FROM Agenda WHERE ID = " & ID
    Set rs = Server.CreateObject("ADODB.Recordset")
    rs.Open sql, con
    Do While Not rs.EOF
        nome = rs(1)
        endereco = rs(2)
        ddd = rs(3)
        fone = rs(4)
        email = rs(5)
        observacoes = rs(6)
        rs.MoveNext
    Loop
    rs.Close
    set rs = Nothing
end function

login = Session("USUARIO")
id = trim(Request.QueryString("ID"))
Dim nome

```

```

Dim endereco
Dim ddd
Dim fone
Dim email
Dim observacoes

if ( ( login <> "" )AND( NOT(isNull(login)) ) ) then
    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if
    call editar_agenda(id)
%>
<html>
<head>
    <title>:: Agenda ASP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="700">
    <tr>
        <td valign=top width="100%" align=center><!-- #include
file="menu.html" --></td></tr>

    <td valign=top>
        <form name="form1" action="atualizar_agenda.asp" method=post>
            <table width="100%" border="0" align="center">
                <tr>
                    <td bgcolor=white> <table width="100%" border="0" align="center">
                        <tr>
                            <td align=right colspan=2>.: <b>Inserir Nomes
                                na Agenda</b> :.</td>
                        </tr>
                        <tr>
                            <td align=right width="30%"><b>Nome</b>: &nbsp;</td>
                            <td><input name="NOME" type=text id="NOME" value="<%=nome%>"
size=30 maxlength="50"></td>
                        </tr>
                        <tr> </tr>
                            <td align=right width="30%"><b>Endere&ccedilo</b>:
                                &nbsp;</td>
                            <td><input name="ENDERECO" type=text id="ENDERECO"
value="<%=endereco%>" size=30 maxlength="100"></td>
                        </tr>
                        <tr>
                            <td><b>Fone</b>: </td>
                            <td><input name="DDD" type=text id="DDD" value="<%=ddd%>"
size=3 maxlength="3">
                                -
                                <input name="FONE" type=text id="FONE" value="<%=fone%>"
size=7 maxlength="10"></td>
                        </tr>
                        <tr>
                            <td><b>Email</b>: </td>
                            <td><input name="EMAIL" type=text id="EMAIL"
value="<%=email%>" size=30 maxlength="50"></td>
                        </tr>
                        <tr>
                            <td><b>Observa&ccedil&otildees</b>: </td>
                            <td><textarea name="OBSERVACOES" cols="22" rows="5"
id="OBSERVACOES"><%=observacoes%></textarea></td>

```

```
<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
/*
Função      : Abre a Agenda para edição
Desenvolvedor: Cerli Rocha
Data        : 20/06/2003
Atualização  : 30/06/2003
*/

public String editar_agenda( String ID ) {
    String valor = "";
    String sql   = "";

    Statement stmt;
    ResultSet rs;

    //Select
    try {
        sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda WHERE ID = " + ID;
        stmt = con.createStatement();
        stmt.execute(sql);
        rs = stmt.getResultSet();
        while(rs.next()) {
            id = rs.getString(1);
            nome = rs.getString(2);
            endereco = rs.getString(3);
            ddd = rs.getString(4);
            fone = rs.getString(5);
            email = rs.getString(6);
            observacoes = rs.getString(7);
        }
    }
}
```

```

        stmt.close();
        if (!con.isClosed()) {
            con.close();
        }
    } catch (Exception e) {
        try {
            if (!con.isClosed()) {
            }
        } catch (Exception ex) {
        }
    }
    return valor;
}

String id = "";
String nome = "";
String endereco = "";
String ddd = "";
String fone = "";
String email = "";
String observacoes = "";

%>

<%
String usuario = (String)session.getAttribute("USUARIO");
String id      = request.getParameter("ID");

if ( ( usuario != null ) ) {

    editar_agenda(id);
%>
<html>
<head>
    <title>:: Agenda JSP ::</title>
    <link rel="stylesheet" href="estilos.css" type="text/css">
</head>
<body bgcolor="#778899" text="#FFFFFF" link="#FFFFFF" vlink="#FFFFFF"
alink="#FFFFFF">
<table cellspacing=2 cellpadding=0 border=0 align=center bgcolor=White
width="700">
    <tr>
        <td valign=top width="100%" align=center>
            <%@ include file="menu.html" %>
        </td></tr>

    <td valign=top>
        <form name="form1" action="atualizar_agenda.jsp" method=post>
            <table width="100%" border="0" cellspacing="0" cellpadding="2"
align="center">
                <tr>
                    <td bgcolor=white> <table width="100%" border="0" cellspacing="1"
cellpadding="1" align="center">
                        <tr>
                            <td align=right class="title" colspan=2>.: <b>Inserir Nomes
na Agenda</b> :.</td>
                        </tr>
                    </tr>
                </tr>
            </table>
        </td>
    </tr>

```

```

        <td align=right class="right" width="30%"><b>Nome</b>:
&nbsp;</td>
        <td class="default1"><input name="NOME" type=text
class="textbox" id="NOME" value="<%=nome%>" size=30 maxlength="50"></td>
    </tr>
    <tr> </tr>
        <td align=right class="right"
width="30%"><b>Endere&ccedil;o</b>:
        &nbsp;</td>
        <td class="default1"><input name="ENDERECO" type=text
class="textbox" id="ENDERECO" value="<%=endereco%>" size=30
maxlength="100"></td>
    </tr>
    <tr>
        <td class="right"><b>Fone</b>: </td>
        <td class="default1"><input name="DDD" type=text
class="textbox" id="DDD" value="<%=ddd%>" size=3 maxlength="3">
        -
        <input name="FONE" type=text class="textbox" id="FONE"
value="<%=fone%>" size=7 maxlength="10"></td>
    </tr>
    <tr>
        <td class="right"><b>Email</b>: </td>
        <td class="default1"><input name="EMAIL" type=text
class="textbox" id="EMAIL" value="<%=email%>" size=30 maxlength="50"></td>
    </tr>
    <tr>
        <td class="right"><b>Observa&ccedil;es</b>: </td>
        <td class="default1"><textarea name="OBSERVACOES" cols="22"
rows="5" class="textbox" id="OBSERVACOES"><%=observacoes%></textarea></td>
    </tr>
    <tr>
        <td class="right">&nbsp;</td>
        <td class="default1">
            <input type="hidden" name="ID" value="<%=id%>">
            <input type=submit name=submit value="Gravar
&gt;&gt;" class="button">
            <input type=reset name=submit2 value="Limpar &gt;&gt;"
class="button"></td>
    </tr>
</table></td>
</tr>
</table>
</form></td>
</tr>
</table>
</body>
</html>
<%
    } else {
        response.sendRedirect("index.jsp");
    }
%>

```

12 - Atualizar Agenda

Este é o script que atualiza os dados na agenda.

atualizar_agenda.php

```
<?
/*
Função      : Atualizar os dados na agenda
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//atualiza os dados na agenda
function atualizar_agenda( $nome, $endereco, $ddd, $fone, $email, $observacoes,
$id ) {
    global $con;
    $valor = false;
    $sql = "UPDATE Agenda SET NOME = '$nome', ENDERECO = '$endereco',
        DDD = '$ddd', FONE = '$fone', EMAIL = '$email',
        OBSERVACOES = '$observacoes' WHERE ID = $id";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = false;
    } else {
        $valor = true;
    }
    return $valor;
}

//Corpo principal do script
//Recebendo os Dados do Formulário
$usuario = trim($_SESSION['USUARIO']); //Lendo a Sessão
$nome = $_POST['NOME'];
$endereco = $_POST['ENDERECO'];
$ddd = $_POST['DDD'];
$fone = $_POST['FONE'];
$email = $_POST['EMAIL'];
$observacoes = $_POST['OBSERVACOES'];
$id = $_POST['ID'];

//Testando se o usuario está logado
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {

    if ( !$con ) { //testa se existe conexão
        conecta(BANCO,USUARIO,SENHA); //Conectando ao banco
    }

    //Chama a função que atualiza os dados alterados
    if ( atualizar_agenda( $nome, $endereco, $ddd, $fone, $email,
$observacoes, $id ) ) {
        header("location: listar.php"); //Direciona para o listar
    } else {
        header("location: index.php"); //Direciona para tela inicial
    }

} else {
    header("location: index.php"); //Direciona para tela inicial
}
}
```

?>

atualizar_agenda.asp

```

<!-- #include file="funcoes.asp" -->
<%
`Função      : Atualizar os dados na agenda
`Desenvolvedor: Cerli Rocha
`Data        : 20/06/2003
`Atualização : 30/06/2003

function atualizar_agenda( nome, endereco, ddd, fone, email, observacoes, id )
    valor = true
    sql = "UPDATE Agenda SET NOME = '" & nome & "', ENDERECO = '" &
endereco & "', DDD = '" & ddd & "', FONE = '" & fone & "', EMAIL = '" & email &
"', " & _
                "OBSERVACOES = '" & observacoes & "' WHERE ID = " & id
    set rs = con.execute(sql)

    if rs is nothing then
        valor = false
    else
        valor = true
    end if

    set rs = nothing
    atualizar_agenda = valor
end function

'Recebendo os Dados do Formulário
login      = Session("USUARIO") 'Lendo a Sessão
nome       = Request.Form("NOME")
endereco   = Request.Form("ENDERECO")
ddd        = Request.Form("DDD")
fone       = Request.Form("FONE")
email      = Request.Form("EMAIL")
observacoes = Request.Form("OBSERVACOES")
id         = Request.Form("ID")

'Tastando se o usuario está logado
if ( ( login <> "" ) AND ( NOT(isNull(login)) ) ) then

    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if

    if ( atualizar_agenda( nome, endereco, ddd, fone, email, observacoes,
id ) ) then 'Executando a 'Função
        Response.Redirect("listar.asp") 'Página Principal
    else
        Response.Redirect("index.asp") 'Login
    end if

    else
        Response.Redirect("index.asp") 'Login
    end if
end if
%>

```

atualizar_agenda.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
/*
Função      : Atualizar os dados na agenda
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003

public boolean atualizar_agenda( String nome, String endereco, String ddd,
String fone, String email, String observacoes, String id ) {
    String sql      = "";
    boolean valor = false;

    Statement stmt;

    try {
        sql = "UPDATE Agenda SET NOME = '" + nome + "', ENDERECO = '" +
endereco + "', DDD = '" + ddd + "', FONE = '" + fone + "', EMAIL = '" + email +
"', " +
                " OBSERVACOES = '" + observacoes + "' WHERE ID = " +
String.valueOf(id);
        stmt = con.createStatement ();
        int linhas = stmt.executeUpdate(sql);

        if (linhas > 0) {
            valor = true;
        } else {
            valor = false;
        }

    } catch(SQLException e) {
        e.printStackTrace();
    }
    return valor;
}
%>
<%

String usuario      = (String)session.getAttribute("USUARIO");
String nome         = request.getParameter("NOME");
String endereco     = request.getParameter("ENDERECO");
String ddd          = request.getParameter("DDD");
String fone         = request.getParameter("FONE");
String email        = request.getParameter("EMAIL");
String observacoes  = request.getParameter("OBSERVACOES");
String id = request.getParameter("ID");

//Tastando se o usuario está logado
if ( ( usuario != null ) ) {

    if ( atualizar_agenda( nome, endereco, ddd, fone, email, observacoes,
id ) ) { //Executando a Função
        response.sendRedirect("listar.jsp");//Página Principal
    } else {
        response.sendRedirect("index.jsp");//Login
    }

} else {
    response.sendRedirect("index.jsp");//Login
}
}

```

```
}
%>
```

13 - Detalhes

Este é o script que exibe os detalhes de cada registro da agenda.

detalhes.php

```
<?
/*
Função      : Exibe os Detalhes
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//Busca os dados da agenda de acordo com o Código recebido
function editar_agenda( $ID ) {
    global $con;
    global $nome, $endereco, $ddd, $fone, $email, $observacoes;
    $valor = false;
    $sql    = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda WHERE ID = $ID";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = false;
    } else {
        while( odbc_fetch_row( $result ) ) {
            $nome      = odbc_result( $result, 2 );
            $endereco   = odbc_result( $result, 3 );
            $ddd        = odbc_result( $result, 4 );
            $fone       = odbc_result( $result, 5 );
            $email      = odbc_result( $result, 6 );
            $observacoes = odbc_result( $result, 7 );
        }
    }
    return $valor;
}

//Corpo principal do script
//Lendo a Sessão
$usuario = trim($_SESSION['USUARIO']);
//Lendo o Código
$id       = trim($_GET['ID']);

//Tastando se o usuario está logado
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {

    if ( !$con ) { //Testa se existe a conexão
        conecta(BANCO,USUARIO,SENHA);//Conectando ao banco
    }
    //Chama a função que exibirá os detalhes
}
```

```

        editar_agenda($id);
//Abaixo o HTML
?>
<html>
<head>
    <title>:: Agenda PHP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="100%">
    <td valign=top>
        <table width="100%" border="0" align="center">
            <tr>
                <td bgcolor=white> <table width="100%" border="0" align="center">
                    <tr>
                        <td align=right colspan=2>.: <b>Inserir Nomes na
                            Agenda</b> :.</td>
                    </tr>
                    <tr>
                        <td align=right width="30%"><b>Nome</b>: &nbsp;</td>
                        <td>
                            <?=$nome?>
                        </td>
                    </tr>
                    <tr> </tr>
                    <td align=right width="30%"><b>Endere&ccedil;o</b>:
                        &nbsp;</td>
                        <td>
                            <?=$endereco?>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Fone</b>: </td>
                        <td>
                            <?=$ddd?>
                            -
                            <?=$fone?>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Email</b>: </td>
                        <td>
                            <?=$email?>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Observa&ccedil;es</b>: </td>
                        <td>
                            <?=$observacoes?>
                        </td>
                    </tr>
                    <tr>
                        <td>&nbsp;</td>
                        <td><a href="#" onClick="self.close()" >fechar</a></td>
                    </tr>
                </table></td>
            </tr>
        </table>
    </td>
</tr>

```

```

</table>
</body>
</html>
<?
    } else {
        print "<script>
            self.close();
        </script>";
    }
?>

```

detalhes.asp

```

<!-- #include file="funcoes.asp" -->
<%
'Função      : Exibe os Detalhes
'Desenvolvedor: Cerli Rocha
'Data        : 20/06/2003
'Atualização : 30/06/2003

'Busca os dados da agenda de acordo com o Código recebido
function editar_agenda( ID )
    sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
Data FROM Agenda WHERE ID = " & ID
    Set rs = Server.CreateObject("ADODB.Recordset")
    rs.Open sql, con
    Do While Not rs.EOF
        nome = rs(1)
        endereco = rs(2)
        ddd = rs(3)
        fone = rs(4)
        email = rs(5)
        observacoes = rs(6)
        rs.MoveNext
    Loop
    rs.Close
    set rs = Nothing
end function

'Lendo a Sessão
login = Session("USUARIO")
'Lendo o Código
id = trim(Request.QueryString("ID"))
Dim nome
Dim endereco
Dim ddd
Dim fone
Dim email
Dim observacoes

'Tastando se o usuario está logado
if ( ( login <> "" ) AND ( NOT(isNull(login)) ) ) then

    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if

    call editar_agenda(id)
%>

```

```

<html>
<head>
    <title>:: Agenda ASP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="100%">
    <td valign=top>
        <table width="100%" border="0" align="center">
            <tr>
                <td bgcolor=white> <table width="100%" border="0" align="center">
                    <tr>
                        <td align=right colspan=2>.: <b>Inserir Nomes na
                            Agenda</b> :.</td>
                    </tr>
                    <tr>
                        <td align=right width="30%"><b>Nome</b>: &nbsp;</td>
                        <td>
<%=nome%>
                            </td>
                    </tr>
                    <tr> </tr>
                    <td align=right width="30%"><b>Endere&ccedilo</b>:
                        &nbsp;</td>
                    <td>
                        <%=endereco%>
                    </td>
                    </tr>
                    <tr>
                        <td><b>Fone</b>: </td>
                        <td>
                            <%=ddd%>
                            -
                            <%=fone%>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Email</b>: </td>
                        <td>
                            <%=email%>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Observa&ccedil&otildees</b>: </td>
                        <td>
                            <%=observacoes%>
                        </td>
                    </tr>
                    <tr>
                        <td>&nbsp;</td>
                        <td><a href="#" onClick="self.close()" >fechar</a></td>
                    </tr>
                </table></td>
            </tr>
        </table>
    </td>
</tr>
</table>
</body>
</html>

```

```
<%
else
    response.Write("<script>" & _
        "self.close()" & _
        "</script>")
end if
%>
```

detalhes.jsp

```
<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
/*
Função      : Exibe os Detalhes
Desenvolvedor: Cerli Rocha
Data        : 20/06/2003
Atualização : 30/06/2003
*/

public String editar_agenda( String ID ) {
    String valor = "";
    String sql   = "";

    Statement stmt;
    ResultSet rs;

    //Select
    try {
        sql = "SELECT ID, NOME, ENDERECO, DDD, FONE, EMAIL, OBSERVACOES,
DATA FROM Agenda WHERE ID = " + ID;
        stmt = con.createStatement();
        stmt.execute(sql);
        rs = stmt.getResultSet();
        while(rs.next()) {
            id          = rs.getString(1);
            nome        = rs.getString(2);
            endereco    = rs.getString(3);
            ddd         = rs.getString(4);
            fone        = rs.getString(5);
            email       = rs.getString(6);
            observacoes = rs.getString(7);
        }
        stmt.close();
        if (!con.isClosed()) {
            con.close();
        }
    } catch (Exception e) {
        try {
            if (!con.isClosed()) {
            }
        } catch (Exception ex) {
        }
    }
    return valor;
}

String id = "";
String nome = "";
String endereco = "";
```

```

        String ddd = "";
        String fone = "";
        String email = "";
        String observacoes = "";
    %>

    <%
String usuario = (String)session.getAttribute("USUARIO");
String id      = request.getParameter("ID");

//Tastando se o usuario está logado
if ( ( usuario != null ) ) {

        editar_agenda(id);
    %>
<html>
<head>
    <title>:: Agenda JSP ::</title>

</head>
<body bgcolor="#778899" >
<table border=0 align=center bgcolor=White width="100%">
    <td valign=top>
        <table width="100%" border="0" align="center">
            <tr>
                <td bgcolor=white> <table width="100%" border="0" align="center">
                    <tr>
                        <td align=right colspan=2>.: <b>Inserir Nomes na
                            Agenda</b> :.</td>
                    </tr>
                    <tr>
                        <td align=right width="30%"><b>Nome</b>: &nbsp;</td>
                        <td>
                            <%=nome%>
                        </td>
                    </tr>
                    <tr> </tr>
                        <td align=right width="30%"><b>Endere&ccedil;o</b>:
                            &nbsp;</td>
                        <td>
                            <%=endereco%>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Fone</b>: </td>
                        <td>
                            <%=ddd%>
                            -
                            <%=fone%>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Email</b>: </td>
                        <td>
                            <%=email%>
                        </td>
                    </tr>
                    <tr>
                        <td><b>Observa&ccedil;&otilde;es</b>: </td>
                        <td>

```

```

                <%=observacoes%>
            </td>
        </tr>
        <tr>
            <td>&nbsp;</td>
            <td><a href="#" onClick="self.close()" >fechar</a></td>
        </tr>
    </table></td>
</tr>
</table>
</td>
</tr>
</table>
</body>
</html>
<%
    } else {
        out.println("<script> " +
            "self.close(); " +
            "</script>");
    }
%>

```

14 - Excluir

Este é o script que exclui os dados da agenda de acordo com o código recebido.

excluir.php

```

<?
/*
Função      : Exclui os dados da agenda
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/

session_start(); //Inicializa a sessão
include("funcoes.php"); //Inclui o arquivo de funções

//Função que exclui os dados da agenda
function excluir_agenda( $id ) {
    global $con;
    $valor = false;
    $sql = "DELETE FROM Agenda WHERE ID = $id";
    $result = odbc_exec( $con, $sql );
    if (!$result) {
        $valor = false;
    } else {
        $valor = true;
    }
    return $valor;
}

//Corpo Principal do script

```

```
$usuario      = trim($_SESSION['USUARIO']); //Recebe os dados da sessão
$id           = $_GET['ID'];

//Testa se o usuario está logado
if ( ( $usuario != "" ) && ( $usuario != "" ) ) {

    if ( !$con ) { //Testa se existe conexão ao banco
        conecta(BANCO,USUARIO,SENHA); //Conectando ao banco
    }

    //Chama a função de exclusão
    if ( excluir_agenda( $id ) ) {
        print "<script>
            opener.location.reload();
            self.close();
        </script>";
    } else {
        print "<script>
            self.close();
        </script>";
    }

} else {
    print "<script>
        opener.location = 'index.php';
        self.close();
    </script>";
}
?>
```

excluir.asp

```
<!-- #include file="funcoes.asp" -->
<%
`Função      : Exclui os dados da agenda
`Desenvolvedor: Cerli Rocha
`Data       : 20/06/2003
`Atualização : 30/06/2003

function excluir_agenda( id )
    valor = true
    sql    = "DELETE FROM Agenda WHERE ID = " & id
    set rs = con.execute(sql)

    if rs is nothing then
        valor = false
    else
        valor = true
    end if

    set rs = nothing
    excluir_agenda = valor
end function

login = Session("USUARIO")
id     = Request.QueryString("ID")

if ( ( login <> "" ) AND( NOT(isNull(login)) ) ) then
    if ( con = "" ) then
        call conecta( BANCO, USUARIO, SENHA, con )
    end if
end if
```

```

        end if

        if ( excluir_agenda( id ) ) then
            response.Write("<script>" & _
                "opener.location.reload();" & _
                "self.close();" & _
                "</script>")
        else
            response.Write("<script>" & _
                "self.close();" & _
                "</script>")
        end if

    else
        response.Write("<script>" & _
            "opener.location = 'index.asp';" & _
            "self.close();" & _
            "</script>")
    end if
%>

```

excluir.jsp

```

<%@ page import = "java.lang.*,java.io.*,java.util.*,java.sql.*" %>
<%@ include file="funcoes.jsp" %>
<%!
/*
Função      : Exclui os dados da agenda
Desenvolvedor: Cerli Rocha
Data       : 20/06/2003
Atualização : 30/06/2003
*/

public boolean excluir_agenda( String id ) {
    String sql = "";
    boolean valor = false;

    Statement stmt;

    try {
        sql = "DELETE FROM Agenda WHERE ID = " + String.valueOf(id);
        stmt = con.createStatement ();
        int linhas = stmt.executeUpdate(sql);

        if (linhas > 0) {
            valor = true;
        } else {
            valor = false;
        }

    } catch(SQLException e) {
        e.printStackTrace();
    }
    return valor;
}
%>
<%
String usuario = (String)session.getAttribute("USUARIO");
String id = request.getParameter("ID");

```

```
if ( ( usuario != null ) ) {  
  
    if ( excluir_agenda( id ) ) {  
        out.println("<script>" +  
            "opener.location.reload();" +  
            "self.close();" +  
            "</script>");  
    } else {  
        out.println("<script> " +  
            "self.close(); " +  
            "</script>");  
    }  
  
    } else {  
        out.println("<script>" +  
            "opener.location = 'index.jsp';" +  
            "self.close();" +  
            "</script>");  
    }  
%>
```

Apêndices

1 - Manipulação de Strings

1 - PHP

1 - HTML

htmlspecialchars

```
htmlspecialchars(string);
```

Retorna a string fornecida, substituindo os seguintes caracteres:

```
& para '&amp;';  
" para '&quot;';  
< para '&lt;';  
> para '&gt;';
```

htmlentities

```
htmlentities(string);
```

Funciona de maneira semelhante ao comando anterior, mas de maneira mais completa, pois converte todos os caracteres da string que possuem uma representação especial em html, como por exemplo:

```
º para '&ordm;';  
ª para '&ordf;';  
á para '&aacute;';  
ç para '&ccedil;';
```

nl2br

```
nl2br(string);
```

Retorna a string fornecida substituindo todas as quebras de linha ("\\n") por quebras de linhas em html ("
").

get_meta_tags

```
get_meta_tags(arquivo);
```

Abre um arquivo html e percorre o cabeçalho em busca de "meta" tags, retornando num array todos os valores encontrados.

strip_tags

```
strip_tags(string);
```

Retorna a string fornecida, retirando todas as tags html e/ou PHP encontradas.

urlencode

```
urlencode(string);
```

Retorna a string fornecida, convertida para o formato urlencode. Esta função é útil para passar variáveis para uma próxima página.

urldecode

```
urldecode(string);
```

Funciona de maneira inversa a urlencode, desta vez decodificando a string fornecida do formato urlencode para texto normal.

2 - Arrays

Implode e join

```
implode(separador, array);
```

e

```
join(separador, array);
```

As duas funções são idênticas. Retornam uma string contendo todos os elementos do array fornecido separados pela string também fornecida.

split

```
split(string padrao, string str, int [limite]);
```

Retorna um array contendo partes da string fornecida separadas pelo padrão fornecido, podendo limitar o número de elementos do array.

explode

```
explode(string padrao, string str);
```

Funciona de maneira bastante semelhante à função split, com a diferença que não é possível estabelecer um limite para o número de elementos do array.

3 - Comparação

similar_text

```
similar_text(string str1, string str2, double [porcentagem]);
```

Compara as duas strings fornecidas e retorna o número de caracteres coincidentes. Opcionalmente pode ser fornecida uma variável, passada por referência (ver *tópico sobre funções*), que receberá o valor percentual de igualdade entre as strings. Esta função é *case sensitive*, ou seja, maiúsculas e minúsculas são tratadas como diferentes.

strcasecmp

```
strcasecmp(string str1, string str2);
```

Compara as duas strings e retorna 0 (zero) se forem iguais, um valor maior que zero se `str1 > str2`, e um valor menor que zero se `str1 < str2`. Esta função é *case insensitive*, ou seja, maiúsculas e minúsculas são tratadas como iguais.

strcmp

```
strcmp(string str1, string str2);
```

Funciona de maneira semelhante à função `strcasecmp`, com a diferença que esta é *case sensitive*, ou seja, maiúsculas e minúsculas são tratadas como diferentes.

strstr

```
string strstr(string str1, string str2);
```

e

```
string strchr(string str1, string str2);
```

As duas funções são idênticas. Procura a primeira ocorrência de `str2` em `str1`. Se não encontrar, retorna uma string vazia, e se encontrar retorna todos os caracteres de `str1` a partir desse ponto.

stristr

```
strstr(string str1, string str2);
```

Funciona de maneira semelhante à função `strstr`, com a diferença que esta é *case insensitive*, ou seja, maiúsculas e minúsculas são tratadas como iguais.

strpos

```
strpos(string str1, string str2, int [offset] );
```

Retorna a posição da primeira ocorrência de `str2` em `str1`, ou zero se não houver. O parâmetro opcional `offset` determina a partir de qual caracter de `str1` será efetuada a busca. Mesmo utilizando o `offset`, o valor de retorno é referente ao início de `str1`.

strrpos

```
strrpos(string haystack, char needle);
```

Retorna a posição da última ocorrência de `str2` em `str1`, ou zero se não houver.

4 - Edição**chop**

```
chop(string);
```

Retira espaços e linhas em branco do final da string fornecida.

ltrim

```
ltrim(string);
```

Retira espaços e linhas em branco do final da string fornecida.

trim

```
trim(string);
```

Retira espaços e linhas em branco do início e do final da string fornecida.

strrev

```
strrev(string);
```

Retorna a string fornecida invertida.

strtolower

```
strtolower(string);
```

Retorna a string fornecida com todas as letras minúsculas.

strtoupper

```
strtoupper(string);
```

Retorna a string fornecida com todas as letras maiúsculas.

ucfirst

```
ucfirst(string);
```

Retorna a string fornecida com o primeiro caracter convertido para letra maiúscula.

ucwords

```
ucwords(string);
```

Retorna a string fornecida com todas as palavras iniciadas por letras maiúsculas.

str_replace

```
str_replace(string str1, string str2, string str3);
```

Altera todas as ocorrências de str1 em str3 pela string str2.

5 - Diversos**chr**

```
chr(código ascii);
```

Retorna o caracter correspondente ao código ASCII fornecido.

ord

```
ord(string);
```

Retorna o código ASCII corresponente ao caracter fornecido.

echo

```
echo(string1, string[argn]... );
```

Imprime os argumentos fornecidos.

print

```
print(string);
```

Imprime o argumento fornecido.

strlen

```
int strlen(string);
```

Retorna o tamanho da string fornecida.

2 - ASP**Asc**

Retorna o código Asc da primeira letra da string.

Chr

Retorna o caracter referente ao código ASCII especificado.

Filter

Retorna um array com um subconjunto de um array de string baseado em um critério de filtro pre-estabelecido.

InStr

Esta função permite que se procure um determinado substring em um string, mesmo sem saber se este substring será encontrado. Esta função retornará sempre a posição da primeira ocorrência da busca.

InStrRev

Semelhante a InStr, porém a partir do final.

Lcase

Substitui todas as letras maiúsculas de uma string por letras minúsculas.

Left

Isola os n primeiros caracteres de uma string.

Len

Retorna o numero de carateres de uma string ou os bytes necessários para armazenar uam determinada variável.

Ltrim

Retira todos espaços e branco no inicio de uma string.

Mid

Isola uma substring do meio de uma string.

Replace

Retorna uma string onde uma substring foi substituída por outra, um determinado numero de vezes.

Right

Isola os n últimos caracteres de uma string.

Rtrim

Retira todos os espaços em branco no final de uma expressão.

StrComp

Retorna um valor decorrente do resultado da comparação entre duas strings.

String

Retorna um determinado character um determinado numero de vezes.

StrReverse

Retorna o inverso de uma string determinada.

Trim

Retorna uma determinada string sem espaços em branco no inicio e no final.

UCase

Substitui todas as letras minúsculas de uma determinada string por maiúsculas.

3 - JSP

length()

Retorna o numero de caracteres de uma string:

```
String variavel = "testando";  
int    tamanho  = variavel.length();
```

charAt()

Retorna o caracter na posição especificada pelo índice, sendo que o primeiro caracter é o índice 0.

```
String variável = "testando";  
char    c       = variavel.charAt(1);\\retorna "e"
```

substring(índice inicial, índice final)

retorna uma substring de acordo com os índices iniciais e finais passados, sendo que o primeiro caracter é o índice 0.

```
String variável = "testando";  
String subvar   = variavel.substring(2,4);\\retorna "sta"
```

toUpperCase()

Retorna a string toda em maiúsculas.

```
String variável = "testando";  
String novavar  = variavel.toUpperCase();\\retorna "TESTANDO"
```

toLowerCase()

Retorna a string toda em minúsculas.

```
String variável = "TESTANDO";  
String novavar  = variavel.toLowerCase();\\retorna "testando"
```

trim()

retorna uma string sem espaços no inicio e no fim.

```
String variável = "  TESTANDO  ";  
String novavar  = variavel.trim();\\retorna "TESTANDO"
```

indexOf(String)

retorna o índice de uma substring determinada.

```
String variável = "TESTANDO";  
String novavar  = variavel.indexOf("TA");\\retorna 3
```

lastIndexOf(String)

retorna o índice da ultima ocorrência de uma substring determinada.

```
String variável = "TESTANDO";  
String novavar = variavel.indexOf("T");\\retorna 3
```

equals()

Compara strings checando maiusculas e minúsculas.

```
String variável = "TESTANDO";  
if ( variável.equals("TeStAnDo") ) {\\retorna false  
    comando;  
}
```

equalsIgnoreCase()

Compara strings ignorando maiusculas e minúsculas.

```
String variável = "TESTANDO";  
if ( variável.equals("TeStAnDo") ) {\\retorna true  
    comando;  
}
```

toString()

Retorna uma String apartir de objetos.

```
Cor vermelho = new Cor();  
out.println(vermelho);
```

valueOf()

Retorna uma String a partir de tipos primitivos.

```
String valor = String.valueOf(5);\\retorna "5"
```

2 - Manipulação de Números

1 - PHP

abs

Retorna o valor absoluto de um número.

acos

Retorna o coseno inverso (arco cosseno).

acosh

Retorna o cosseno hiperbólico inverso.

asin

Retorna o seno inverso (arco seno).

asinh

Retorna o seno hiperbólico inverso.

atan2

retorna a tangente inversa de duas variáveis.

atan

Retorna a tangente inversa (arco tangente).

atanh

Retorna a tangente hiperbólica inversa.

base_convert

Converte um número entre bases arbitrariedades.

bindec

Converte de binário para decimal.

ceil

Arredonda frações para cima.

cos

Retorna o coseno.

cosh

Retorna o Coseno hiperbólico.

decbin

Converte um número decimal para binário.

dechex

Converte um número decimal para hexadecimal.

decoct

Converte um número decimal para octal.

deg2rad

Converte o número em graus ao equivalente em radianos.

exp

Calcula o expoente de **e** (Neperiano ou base logarítmica natural)

expm1

Retorna $\exp(\text{numero}) - 1$, computado de forma que é preciso mesmo quando o valor do número é perto de zero.

floor

Arredonda frações para baixo.

fmod

Retorna o módulo de divisões de números de ponto flutuante.

getrandmax

Retorna o maior valor aleatório possível.

hexdec

Converte um número hexadecimal para decimal.

hypot

Retorna a raiz quadrada de $(num1*num1 + num2*num2)$.

is_finite

Retorna **true** se for um número finito legal dentro dos limites permitidos para um número float PHP nesta plataforma.

is_infinite

Retorna **true** se for um número infinito(positivo ou negativo).

is_nan

Retona **true** se for um número.

lcg_value

Gerador congruente linear combinado.

log10

Logaritmo Base 10.

log1p

Retorna o $\log(1 + \text{numero})$, calculado de forma que o valor do número seja próximo de zero.

log

Logaritmo natural

max

Localiza o maior número

min

Localiza o menor número

mt_getrandmax

Retorna o maior valor aleatório possível.

mt_rand

Gerador melhorado de números aleatórios

mt_srand

Semeia o gerador melhorado de números aleatórios

octdec

Converte de octal para decimal.

pi

Obtêm o valor de pi

pow

Retorna a *base* elevada ao expoente;

rad2deg

Converte o número em radianos para o equivalente em graus

rand

Gera um número aleatório

round

Arredonda um número

sin

Seno

sinh

Seno hiperbólico

sqrt

Reatorna a raiz quadrada.

srand

Semeia o gerador de números aletórios

tan

Tangente

Tanh

Tangente hiperbólica

2 - ASP

FormatCurrency

Formata uma expressão como um dado do tipo currency(moeda).

FormatNumber

Formata valores numéricos, indicando o número de casas decimais, o numero de zeros inciais ou finais.

FormatPercent

Converte uma expressão em formato de percentagem, incluindo o caracter %.

Round

Retorna uma numero arredondado para um determinado número de casa decimais.

3 - JSP

intValue

```
public abstract int intValue()
```

Retorna o valor de um número como um tipo **int**.

longValue

```
public abstract long longValue()
```

Retorna o valor de um número como um tipo **long**.

floatValue

```
public abstract float floatValue()
```

Retorna o valor de um número como um tipo **float**.

doubleValue

```
public abstract double doubleValue()
```

Retorna o valor de um número como um tipo **double**.

byteValue

```
public byte byteValue()
```

Retorna o valor de um número como um tipo **byte**.

shortValue

```
public short shortValue()
```

Retorna o valor de um número como um tipo **short**.

3 - Manipulação de Data e Hora

1 - PHP

date(formato)

Retorna uma data de acordo com o formato da string recebida como parâmetro.

Os seguintes caracteres são reconhecidos na estrutura da string:

```
a - "am" ou "pm"
A - "AM" ou "PM"
B - Swatch Internet time
d - dia do mês, 2 dígitos com zeros à esquerda; i.e. "01" até "31"
D - dia da semana, texto, 3 letras; e.g. "Fri"
F - mês, texto, longo; e.g. "January"
g - hora, Forma com 12-horas sem zeros à esquerda; i.e. "1" até "12"
G - hora, Forma com 24-horas sem zeros à esquerda; i.e. "0" até "23"
h - hora, Forma com 12-horas; i.e. "01" até "12"
H - hora, Forma com 24-horas; i.e. "00" até "23"
i - minutos; i.e. "00" até "59"
I (capital i) - "1" Se no horário de verão, senão "0".
j - Dia do mês sem zeros à esquerda; i.e. "1" até "31"
l (minúscula 'l') - dia da semana, texto, longo; e.g. "Friday"
L - booleano se é um ano bissexto; i.e. "0" ou "1"
m - mês; i.e. "01" até "12"
M - mês, texto, 3 letras; e.g. "Jan"
n - mês sem zeros à esquerda; i.e. "1" até "12"
O - Diferença entre o horário de Greenwich em horas; e.g. "+0200"
r - RFC 822 formatted date; e.g. "Thu, 21 Dec 2000 16:01:07 +0200" (adicionada no PHP 4.0.4)
s - segundos; i.e. "00" até "59"
S - Sufixo ordinal para o dia do mês, 2 caracteres; i.e. "st", "nd", "rd" or "th"
t - número de dias do dado mês; i.e. "28" até "31"
T - Timezone setting desta máquina; e.g. "EST" or "MDT"
U - segundos desde a época Unix (January 1 1970 00:00:00 GMT)
w - dia da semana, numérico, i.e. "0" (domingo) até "6" (Sábado)
W - ISO-8601 números de semanas do ano, semana começa na segunda-feira (adicionado no PHP 4.1.0)
Y - ano, 4 dígitos; e.g. "1999"
y - ano, 2 dígitos; e.g. "99"
z - dia do ano; i.e. "0" até "365"
Z - timezone offset em segundos (i.e. "-43200" to "43200"). O offset para as timezones oeste de UTC é sempre negativa, e para as leste de UTC é sempre positiva.
```

2 - ASP

WeekDayName

Retorna o nome do dia da semana, de acordo com o parâmetro passado.

MonthName

Retorna uma string indicando o nome do mês, de acordo com o parâmetro passado.

FormatDateTime

Retorna uma expressão em formato de data e hora.

Date

Data do sistema.

DateAdd

Retorna uma data a qual sofreu uma adição de tempo.

DateDiff

Retorna o intervalo de tempo entre duas datas.

DatePart

Contém um intervalo de tempo de uma data determinada.

Now

Retorna a data e hora do sistema operacional.

4 - Manipulação de Arrays

1 - PHP

1 - Genéricas

array

```
array array(...);
```

É a função que cria um array a partir dos parâmetros fornecidos. É possível fornecer o índice de cada elemento. Esse índice pode ser um valor de qualquer tipo, e não apenas de inteiro. Se o índice não for fornecido o PHP atribui um valor inteiro sequencial, a partir do 0 ou do último índice inteiro explicitado.

range

```
array range(minimo, maximo);
```

A função range cria um array cujos elementos são os inteiros pertencentes ao intervalo fornecido, inclusive. Se o valor do primeiro parâmetro for maior do que o do segundo, a função retorna false (valor vazio).

shuffle

```
shuffle(array);
```

Esta função "embaralha" o array, ou seja, troca as posições dos elementos aleatoriamente e não retorna valor algum.

sizeof

```
sizeof(array);
```

Retorna um valor inteiro contendo o número de elementos de um array. Se for utilizada com uma variável cujo valor não é do tipo array, retorna 1. Se a variável não estiver setada ou for um array vazio, retorna 0.

2 - Navegação

Toda variável do tipo array possui um ponteiro interno indicando o próximo elemento a ser acessado no caso de não ser especificado um índice. As funções seguintes servem para modificar esse ponteiro, permitindo assim percorrer um array para verificar seu conteúdo (chaves e elementos).

reset

```
reset(array);
```

Seta o ponteiro interno para o primeiro elemento do array, e retorna o conteúdo desse elemento.

end

```
end(array);
```

Seta o ponteiro interno para o último elemento do array, e retorna o conteúdo desse elemento.

next

```
next(array);
```

Seta o ponteiro interno para o próximo elemento do array, e retorna o conteúdo desse elemento.

prev

```
prev(array);
```

Seta o ponteiro interno para o elemento anterior do array, e retorna o conteúdo desse elemento. Funciona de maneira inversa a next.

pos

```
pos(array);
```

Retorna o conteúdo do elemento atual do array, indicado pelo ponteiro interno.

key

```
key(array);
```

Funciona de maneira bastante semelhante a pos, mas ao invés de retornar o elemento atual indicado pelo ponteiro interno do array, retorna seu índice.

each

```
each(array);
```

Retorna um array contendo o índice e o elemento atual indicado pelo ponteiro interno do array. o valor de retorno é um array de quatro elementos, cujos índices são 0, 1, "key" e "value". Os elementos de índices 0 e "key" armazenam o índice do valor atual, e os elementos de índices 1 e "value" contém o valor do elemento atual indicado pelo ponteiro.

3 - Ordenação

sort

```
sort(array);
```

A função mais simples de ordenação de arrays. Ordena os elementos de um array em ordem crescente, sem manter os relacionamentos com os índices.

rsort

```
rsort(array);
```

Funciona de maneira inversa à função sort. Ordena os elementos de um array em ordem decrescente, sem manter os relacionamentos com os índices.

asort

```
asort(array);
```

Tem o funcionamento bastante semelhante à função sort. Ordena os elementos de um array em ordem crescente, porém mantém os relacionamentos com os índices.

arsort

```
arsort(array);
```

Funciona de maneira inversa à função asort. Ordena os elementos de um array em ordem decrescente e mantém os relacionamentos dos elementos com os índices.

ksort

```
ksort(array);
```

Função de ordenação baseada nos índices. Ordena os elementos de um array de acordo com seus índices, em ordem crescente, mantendo os relacionamentos.

usort

```
usort(array, function compara);
```

Esta é uma função que utiliza outra função como parâmetro. Ordena os elementos de um array sem manter os relacionamentos com os índices, e utiliza para efeito de comparação uma função definida pelo usuário, que deve comparar dois elementos do array e retornar 0, 1 ou -1, de acordo com qualquer critério estabelecido pelo usuário.

uasort

```
uasort(array, function compara);
```

Esta função também utiliza outra função como parâmetro. Ordena os elementos de um array e mantém os relacionamentos com os índices, utilizando para efeito de comparação uma função definida pelo usuário, que deve comparar dois elementos do array e retornar 0, 1 ou -1, de acordo com qualquer critério estabelecido pelo usuário.

uksort

```
uksort(array, function compara);
```

Esta função ordena o array através dos índices, mantendo os relacionamentos com os elementos., e utiliza para efeito de comparação uma função definida pelo usuário, que deve comparar dois índices do array e retornar 0, 1 ou -1, de acordo com qualquer critério estabelecido pelo usuário.

5 - Templates

A principal vantagem no uso templates para gerar páginas dinâmicas se dá devido a separação do código HTML(layout) dos scripts PHP, ASP ou JSP(lógica), pois num grande projeto normalmente quem desenvolve o layout(HTML) não é a mesma pessoa que desenvolve a lógica(Scripts e Bancos de Dados) e também no caso de necessidade de manutenção do layout, por exemplo, não se torna necessário que o desenvolvedor se envolva, visto que o HTML e scripts estão separados.

Existem vários sistemas de **templates** para PHP, ASP e JSP na web, mas todos acabam necessitando que certas **tags** sejam inseridas no documento HTML e que certas bibliotecas ou plug-ins sejam instalados no servidor, o que, na minha opinião, faz com que seu código fique dependente de terceiros e de uma certa mistura de lógica também.

Mas como usar templates sem necessitar de terceiros?

1 - Todas minhas **classes** e **funções** armazenam seu valor de retorno em uma variável;

2 - Uso uma simples função que lê um arquivo HTML e retorna o código-fonte da mesma:

Em PHP seria assim:

```
<?
function read_file($strfile) {
    if(!file_exists($strfile)) return;
    $thisfile = file($strfile);
    while(list($line,$value) = each($thisfile)) {
        $value = ereg_replace("\r","", $value);
        $value = ereg_replace("\n","", $value);
        $result .= "$value\r\n";
    }
    return $result;
}
```

?>

Em ASP seria assim:

```
<%
'Função para ler os arquivos
function read_file( arquivo )
    Dim ObjAbreArq, ObjFso, Var_Caminho, Linha
    Var_Caminho = Server.MapPath (arquivo)
    Set ObjFso = Server.CreateObject("Scripting.FileSystemObject")
    Set ObjAbreArq = ObjFso.OpenTextFile(Var_Caminho, 1, True, False)
    linha = ""
    Do While Not ObjAbreArq.AtEndOfStream
        linha = linha & ObjAbreArq.ReadLine & Chr(13) + Chr(10)
    Loop
    ObjAbreArq.Close
    Set ObjAbreArq = Nothing
    read_file = linha
end function
%>
```

Em JSP seria assim:

```
<%!
public String get_Source (String pagina) {
    DataInputStream dis;
    String source = "";
    try {
        URL u = new URL (pagina);
        dis = new DataInputStream (u.openConnection().getInputStream());
        String s;
        while ( ( s = dis.readLine() ) != null ) {
            source += s;
        }

        } catch (Exception e){
            out.println(e);
        }
        return source;
    }
}%>
```

3 - Uso funções nativas de PHP/ASP/JSP para substituir no código-fonte, os comentários HTML pelo conteúdo;

Em PHP seria assim:

```
str_replace("<!--menu-->",$menu,$template)
```

Em ASP seria assim:

```
replace(template, "<!--menu-->", menu)
```

Em JSP seria assim:

```
template.replaceAll("<!--menu-->", menu);
```

Parece simples, e é.

Agora vejamos tudo junto, criando uma página HTML que à esquerda terá um menu dinâmico e a direita terá as notícias.

O primeiro passo é criamos nosso arquivo HTML, que deve ser salvo com o nome de **template.html** e será usado como nosso primeiro template:

```
<html>
<head><title>Primeiro Template</title></head>
<body>
<table width=100% border=1>
<tr>
<td width=20%><!--menu--></td>
<td width=80%><!--noticias--></td>
</tr>
</table>
</body>
</html>
```

Agora que temos o template vamos usá-lo com o script, criando antes o arquivo abaixo que eu uso como repositório de todas funções, variáveis e constantes comuns a todos os scripts, salve-o como **funções.php,asp e jsp respectivamente**, mais tarde ele será um **include**:

Em PHP seria assim:

```
<?
function read_file($strfile) {
    if(!file_exists($strfile)) return;
    $thisfile = file($strfile);
    while(list($line,$value) = each($thisfile)) {
        $value = ereg_replace("\r","", $value);
        $value = ereg_replace("\n","", $value);
        $result .= "$value\r\n";
    }
    return $result;
}

//LISTA DE TEMPLATES
$home = "template.html";
?>
```

Em ASP seria assim:

```
<%
'Função para ler os arquivos
function read_file( arquivo )
    Dim ObjAbreArq, ObjFso, Var_Caminho,Linha
    Var_Caminho = Server.MapPath (arquivo)
    Set ObjFso = Server.CreateObject("Scripting.FileSystemObject")
    Set ObjAbreArq = ObjFso.OpenTextFile(Var_Caminho, 1, True, False)
```

```

        linha = ""
        Do While Not ObjAbreArq.AtEndOfStream
        linha = linha & ObjAbreArq.ReadLine & Chr(13) + Chr(10)
        Loop
        ObjAbreArq.Close
        Set ObjAbreArq = Nothing
        read_file = linha
    end function

'LISTA DE TEMPLATES
Dim home = "template.html"
%>

```

Em JSP seria assim:

```

<%!
public String read_file(String pagina) {
    DataInputStream dis;
    String source = "";
    try {
        URL u = new URL (pagina);
        dis = new DataInputStream (u.openConnection().getInputStream());
        String s;
        while ( ( s = dis.readLine() ) != null ) {
            source += s;
        }

        } catch (Exception e){
            out.println(e);
        }
        return source;
    }

//LISTA DE TEMPLATES
String home = "template.html";
%>

```

Então, agora temos nosso template e nosso arquivo de funções comuns, finalmente implementamos o uso do nosso template no script principal:

Em PHP seria assim:

```

<?
//Fazendo o include do arquivo de funções
include("funcoes.php");

//lendo o arquivo de template
$template = read_file($home);

/* Carregando as variaveis que deverão conter o conteúdo dinâmico
   sendo que estas informações podem vir do banco de dados, arquivos, etc.
*/
$menu      = "principal<br>noticias<br>chat<br>enquete";
$noticias  = "Noticia um<br>Noticia 2<br>Noticia 3";

//Substituindo os comentários do template pelos conteúdos das variáveis
$template = str_replace("<!--menu-->", $menu, $template);

```

```
$template = str_replace("<!--noticias-->",$noticias,$template);  
  
//Imprimindo a página pronta  
print $template;  
?>
```

Em ASP seria assim:

```
<!-- #include file="funcoes.asp" -->  
<%  
'lendo o arquivo de template  
template = read_file(home)  
  
'Carregando as variaveis que deverão conter o conteúdo dinâmico  
'sendo que estas informações podem vir do banco de dados, arquivos, etc.  
menu      = "principal<br>noticias<br>chat<br>enquete"  
noticias = "Noticia um<br>Noticia 2<br>Noticia 3"  
  
'Substituindo os comentários do template pelos conteúdos das variáveis  
template = replace(template, "<!--menu-->", menu)  
template = replace(template, "<!--noticias-->", noticias)  
  
'Imprimindo a página pronta  
response.write(template)  
%>
```

Em JSP seria assim:

```
<%@ page import = "java.lang.*,java.io.*,java.util.*" %>  
<%@ include file="funcoes.jsp" %>  
<%  
//lendo o arquivo de template  
String template = read_file(home);  
  
//Carregando as variaveis que deverão conter o conteúdo dinâmico  
//sendo que estas informações podem vir do banco de dados, arquivos, etc.  
String menu      = "principal<br>noticias<br>chat<br>enquete";  
String noticias = "Noticia um<br>Noticia 2<br>Noticia 3";  
  
//Substituindo os comentários do template pelos conteúdos das variáveis  
template.replaceAll("<!--menu-->", menu);  
template.replaceAll("<!--noticias-->", noticias);  
  
//Imprimindo a página pronta  
out.println(template);
```

Pronto, este script mandará para o browser uma página HTML com os comentários substituídos pelo conteúdo dinâmico que poderá vir de um banco de dados, de um arquivo texto, um arquivo XML, etc.

6 - Instalação dos Ambientes

1 - Ambiente Java/PHP

A extensão Java para PHP é uma ferramenta extremamente excitante. Utilizando este módulo é possível estender ainda mais as capacidades do PHP através do poder das classes Java. Veremos a partir daqui como proceder a instalação deste módulo assim como alguns exemplos.

1 - Instalação em Ambiente Windows

As seguintes configurações foram testadas com Servidor Web Apache 1.3.12(atualmente está na versão 2.0.44), interpretador PHP 4.0.3(atualmente está na versão 4.3.3) e JDK 1.2.2. Estas configurações também foram testadas com versões mais antigas do JDK e outros Webservers, como Microsoft (PWS and IIS), em Plataformas Windows 95, Windows 98, NT4 e Windows 2000.

Instalando o JDK:

Esta parte é muito simples, visto que o JDK instala-se sem muitas perguntas. Será necessário checar suas variáveis de ambiente(autoexec.bat no Windows 9x e Sistema dentro do Painel de Controle no NT e 2000) e certificar-se que o diretório jdk1.x.x\bin está no seu PATH. Isto fará a compilação de suas classes Java facilmente.

No Win 9x adicione ao autoexec.bat:

```
PATH=%PATH%;C:\jdk1.2.2\bin
```

No Win NT adicione no fim da variavel de ambiente PATH:

```
;C:\jdk1.2.2\bin
```

Modificando seu PHP.INI:

Você deve adicionar algo similar a isto no seu PHP.INI, na seção [java]:

```
[java]
extension=php_java.dll
java.library.path=c:\web\php4\extensions\
java.class.path="c:\web\php4\extensions\jdk1.2.2\php_java.jar;c:\myclasses"
java.home = c:\jre1.3.1
java.library = c:\jre1.3.1\bin\hotspot\jvm.dll
```

Normalmente as pessoas colocam a diretiva **extension=php_java.dll** com as demais extensões, mas funciona também se for logo colocado abaixo da seção [java] do PHP.INI.

A diretiva **java.library.path** deve ser setada com a localização da **php_java.dll**, e a diretiva **java.class.path** deve ser setada com a localização de **php_java.jar**.

A diretiva **java.class.path** deve também incluir os **path's** para as demais classes que deseja usar.

Testando sua Instalação:

Criar um arquivo PHP similar a este:

```
<?
$system = new Java("java.lang.System");
print "Java version=".$system->getProperty("java.version")." <br>\n";
print "Java vendor=".$system->getProperty("java.vendor")." <p>\n\n";
print "OS=".$system->getProperty("os.name")." ".
    $system->getProperty("os.version")." on ".
    $system->getProperty("os.arch")." <br>\n";
$formatter = new Java("java.text.SimpleDateFormat","EEEE, MMMM dd, yyyy 'at' h:mm:ss a zzzz");
print $formatter->format(new Java("java.util.Date"))."\n";
?>
```

Se tudo estiver instalado e configurado corretamente a saída deverá ser algo do tipo:

```
Java version=1.2.2
Java vendor=Sun Microsystems Inc.
OS=Windows 95 4.10 on x86
Wednesday, October 18, 2000 at 10:22:45 AM China Standard Time
```

Um pequeno exemplo para teste, mas mostra como acessar classes Java disponíveis. Se o exemplo acima funcionar, você teve sucesso na sua instalação da extensão Java.

Criando e Usando suas próprias Classes Java:

Criar suas próprias classes Java é muito fácil. Crie um novo arquivo texto chamado **phptest.java**.

Este arquivo deve ser salvo na pasta que você definiu no seu **java.class.path**,. Neste arquivo escreva o seguinte:

```
public class phptest{
    public String valor;

    public String test(String str) {
        if(str.equals("")) {
            str = "A string está vazia. ";
        }
        return str;
    }

    public String retornavalor() {
        return "valor = " + valor;
    }
}
```

Uma vez criado este arquivo, você deve compilá-lo com **javac phptest.java** na linha de comando. Isto dependerá da perfeita configuração da variável de ambiente PATH com a pasta java/bin.

Para testar a sua nova classe Java com o PHP, crie um novo arquivo PHP chamado phptest.php. Ele deve conter o seguinte:

```
<?
    $myjava = new Java("phptest");
    print $myjava->test("Hello World") ;
    $myjava->valor = "Este é meu valor...";
    print "Eu setei o valor para: <b>". $myjava->valor "</b>";
    print "Meu método Java retorna: <b>" . $myjava->retornavalor() "</b>";
?>
```

java.lang.ClassNotFoundException

Se você receber a mensagem de erro: **Warning: java.lang.ClassNotFoundException**, significa simplesmente que seu arquivo **phptest.class** não está salvo no **java.class.path** especificado em seu PHP.INI.

Um importante fato a lembrar é que a linguagem Java é fortemente tipada, mas PHP não. Isto pode causar problemas quando o Java espera uma **String** e recebe um **Integer**, ou vice-versa. Experimente substituir a linha:

```
$myjava->valor = " Este é meu valor...";  
por
```

```
$myjava->valor = 12345678;
```

Você deve fazer o **casting** (conversão explícita) de suas variáveis, para corrigir o tipo, antes de passá-las para o Java.

```
$myjava->valor = (string)12345678;  
ou
```

```
$myjava->valor = "12345678";
```

O exemplo `phptest.java` é o mais simples exemplo de como você pode criar suas próprias classes Java e fazer o PHP acessá-las.

2 - Ambiente PHP no Windows

1 - Apache

O servidor web que será utilizado é o Apache, que está disponível para download em:

```
http://httpd.apache.org/
```

A instalação do Apache é bastante simples, similar a qualquer aplicação windows. A única restrição é que o **winsock2** deve estar instalado no sistema. Se não estiver, o download pode ser feito em:

```
http://www.microsoft.com/windows95/downloads/contents/wuadmintools/s\_wunetworkingtools/w95sockets2/
```

Depois de instalado, é necessário fazer a configuração do servidor, através do arquivo **httpd.conf**. Todas as configurações estão comentadas. O mínimo que deve ser configurado é o diretório onde os documentos estarão, através da opção **DocumentRoot**. Basta procurar a opção e escrever o nome do diretório em seguida, como no exemplo:

```
DocumentRoot "C:\teste\"
```

Uma outra configuração básica é a **DirectoryIndex**, que informa ao servidor quais os arquivos serão exibidos automaticamente como índice do diretório. É isso que faz com que ao digitar, por exemplo, "www.naredemundial.com.br", o servidor saiba qual dos arquivos do diretório deve ser exibido. Abaixo temos um exemplo da utilização do **DirectoryIndex**:

```
DirectoryIndex index.html index.htm index.php
```

Feito isso, crie um arquivo com um dos nomes definidos como índice e coloque no diretório definido como root. Execute o servidor Apache e tente acessar o endereço "http://localhost" pelo browser. Se a página for exibida, é porque o servidor foi instalado corretamente.

2 - PHP

O PHP pode ser conseguido em "www.php.net", e sua instalação também é simples. Basta descompactar os arquivos para o diretório "c:\php" e editar o arquivo de configuração.

O arquivo "php.ini-dist" deve ser copiado para o diretório do windows (geralmente c:\windows ou c:\winnt) com o nome php.ini. Depois de copiado, o arquivo pode ser editado, bastando modificar a linha `extension_dir`, que deve conter o diretório onde estão os módulos (c:\php). Veja o exemplo:

```
extension_dir = c:\php
```

Feito isso, podemos partir para a configuração do servidor Apache, necessária para que este reconheça o PHP. Editando novamente o arquivo `httpd.conf`, as linhas a seguir devem ser adicionadas no final do arquivo:

```
ScriptAlias /php/ "c:/php/"
AddType application/x-httpd-php .php
Action application/x-httpd-php "/php/php.exe"
```

A primeira linha define o diretório onde está o PHP. A segunda cria um "tipo" para o PHP, definido que todos os arquivos com a extensão ".php" devem passar pelo interpretador PHP. A terceira linha define o executável do interpretador PHP.

Depois de salvar o arquivo, podemos testar se a instalação do PHP foi bem sucedida. A melhor maneira é criar um arquivo chamado teste.php e salvar no diretório raiz do servidor Apache. O arquivo deve conter a seguinte informação:

```
<?
phpinfo();
?>
```

Acessando a página através do servidor (http://localhost/teste.php), devemos ter como resultado uma listagem de todas as configurações do PHP:

3 - MySQL

O banco de dados MySQL pode ser feito o download em:

<http://www.mysql.com/download.html>

Sua instalação também é bastante simples, também no modelos de instalação de qualquer aplicativo para Windows.

As configurações necessárias são relativas a segurança, e exigem um conhecimento mais avançado de administração de servidores. Como essa instalação destina-se apenas a praticar o PHP, não é necessário fazer muitas alterações na segurança, bastando apenas saber como adicionar usuários.

Para isto, basta utilizar o comando GRANT, que tem a seguinte sintaxe:

```
GRANT privilegio [(lista_colunas)]
[, privilegio [(colunas)] ...]
ON {tabela | * | *.* | db.*}
TO usuario [IDENTIFIED BY 'senha']
[, usuario [IDENTIFIED BY 'senha'] ...]
[WITH GRANT OPTION]
```

Onde privilégio é uma das palavras reservadas listadas a seguir:

ALL PRIVILEGES	INDEX	SHUTDOWN	DROP
FILE	SELECT	DELETE	REFERENCES
RELOAD	CREATE	PROCESS	USAGE
ALTER	INSERT	UPDATE	

Cada palavra representa um tipo de acesso à(s) coluna(s), tabela(s) ou base(s) de dados listadas logo depois da cláusula ON.

Usuário deve conter o nome do usuário (login) e o host permitido (ex.: teste@localhost).

Abaixo temos um exemplo da utilização do comando grant:

```
GRANT SELECT, INSERT, UPDATE ON *  
TO teste@localhost IDENTIFIED BY "senhateste";
```

O exemplo cria o usuário "teste", com a senha "senhateste", que só pode acessar da mesma máquina onde está o servidor (localhost), e só pode utilizar os comandos select, insert e update.

Também é possível adicionar usuários utilizando o comando INSERT, pra alterar diretamente na tabela de privilégios, que é a tabela "user" da base de dados "mysql", que possui os campos para definir nome de usuário, host, senha, e permissões específicas.

3 - Ambiente PHP no Linux

1 - Servidor Apache e PHP

Faça os downloads dos programas de instalação, respectivamente, em:

```
http://httpd.apache.org/
```

e

```
http://www.php.net/downloads.php
```

Depois siga os passos abaixo:

Vá para o diretório em que serão descompactados os arquivos (/usr/src):

```
# cd /usr/src
```

Descompacte o Apache e o PHP:

```
# tar xvzf apache_1_3.x.tar.gz  
# tar xvzf php-4.x.x.tar.gz
```

Vá para o diretório do Apache e configure-o

```
# cd apache_1.3.x
# ./configure --prefix=/www
```

Vá agora para o diretório do PHP e proceda a instalação

```
# cd ../php-x.x.x
# ./configure --with-mysql --with-apache=../apache_1.3.x --enable-track-vars --enable-trans-sid
```

Compile e instale o PHP

```
# make
# make install
```

Retorne ao diretório do Apache e configure-o para o PHP, executando a compilação e a instalação do Apache com PHP:

```
# cd ../apache_1.3.x
# ./configure --activate-module=src/modules/php4/libphp4.a
# make
# make install
```

Volte ao diretório do PHP e copie o arquivo php.ini:

```
# cd ../php-x.x.x
# cp php.ini-dist /usr/local/lib/php.ini
```

Vá até o diretório de configuração do Apache (/www/conf) e altere o arquivo httpd.conf, inserindo a seguinte linha:

```
AddType application/x-httpd-php .php
```

Por fim, inicialize o Apache

```
# cd /www/bin
# ./apachectl start
```

2 - MySQL

Faça os download do MySQL em:

```
http://www.mysql.com
```

Crie um grupo de usuários chamado MySQL

```
# groupadd mysql
```

Adicione um novo usuário MySQL relacionado ao grupo MySQL

```
# adduser -g mysql mysql
```

Vá para o diretório base de instalação (em geral /usr/local)

```
# cd /usr/local
```

Descompacte e extraia os fontes do MySQL

```
# tar xvfz /diretorio/mysql-VERSAO-OS.tar.gz
```

Em que <diretório> é o local onde está o arquivo tar e VERSAO-OS é a versão e o sistema operacional dos fontes.

Altere o nome do diretório de mysql-VERSAO-OS para mysql

```
# mv mysql-VERSAO-OS mysql
```

Vá para o diretório do MySQL

```
# cd mysql
```

Execute o script de instalação do Banco de dados

```
# scripts/mysql_install_db
```

Altere os donos e os grupos dos diretórios do MySQL

```
# chown -R root /usr/local/mysql
# chown -R mysql /usr/local/mysql/var
# chown -R mysql /usr/local/mysql
# chown -R root /usr/local/mysql/bin/
```

Inicialize o MySQL

```
# bin/safe_mysqld -user=mysql &
```

O Site abaixo possui instruções mais detalhadas para instalar PHP, Apache, Mysql, Oracle e muito mais no ambiente Linux:

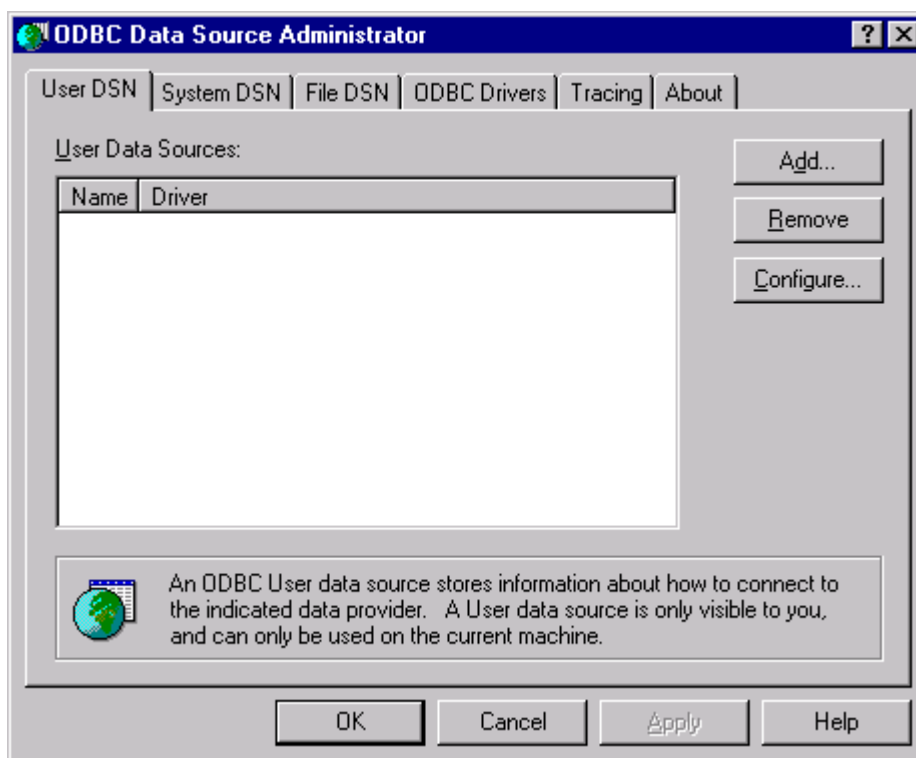
```
http://www.e-gineer.com/instructions/
```

7 - Instalação e Configuração do Ambiente ODBC

Para criar uma conexão ODBC, Abra o **Painel de Controle**, depois clique no ícone **ODBC**:



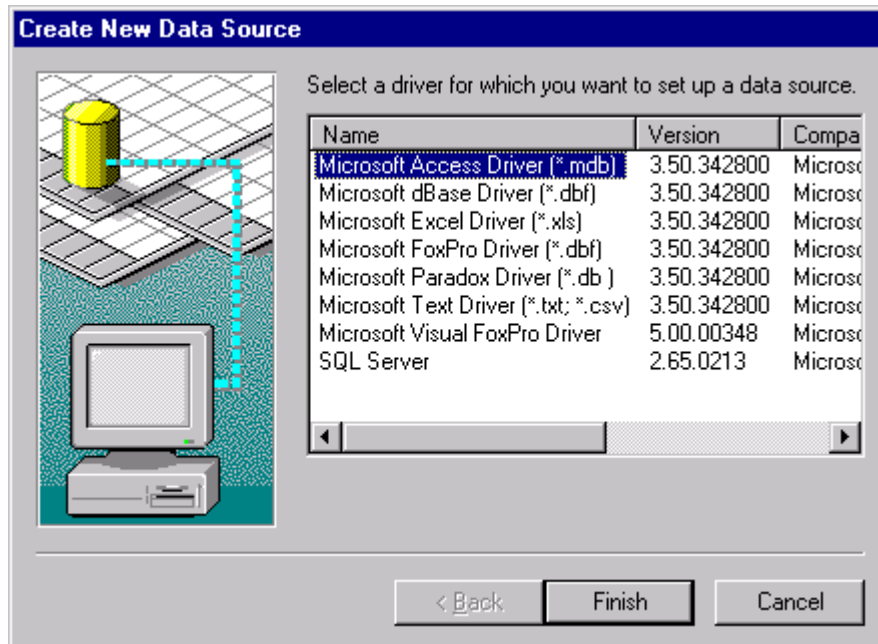
Depois aparecerá uma tela semelhante a esta:



Selecione o marcador **System DSN** (DSN de Sistema).

Selecione o botão **Add** (Adicionar).

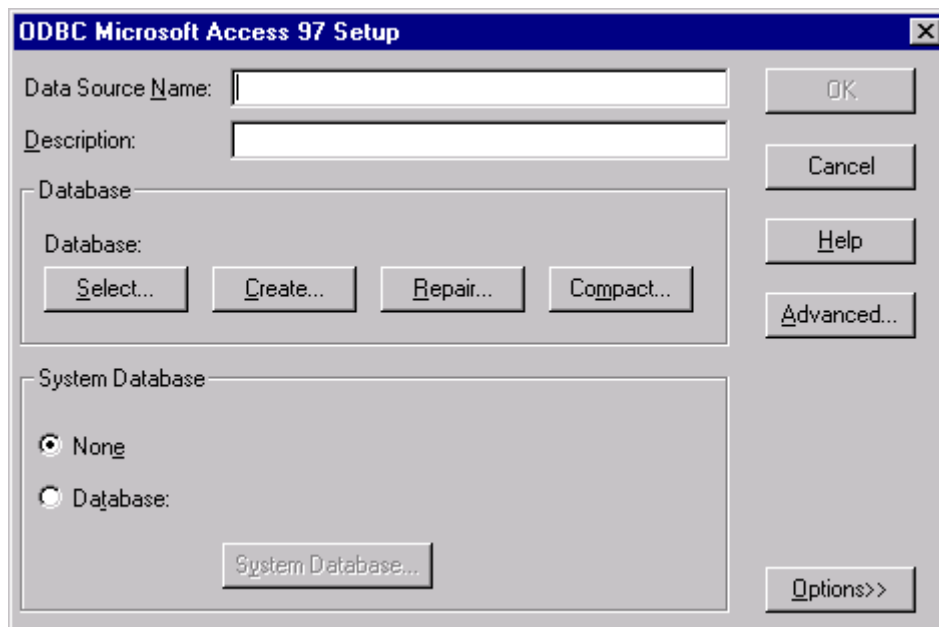
Você terá uma tela como esta:



Selecione o **Microsoft Access Driver** para Microsoft Access, o **SQL Server** para Microsoft SQL SERVER, e assim por diante .

Clique no botão **finish**(Finalizar) .

Você terá uma tela como esta:



No campo **Data Source Name**(Nome da Fonte de Dados), digite um nome para seu **Data Source**(Fonte de Dados), este será o nome que você chamará sua Base de Dados.

No campo **Description**(Descrição) digite uma breve descrição sobre sua Base de Dados.

Finalmente, clique no botão **Select**(Selecionar). A partir deste menu selecione a Base de Dados Access que você quer acessar através da sua Fonte ODBC.

Pronto, está feito!